

COMPLETE EDITION · 2026

Mobile *Speech-to-Text* Engineering Guide

The definitive 2026 reference for building production-grade STT systems on Android, iOS, edge, and cloud. From first microphone sample to post-processed transcript — covers the complete signal → inference → language pipeline with verified working code, benchmark data, and every tool in the modern ASR stack.

2026 EDITION

SIGNAL → TEXT

33 SECTIONS

ALL LEVELS

MOONSHINE V2 · WHISPER

MADE IN JURGISTAN

● **BEGINNER** Audio fundamentals, API setup, cloud integration

● **INTERMEDIATE** VAD, streaming, on-device model deployment

● **EXPERT** Custom decoders, JNI bridges, quantization, real-time tuning

SECTION 01

About This Guide

● **BEGINNER**

Scope, prerequisites, reading paths, and the philosophy behind this edition

Automatic Speech Recognition (ASR), also called Speech-to-Text (STT), is the technology that converts spoken audio into written text. A production-grade STT system is a **multi-layer signal processing and machine learning pipeline** that must handle acoustic noise, speaker variation, language modeling, streaming latency, battery constraints, and privacy requirements — simultaneously. This guide is about building that pipeline, not calling an API.

The difference between `AssemblyAI.transcribe(file)` and understanding what happens inside is the difference between *using* speech recognition and *engineering* it. When you're building a keyboard with voice input, an offline dictation app, a real-time voice agent, or anything that must work without a network connection, you need the full picture.

This 2026 edition reflects the decisive industry shift toward **on-device inference**. Models like Moonshine v2, whisper.cpp, and Sherpa-ONNX have matured to the point where sub-200ms latency is achievable on a mid-range Android phone without cloud round-trips. Every section has been written and code-verified against this new reality.

Prerequisites

- **Kotlin basics** — Android code is Kotlin throughout. Java equivalents exist but are not shown.
- **Python familiarity** — Python is used for server-side, tooling, and model evaluation code.
- **Basic signal intuition** — You don't need a DSP degree, but understanding sample rate, bit depth, and mono vs. stereo saves hours of debugging. [Section 03](#) covers these.
- **No ML expertise required** — Model internals are explained from scratch in each deep-dive section.

Reading Paths by Role



Android Developer

Start: [Section 04 Capture](#) → [Section 06 VAD](#) → [Section 16 Android Stack](#) → [Section 20 whisper.cpp](#)



Keyboard / IME Engineer

Start: [Section 02 Pipeline](#) → [Section 16 Android](#) → [Section 24 VoiceKeyboard](#) → [Section 25 Perf](#)



ML Engineer

Start: [Section 09 Models](#) → [Section 10 Whisper](#) → [Section 11 Moonshine](#) → [Section 23 Benchmarks](#)



Backend / Cloud Engineer

Start: [Section 07 Streaming](#) → [Section 13 LM](#) → [Section 18 Cloud](#) → [Section 15 LLM Layer](#)

Why "Just Call the API" Is Sometimes Wrong

Cloud APIs — [Deepgram](https://www.deeagram.com/) (<https://www.deeagram.com/>), [AssemblyAI](https://www.assemblyai.com/) (<https://www.assemblyai.com/>), [Google Cloud Speech-to-Text](https://cloud.google.com/speech-to-text) (<https://cloud.google.com/speech-to-text>) — are legitimate engineering choices for many products. This guide exists for when they are not:

- **Real-time voice agents** — Cloud round-trips add 200–800ms. On-device avoids this entirely.
 - **Offline applications** — Field workers, embedded devices, remote deployments with no network.
 - **Privacy-sensitive deployments** — Medical, legal, or personal audio that must never leave the device.
 - **Keyboard / IME integration** — System-level voice input can't depend on internet availability.
 - **Cost at scale** — A cloud STT API at \$0.006/minute costs \$360/month for a single user transcribing 8 hours/day.
-

SECTION 02

Pipeline Architecture Overview

● BEGINNER

The complete 10-stage signal → inference → language pipeline. The mental model for every debugging session and optimization decision you will make.

A production STT system is not a single model. It is a **ten-stage pipeline** where each stage has its own tooling, failure modes, and optimization surface. End-to-end models like Moonshine and whisper.cpp collapse stages 5–7, but they don't eliminate the others.



Stage Responsibilities

#	STAGE	INPUT → OUTPUT	KEY TOOLS (2026)	SECTION
1	Audio Capture	Mic → 16kHz mono PCM	AudioRecord, AVAudioEngine, PortAudio	<u>Section 04</u>

#	STAGE	INPUT → OUTPUT	KEY TOOLS (2026)	SECTION
2	Pre-Processing	Raw PCM → Clean PCM	RNNoise, WebRTC APM, Krisp SDK	Section 05
3	VAD	Clean stream → Speech segments	Silero VAD v6, Moonshine VAD, WebRTC VAD	Section 06
4	Streaming / Chunking	Segments → Buffered chunks	LiveKit, WebSocket, gRPC, ring buffer	Section 07
5	Feature Extraction	Chunks → Log-mel tensors	torchaudio, librosa, whisper.cpp mel	Section 08
6	Acoustic Model	Features → Token probabilities	Whisper, Moonshine, Parakeet, Wav2Vec2	Section 09 – Section 11
7	Decoder	Token probs → Word sequences	CTC greedy, beam search, Flashlight	Section 12
8	Language Model	Hypotheses → Re-scored output	KenLM, SentencePiece, n-gram	Section 13
9	Post-Processing	Raw text → Formatted text	DeepMultilingualPunc, spaCy, custom rules	Section 14
10	LLM Formatting	Transcript → Final output	Claude Haiku 4.5, Qwen3, Gemma 3	Section 15



Modern shortcut: end-to-end models collapse stages 5–8

Moonshine Voice, whisper.cpp, and Sherpa-ONNX bundle feature extraction, acoustic modeling, decoding, and sometimes language model bias into a single deployable artifact. You still need to manage stages 1–4 and 9–10. Understanding all ten stages lets you debug failures, optimize the true bottleneck, and know what "batteries included" frameworks are actually doing.

Core Concepts & Terminology ● BEGINNER

WER, RTF, CTC, RNN-T, MFCC, log-mel — demystified. The vocabulary you need to read papers, documentation, and benchmark cards without confusion.

Before diving into any STT implementation, you need the vocabulary. This section defines every metric, acronym, and architecture term you will encounter in research papers, model cards, and API documentation — from WER and RTF to CTC, RNN-T, and attention-based decoders.

Accuracy Metrics

Word Error Rate (WER) is the standard accuracy metric for STT. It measures the minimum edit distance (Levenshtein distance) between the model output (hypothesis) and the ground truth (reference), normalized by reference length:

FORMULA

$$\text{WER} = (\text{Substitutions} + \text{Deletions} + \text{Insertions}) / \text{Total Reference Words} \times 100\%$$

Example:

Reference: "the quick brown fox jumps" (5 words)

Hypothesis: "the quick fox jumps" (4 words — "brown" deleted)

WER = 1 deletion / 5 words = 20%

Lower is better. A WER of 5–7% is excellent for clear conversational English. Human transcriptionists achieve ~5%. Context matters enormously: 10% on studio audio is poor; 10% on factory-floor noise is outstanding.

Character Error Rate (CER) is similar but operates on characters, not words. Better for logographic languages (Chinese, Japanese) where "word" boundaries are ambiguous.

Speed Metrics

Real-Time Factor (RTF) measures processing time relative to audio duration:

$RTF = \text{Processing Time} / \text{Audio Duration}$

$RTF < 0.1 \rightarrow 10\times$ faster than real-time (excellent for streaming)

$RTF = 0.3 \rightarrow$ target threshold for real-time voice agents

$RTF = 1.0 \rightarrow$ exactly real-time (marginal for live use)

$RTF > 1.0 \rightarrow$ slower than real-time (batch only)

RTFx is the inverse: $RTFx = 1/RTF$. Higher = faster. NVIDIA Parakeet TDT achieves $RTFx > 2000$ on an A100 GPU. Moonshine Tiny achieves $RTFx \approx 20\text{--}40$ on a Pixel 7.

Architecture Patterns

CTC (Connectionist Temporal Classification) is a loss function and decoding strategy for sequence models. It enables training without explicit audio-to-phoneme alignment, outputting per-frame token probabilities that collapse blanks and repetitions. CTC models are fast for streaming because every frame can produce output immediately — there's no need to wait for a full sequence.

Encoder-Decoder (Seq2Seq) is the dominant architecture for high-accuracy offline models (Whisper, Moonshine, Canary). The encoder embeds audio features into a latent space; the decoder autoregressively generates tokens conditioned on the encoded representation and previously generated tokens. Excellent accuracy but inherently non-streaming — the decoder needs the full encoded context.

RNN-Transducer (RNN-T / Transducer) is used by NVIDIA Parakeet and Google's production ASR. Combines a streaming encoder, a prediction network (language model branch), and a joint network. True streaming — outputs tokens as they're generated without waiting for end-of-utterance. The best architecture for latency-critical production deployments.

Ergodic Streaming Encoder (Moonshine v2) replaces full attention with sliding-window self-attention, achieving bounded TTFT (Time to First Token) independent of utterance length. This eliminates the fundamental latency limitation of encoder-decoder models for streaming use cases. See [Section 11 Moonshine Deep Dive](#).

The Golden Audio Standard

16kHz Sample Rate *16,000 samples/sec*

Mono Channels *1 channel — always*

16-bit Bit Depth *Signed PCM*

Float32 Model Input *Normalized [-1.0, 1.0]*

25ms Frame Size *400 samples @ 16kHz*

10ms Frame Stride *160 samples overlap*



Wrong sample rate = silent failure

If you feed 44.1kHz or 48kHz audio into a 16kHz model, it will not error. It will produce plausible-sounding but completely wrong transcriptions. Always verify your capture pipeline with

`np.max(np.abs(audio))` (should be ≈ 0.1 – 0.9 for speech) and confirm your sample rate with `audioRecord.sampleRate` before debugging the model.

Feature Representations

Log-Mel Spectrogram is the input format for all modern end-to-end models (Whisper, Moonshine, Canary). Process: Short-Time Fourier Transform (STFT) → map frequency bins to mel scale → take log → normalize. Whisper uses 80 mel bins over a 25ms window with 10ms stride, producing an 80×3000 tensor for 30 seconds of audio.

MFCC (Mel-Frequency Cepstral Coefficients) is the traditional representation used by Kaldi, Vosk, and older systems. More compact than spectrograms but loses some information that modern models benefit from. Still relevant for embedded deployments with very tight memory budgets.

Raw Waveform — Moonshine's encoder directly convolves raw 16kHz samples without explicit feature extraction, simplifying the pipeline and enabling learnable feature representations.

Audio Capture Layer • BEGINNER

Getting low-latency, correctly formatted audio from the microphone. The foundation of every STT pipeline — get this wrong and nothing downstream saves you.

AudioRecord is Android's low-level microphone API and the correct choice for STT. Do not use `MediaRecorder` — it compresses to AAC/MP4, adds encoder latency, and doesn't give you raw PCM. Do not use `MediaRecorder.AudioSource.VOICE_RECOGNITION` unless you want Android's built-in AGC applied invisibly — use `MIC` and handle your own signal chain.

Manifest & Runtime Permission

```

<!-- AndroidManifest.xml -->
<uses-permission android:name="android.permission.RECORD_AUDIO" />
<uses-feature
    android:name="android.hardware.microphone"
    android:required="true" />
  
```

XML

```

// Runtime permission check – required since API 23
fun hasMicPermission(context: Context): Boolean =
    ContextCompat.checkSelfPermission(context,
        Manifest.permission.RECORD_AUDIO) ==
        PackageManager.PERMISSION_GRANTED

// In Activity / Fragment using ActivityResultContracts:
private val requestMic = registerForActivityResult(
    ActivityResultContracts.RequestPermission()
) { granted →
  
```

KOTLIN

```

        if (granted) startCapture() else showPermissionRationale()
    }
    // Launch: requestMic.launch(Manifest.permission.RECORD_AUDIO)

```

Full AudioRecord Implementation (Kotlin Coroutines)

```

import android.media.*
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import java.nio.ByteBuffer
import java.nio.ByteOrder

object AudioCaptureConfig {
    const val SAMPLE_RATE = 16_000 // Hz - mandatory for STT models
    const val CHANNEL_IN = AudioFormat.CHANNEL_IN_MONO
    const val ENCODING = AudioFormat.ENCODING_PCM_16BIT
    const val FRAME_SAMPLES = 512 // 32ms @ 16kHz - Silero VAD v6 window
    size
    const val BUFFER_FRAMES = 10 // 320ms ring buffer
}

class AudioCaptureManager(private val scope: CoroutineScope) {

    private var audioRecord: AudioRecord? = null
    private var captureJob: Job? = null

    // Emits Float32 frames (normalized [-1, 1]) on a background thread
    private val _frames = MutableSharedFlow<FloatArray>(
        extraBufferCapacity = AudioCaptureConfig.BUFFER_FRAMES
    )
    val frames: SharedFlow<FloatArray> = _frames

    fun start() {
        if (audioRecord != null) return // Already running

        val minBuf = AudioRecord.getMinBufferSize(
            AudioCaptureConfig.SAMPLE_RATE,
            AudioCaptureConfig.CHANNEL_IN,
            AudioCaptureConfig.ENCODING
        )
        // Use at least 2x minBuf to avoid overruns
        val bufBytes = maxOf(minBuf, AudioCaptureConfig.FRAME_SAMPLES * 2 * 2)
    }

```

KOTLIN

```

audioRecord = AudioRecord(
    MediaRecorder.AudioSource.MIC, // Use MIC, not VOICE_RECOGNITION
    AudioCaptureConfig.SAMPLE_RATE,
    AudioCaptureConfig.CHANNEL_IN,
    AudioCaptureConfig.ENCODING,
    bufBytes
).also { rec →
    check(rec.state == AudioRecord.STATE_INITIALIZED) {
        "AudioRecord init failed – check RECORD_AUDIO permission"
    }
    rec.startRecording()
}

captureJob = scope.launch(Dispatchers.IO) {
    val buf = ShortArray(AudioCaptureConfig.FRAME_SAMPLES)
    while (isActive) {
        val read = audioRecord!!.read(buf, 0, buf.size)
        if (read > 0) {
            // Convert Int16 PCM → Float32 normalized to [-1, 1]
            val floats = FloatArray(read) { buf[it] / 32768f }
            _frames.tryEmit(floats)
        }
    }
}

fun stop() {
    captureJob?.cancel()
    captureJob = null
    audioRecord?.stop()
    audioRecord?.release()
    audioRecord = null
}
}

```



Always call `getMinBufferSize()` first

The minimum buffer size varies by device. A Pixel 7 returns 640 bytes; a Samsung S24 Ultra may return 1280 bytes. Creating an `AudioRecord` with a buffer smaller than the device minimum causes silent data loss — the `AudioRecord` initializes but returns garbage. Always call

`AudioRecord.getMinBufferSize()` and use `maxOf(minBuf, yourDesiredSize)`.

IME / Service Integration (Voice Keyboard)

When running audio capture inside an `InputMethodService` (keyboard IME), you must manage the capture lifecycle from IME callbacks rather than Activity lifecycle:

```
class VoiceKeyboardService : InputMethodService() {KOTLIN

    private val serviceScope = CoroutineScope(SupervisorJob() +
Dispatchers.Main)
    private lateinit var captureManager: AudioCaptureManager
    private lateinit var sttEngine: STTEngine

    override fun onCreate() {
        super.onCreate()
        captureManager = AudioCaptureManager(serviceScope)
        sttEngine = STTEngine(applicationContext)
    }

    // Start recording when user activates voice input
    fun startVoiceInput() {
        captureManager.start()
        serviceScope.launch {
            captureManager.frames.collect { frame →
                sttEngine.feedAudio(frame)
            }
        }
    }

    fun stopVoiceInput() {
        captureManager.stop()
        // Flush remaining audio, commit text to currentInputConnection
        serviceScope.launch {
            val finalText = sttEngine.flush()
            currentInputConnection?.commitText(finalText, 1)
        }
    }
}
```

```
}  
  
    override fun onDestroy() {  
        serviceScope.cancel()  
        sttEngine.release()  
        super.onDestroy()  
    }  
}
```

SECTION 05

Pre-Processing & Noise Reduction

● INTERMEDIATE

PREREQUISITES:

Audio Capture Layer

Core Concepts

Noise suppression, echo cancellation, and automatic gain control — the signal conditioning layer that determines model accuracy before a single feature is computed.

Pre-processing is invisible when done correctly. It's catastrophically visible when done wrong or omitted. Models trained on clean studio audio suffer 30–60% WER degradation in noisy environments without pre-processing. This section covers every DSP stage between raw PCM and model input.

The Pre-Processing Chain

The recommended ordering is critical — each stage assumes the previous one has been applied:

1. **AEC (Acoustic Echo Cancellation)** — Remove speaker playback captured by the microphone. Must happen first because NS and VAD will treat echo as speech.
2. **Noise Suppression (NS)** — Attenuate background noise. RNNoise or WebRTC NS. Apply before AGC to avoid amplifying residual noise.
3. **AGC (Automatic Gain Control)** — Normalize amplitude across speakers and environments.

4. **VAD** — Gate the model on confirmed speech segments (see [Section 06](#)).

Tool 1: WebRTC Audio Processing Module (APM)

WebRTC APM (https://webrtc.googlesource.com/src/+refs/heads/main/modules/audio_processing/) is the battle-tested C++ library behind Google Meet, Zoom, and every major real-time audio platform. It provides AEC, NS, and AGC in a single API. For Android, the preferred approach is to use it via the `webrtc-android` AAR or build it from source using the `libwebrtc` CMake build.

Tool 2: RNNoise

RNNoise (<https://jmvalin.ca/demo/rnnoise/>) (Mozilla) is a small (~100KB) neural noise suppressor trained on a diverse noise corpus. It processes 48kHz audio in 10ms frames, requiring a 16kHz→48kHz resample before processing and 48kHz→16kHz resample after. It excels at stationary and semi-stationary noise (HVAC, traffic, fan). For non-stationary noise (crowd chatter), Krisp outperforms it significantly.

Tool 3: Android NoiseSuppressor (API 16+)

Android provides a system-level `NoiseSuppressor` as an `AudioEffect`. It's hardware-dependent — quality varies dramatically across devices. Use it as a fallback when you can't include a third-party NS library:

```
import android.media.audiofx.NoiseSuppressor

fun tryEnableSystemNS(audioSession: Int): NoiseSuppressor? {
    if (!NoiseSuppressor.isAvailable()) return null
    return try {
        NoiseSuppressor.create(audioSession)?.also { ns →
            ns.enabled = true
        }
    } catch (e: RuntimeException) {
        null // Effect not supported on this device
    }
}
```

KOTLIN

```

    }
}
// audioSession = audioRecord.audioSessionId

```

RMS-Based AGC (Python — for server-side pipelines)

```

import numpy as np

def normalize_rms(audio: np.ndarray, target_rms: float = 0.1) → np.ndarray:
    """RMS-based amplitude normalization. Safe for silence."""
    rms = np.sqrt(np.mean(audio ** 2))
    if rms < 1e-8: # Silence — don't amplify
        return audio
    gain = target_rms / rms
    return np.clip(audio * gain, -1.0, 1.0)

class AdaptiveAGC:
    """Streaming AGC with asymmetric attack/release time constants."""
    def __init__(self, target_rms=0.1, attack=0.01, release=0.001):
        self.target_rms = target_rms
        self.attack = attack # Fast gain reduction
        self.release = release # Slow gain increase (prevents pumping)
        self.current_gain = 1.0

    def process(self, frame: np.ndarray) → np.ndarray:
        rms = np.sqrt(np.mean(frame ** 2) + 1e-10)
        desired = self.target_rms / rms
        alpha = self.attack if desired < self.current_gain else self.release
        self.current_gain = alpha * desired + (1 - alpha) * self.current_gain
        self.current_gain = np.clip(self.current_gain, 0.01, 10.0)
        return np.clip(frame * self.current_gain, -1.0, 1.0)

```



The canonical pre-processing order

AEC → Noise Suppression → AGC → VAD → Feature Extraction.

Never swap NS and VAD — your VAD will trigger on noise artifacts. Never apply AGC before NS — you'll amplify the noise you're about to suppress. Never skip AEC when your app plays audio the microphone can hear.

Voice Activity Detection (VAD)

● INTERMEDIATE

PREREQUISITES:

Pre-Processing & Noise Reduction

Pipeline Architecture

Detecting when the user is actually speaking. The most impactful latency and battery optimization in the pipeline — reduces model compute by 80–90% and eliminates Whisper's hallucination-on-silence problem.

Without VAD, your STT model processes continuous audio including every second of silence and background noise. This wastes compute, drains battery, and causes Whisper — and most other encoder-decoder models — to hallucinate text during silence. A properly tuned VAD is often worth more accuracy points than a bigger model.

Tool 4: Silero VAD v6 (Recommended)

Silero VAD (<https://github.com/snakers4/silero-vad>) is the most widely-used VAD in production voice pipelines as of 2026. Key facts: ~2MB ONNX model, <1ms per 32ms chunk on CPU, language-agnostic (VAD operates on signal energy, not language), MIT license, no API keys. Version 6 (August 2025) improved quality with 16% fewer errors on noisy data. Version 6.2 (November 2025) improved stability on edge cases (unusual voices, child voices, cartoon voices, muted speech, lower-quality phone calls). Version 6.2.1 (February 2026) made ONNX Runtime an optional dependency. Silero is used natively inside Moonshine Voice and can be enabled with a flag in whisper.cpp.

Silero VAD — Python Streaming Integration

```
import numpy as np
import torch
import onnxruntime as ort
from typing import Optional, Callable

class SileroVADv6:
    """
    Silero VAD v6 ONNX wrapper for streaming STT pipelines.
```

PYTHON

Operates on 32ms frames @ 16kHz (512 samples per chunk).

Downloads model: `pip install silero-vad`

"""

`CHUNK_SIZE = 512` # 32ms @ 16kHz - v6 fixed window size

`SR = 16_000`

```
def __init__(self, threshold: float = 0.5, min_speech_ms: int = 250,
              min_silence_ms: int = 500):
```

```
    from silero_vad import load_silero_vad
```

```
    self.model = load_silero_vad(onnx=True)
```

```
    self.threshold = threshold
```

```
    self.min_speech_samples = int(min_speech_ms * self.SR / 1000)
```

```
    self.min_silence_samples = int(min_silence_ms * self.SR / 1000)
```

```
    self._reset()
```

```
def _reset(self):
```

```
    self.speaking = False
```

```
    self.speech_buffer: list[np.ndarray] = []
```

```
    self.silence_samples = 0
```

```
    self.speech_samples = 0
```

```
def process_chunk(self, chunk: np.ndarray,
```

```
                  on_speech_end: Callable[[np.ndarray], None]) →
```

```
Optional[float]:
```

```
    """
```

```
    Feed a 32ms chunk (512 float32 samples @ 16kHz).
```

```
    Calls on_speech_end(audio) when a complete utterance is detected.
```

```
    Returns current VAD probability (0-1).
```

```
    """
```

```
    assert len(chunk) == self.CHUNK_SIZE, f"Expected {self.CHUNK_SIZE} sam-  
ples"
```

```
    prob = float(self.model(torch.from_numpy(chunk), self.SR).item())
```

```
    if prob ≥ self.threshold:
```

```
        self.speech_buffer.append(chunk.copy())
```

```
        self.speech_samples += self.CHUNK_SIZE
```

```
        self.silence_samples = 0
```

```
        self.speaking = True
```

```
    elif self.speaking:
```

```
        self.speech_buffer.append(chunk.copy()) # Include trailing silence
```

```
        self.silence_samples += self.CHUNK_SIZE
```

```
        if self.silence_samples ≥ self.min_silence_samples:
```

```
            if self.speech_samples ≥ self.min_speech_samples:
```

```
                utterance = np.concatenate(self.speech_buffer)
```

```

        on_speech_end(utterance)
        self._reset()
    return prob

```

Silero VAD — Android Kotlin via ONNX Runtime

```

// build.gradle.kts dependency:
// implementation("com.microsoft.onnxruntime:onnxruntime-android:1.27.0")
// Copy silero_vad.onnx to app/src/main/assets/

import ai.onnxruntime.*
import java.nio.FloatBuffer

class SileroVAD(context: Context) : AutoCloseable {

    private val session: OrtSession
    private val env = OrtEnvironment.getEnvironment()

    // Silero VAD v5/v6 uses a single state tensor [2, 1, 128] (LSTM hidden + cell)
    private var state = OnnxTensor.createTensor(
        env, FloatBuffer.wrap(FloatArray(256)), LongArrayOf(2, 1, 128)
    )
    // 64-sample context carried from previous chunk's tail
    private var ctx = FloatArray(64)

    init {
        val bytes = context.assets.open("silero_vad.onnx").readBytes()
        val opts = OrtSession.SessionOptions().apply {
            setIntraOpNumThreads(1) // VAD runs on 1 thread – keeps it fast
            addNnapi() // Use NNAPI if available on device
        }
        session = env.createSession(bytes, opts)
    }

    /**
     * Process one 512-sample (32ms @ 16kHz) Float32 chunk.
     * Returns speech probability 0.0-1.0.
     * Silero VAD v5/v6 is an LSTM – state tensor must persist between chunks.
     * Input is context (64) + chunk (512) = 576 samples.
     */
    fun predict(chunk: FloatArray): Float {
        require(chunk.size == 512) { "VAD requires exactly 512 samples (32ms)"

```

```

}

    // Prepend 64-sample context from previous chunk
    val input = FloatArray(576)
    System.arraycopy(ctx, 0, input, 0, 64)
    System.arraycopy(chunk, 0, input, 64, 512)

    val inputTensor = OnnxTensor.createTensor(
        env, FloatBuffer.wrap(input), longArrayOf(1, 576)
    )
    val srTensor = OnnxTensor.createTensor(
        env, LongBuffer.wrap(longArrayOf(16000)), longArrayOf(1)
    )

    val inputs = mapOf("input" to inputTensor, "state" to state,
        "sr" to srTensor)
    val result = session.run(inputs)

    val prob = (result[0].value as Array<*>)[0] as FloatArray
    state = result[1] as OnnxTensor // Update LSTM state

    // Save last 64 samples as context for next chunk
    System.arraycopy(input, 512, ctx, 0, 64)

    inputTensor.close(); srTensor.close()
    return prob[0]
}

fun resetState() {
    state = OnnxTensor.createTensor(
        env, FloatBuffer.wrap(FloatArray(256)), longArrayOf(2, 1, 128)
    )
    ctx = FloatArray(64)
}

override fun close() { session.close() }
}

```



Silero VAD is stateful — never skip `resetState()`

Silero VAD is an RNN. Its LSTM `state` tensor and 64-sample context must be preserved between chunks within a single audio stream. When a new recording session starts, call `resetState()`. Forgetting this produces systematically wrong probability scores on the second and subsequent utterances.

VAD Comparison

VAD	SIZE	LATENCY/CHUNK	LANGUAGES	LICENSE	BEST FOR
Silero VAD v6	~2MB	<1ms	6000+	MIT	General production use — recommended
WebRTC VAD	<100KB	<0.1ms	Language-agnostic	BSD	Ultra-low overhead, embedded
Moonshine VAD	Bundled	<2ms	Multilingual	MIT	Already using Moonshine pipeline
Cobra (Picovoice)	<1MB	<1ms	Multilingual	Commercial	Highest accuracy, noise resilience
Energy/RMS threshold	OKB	<0.01ms	Universal	None	Quick prototype only — fails in noise

Streaming & Chunking

● INTERMEDIATE

How to buffer, chunk, and stream audio between capture and model inference – including ring buffers, overlap strategies, and transport protocols.

Streaming is where latency is won or lost. A model that achieves RTF 0.05 on a test file can feel sluggish in a voice agent if the chunking strategy forces it to buffer 10 seconds before starting inference. This section covers every pattern for efficiently moving audio from capture to model.

Chunk Size vs. Latency vs. Accuracy

Larger chunks give models more context and typically improve accuracy; smaller chunks reduce first-token latency. The right balance depends on your model architecture:

ARCHITECTURE	MIN USEFUL CHUNK	OPTIMAL STREAMING CHUNK	NOTES
Whisper (encoder-decoder)	Full utterance or 30s	30s padded segments	Fixed-window model — doesn't stream natively
Moonshine v2 (ergodic)	32ms	512–1024 samples	Streaming-native, bounded TTFT regardless of length
Parakeet TDT (transducer)	40ms	160–320ms chunks	Best streaming architecture for low latency
Vosk / Kaldi (CTC)	10ms	100–200ms	CTC output per frame, greedy decoding

Ring Buffer Pattern (Kotlin)

```
/**
 * Thread-safe ring buffer for audio samples.
 * Producer: AudioRecord callback thread.
 * Consumer: STT inference thread.
 */
class AudioRingBuffer(private val capacity: Int) {
    private val buffer = FloatArray(capacity)
    private var writePos = 0
```

KOTLIN

```

private var readPos = 0
private var available = 0

@Synchronized
fun write(samples: FloatArray) {
    for (s in samples) {
        buffer[writePos % capacity] = s
        writePos++
        if (available < capacity) available++
        else readPos++ // Overwrite oldest when full
    }
}

@Synchronized
fun read(n: Int): FloatArray? {
    if (available < n) return null
    val out = FloatArray(n) { buffer[(readPos + it) % capacity] }
    readPos += n; available -= n
    return out
}

@Synchronized
fun availableSamples(): Int = available
}

```

WebSocket Streaming to Server (Kotlin + OkHttp)

```

// Streaming PCM audio to a cloud STT WebSocket endpoint
// Compatible with Deepgram, AssemblyAI real-time API, and custom servers
import okhttp3.*
import java.nio.ByteBuffer
import java.nio.ByteOrder

class STTWebSocketClient(private val url: String, private val apiKey: String) {

    private val client = OkHttpClient()
    private var socket: WebSocket? = null
    var onTranscript: ((String) → Unit)? = null

    fun connect() {
        val request = Request.Builder()
            .url(url)
            .addHeader("Authorization", "Token $apiKey")

```

KOTLIN

```

        .build()
        socket = client.newWebSocket(request, object : WebSocketListener() {
            override fun onMessage(ws: WebSocket, text: String) {
                // Parse JSON transcript from server
                onTranscript?.invoke(text)
            }
        })
    }

    /** Send Float32 audio chunk as raw Int16 PCM bytes over WebSocket. */
    fun sendAudio(floatChunk: FloatArray) {
        val pcm = ByteBuffer.allocate(floatChunk.size * 2)
            .order(ByteOrder.LITTLE_ENDIAN)
        floatChunk.forEach { s → pcm.putShort((s *
32767).toInt().coerceIn(-32768, 32767).toShort()) }
        socket?.send(ByteString.of(*pcm.array()))
    }

    fun close() = socket?.close(1000, "done")
}

```



Overlap your chunks.

When using batch models like Whisper with a chunked pipeline, always overlap adjacent chunks by 1–2 seconds. Without overlap, words that straddle a chunk boundary will be split or dropped — a common source of silently degraded accuracy that won't show up in single-utterance tests.

Feature Extraction

● INTERMEDIATE

PREREQUISITES:

Voice Activity Detection

Streaming & Chunking

Converting raw PCM into the tensor representations that acoustic models consume — log-mel spectrograms, MFCC, and why Moonshine bypasses this layer entirely.

Feature extraction transforms time-domain audio samples into frequency-domain representations that expose the spectral patterns models use to recognize phonemes. For most modern models — Whisper, Canary, Parakeet — this means computing an 80-bin log-mel spectrogram. For Moonshine v2, this layer is learnable convolutional layers inside the encoder itself.

Log-Mel Spectrogram (Python — matches Whisper's exact spec)

```
import numpy as np

def log_mel_spectrogram(audio: np.ndarray, sr: int = 16000,
                        n_mels: int = 80, n_fft: int = 400,
                        hop_length: int = 160) → np.ndarray:
    """
    Compute log-mel spectrogram matching Whisper's preprocessing exactly.
    Input: float32 audio, normalized [-1, 1], mono 16kHz
    Output: float32 tensor of shape (n_mels, T) where T = ceil(len/hop_length)
    """
    import librosa

    # Compute mel spectrogram (power=2 for energy spectrogram)
    mel = librosa.feature.melspectrogram(
        y=audio, sr=sr, n_fft=n_fft, hop_length=hop_length,
        n_mels=n_mels, fmin=0, fmax=sr // 2
    )

    # Convert to log scale — clamp at 1e-10 to avoid log(0)
    log_mel = np.log10(np.maximum(mel, 1e-10))

    # Normalize to [-1, 1] range: Whisper uses (log_mel - max/8) * 4/max
    log_mel = np.maximum(log_mel, log_mel.max() - 8.0)
```

PYTHON

```
log_mel = (log_mel + 4.0) / 4.0
```

```
return log_mel.astype(np.float32)
```

```
# For 30-second audio at 16kHz: output shape is (80, 3000)
```

```
# 3000 = 30s × 16000 / 160 (hop_length)
```



Moonshine skips explicit feature extraction

Moonshine's encoder takes raw 16kHz float32 audio directly and learns its own convolutional front-end. This simplifies your pipeline by two steps — no STFT, no mel filterbank computation. whisper.cpp implements a highly optimized mel spectrogram in C++ with SIMD vectorization; if you're using it, you never need to compute spectrograms in application code.

Tool: torchaudio — GPU-Accelerated Mel Spectrogram

torchaudio (<https://pytorch.org/audio/stable/>) computes mel spectrograms on CUDA tensors in a single fused pass — under 2 ms for 30 s of audio on a V100. Use it for server-side training pipelines, batch transcription evaluation, and offline data preprocessing.

```
# pip install torchaudio torch
import torch
import torchaudio
import torchaudio.transforms as T
```

PYTHON

```
def log_mel_torchaudio(
    wav_path: str,
    device: str = "cuda",
) → torch.Tensor:
    waveform, sr = torchaudio.load(wav_path)          # [C, T]
    waveform = waveform.mean(0, keepdim=True)        # Mono
    if sr != 16_000:
        waveform = T.Resample(sr, 16_000)(waveform)
    waveform = waveform.to(device)

    mel_fn = T.MelSpectrogram(
        sample_rate=16_000,
        n_fft=400,          # 25 ms analysis window
        hop_length=160,     # 10 ms stride → 100 frames/s
        n_mels=80,
        f_min=0.0,
```

```

        f_max=8_000.0,
    ).to(device)

    mel      = mel_fn(waveform)                # [1, 80, T]
    mel_db   = T.AmplitudeToDB(top_db=80)(mel)  # Log scale
    return mel_db.squeeze(0)                  # [80, T]

```

Tool: librosa — Analysis & Debugging

librosa (<https://librosa.org>) is pure NumPy, CPU-only, but indispensable for visualizing spectrograms, diagnosing silence, clipping, or sample-rate mismatches in development. Generate a debug image whenever WER suddenly spikes — the spectrogram usually reveals the problem immediately.

```

# pip install librosa soundfile matplotlib
import librosa, librosa.display
import numpy as np
import matplotlib.pyplot as plt

y, sr = librosa.load("speech.wav", sr=16_000)

# 80-band log-mel (Whisper / Moonshine compatible)
mel      = librosa.feature.melspectrogram(
    y=y, sr=sr, n_fft=400, hop_length=160,
    n_mels=80, fmin=0.0, fmax=8_000.0,
)
mel_db   = librosa.power_to_db(mel, ref=np.max)  # [80, T]

# 39-dim MFCC + delta + delta^2 for Kaldi / Vosk pipelines
mfcc     = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
d1       = librosa.feature.delta(mfcc)
d2       = librosa.feature.delta(mfcc, order=2)
feats    = np.vstack([mfcc, d1, d2])            # [39, T]

# Save debug spectrogram – spot silence, clipping, DC offset at a glance
fig, ax = plt.subplots(figsize=(12, 4))
librosa.display.specshow(
    mel_db, sr=sr, hop_length=160,
    x_axis="time", y_axis="mel", ax=ax,
)

```

PYTHON

```
plt.colorbar(format="%+2.0f dB")
plt.title("Mel spectrogram - 16 kHz, 80 bands")
plt.savefig("mel_debug.png", dpi=150, bbox_inches="tight")
```



MFCC vs Log-Mel

Whisper and Moonshine consume

80-band log-mel spectrograms

. Only feed 13- or 40-dimensional MFCCs to models explicitly trained on them (Vosk, Kaldi, older CMU Sphinx). Mixing feature types silently degrades WER by 10–30 percentage points — the encoder learns a completely different statistical space.

On-Device ONNX Feature Extraction (Android)

Export the mel filterbank as a constant node inside an ONNX graph so that Android computes features on-device without a Python dependency. The standalone ONNX accepts raw 16 kHz PCM floats and emits a [1, 80, T] mel tensor, consumed directly by the ASR model session.

Export stand-alone mel spectrogram ONNX for Android bundling

PYTHON

```
import torch
import torchaudio.transforms as T

class MelExtractor(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.mel = T.MelSpectrogram(
            sample_rate=16_000, n_fft=400,
            hop_length=160, n_mels=80,
        )
        self.db = T.AmplitudeToDB(top_db=80)

    def forward(self, pcm: torch.Tensor) → torch.Tensor:
        return self.db(self.mel(pcm)) # [1, 80, T]

model = MelExtractor().eval()
dummy = torch.zeros(1, 16000) # 1 second dummy input
torch.onnx.export(
    model, (dummy,), "mel_extractor.onnx",
    input_names=["pcm"], output_names=["mel"],
    dynamic_axes={
```

```
    "pcm": {1: "samples"},  
    "mel": {2: "frames"},  
  },  
  opset_version=17,  
)
```

Acoustic Models Landscape

● BEGINNER

Every major open-source STT model as of 2026 – architecture, accuracy, size, on-device viability, and the right choice for each deployment scenario.

The 2026 open-source STT landscape has stratified into clear tiers. Canary Qwen 2.5B leads on accuracy; Moonshine v2 leads on edge/mobile latency; Parakeet leads on server-side throughput; whisper.cpp leads on portable embedding. Choose based on your constraints, not on benchmark headlines.

MODEL	WER (OPEN ASR AVG.)	RTFX (GPU)	SIZE	ON-DEVICE?	LANGUAGES	LICENSE
Canary Qwen 2.5B Cloud/Server	5.63%	418×	~5GB	✗ Too large	EN (primary)	CC-BY-4.0
IBM Granite Speech 3.3 8B Cloud/Server	5.74%	~145×	~18GB	✗	EN, FR, DE, ES, PT	Apache 2.0
Whisper Large v3 Cloud/Server	7.44%	~145×	~3.1GB	⚠ High-end only	99 languages	MIT
Whisper Large v3 Turbo Cloud	7.83%	~200×	~1.6GB	⚠ Server preferred	99 languages	MIT
Distil-Whisper Large-v3 Server/Edge	7.52%	~214×	756 MB	⚠ Possible	EN only	MIT
Whisper Small (.en) Edge/Mobile	8.59%	~269×	~48 MB	✓	EN (multi-lingual variant available)	MIT
Whisper Tiny (.en) Mobile	12.81%	~348×	~78 MB	✓	EN (multi-lingual variant available)	MIT
Moonshine Base Mobile/Edge	9.99%	~566×	~59 MB	✓ Recommended	EN (+ lang variants)	MIT
Moonshine Tiny Mobile/IoT	12.65%	~753×	~26 MB	✓ Recommended	EN (+ lang variants)	MIT
NVIDIA Parakeet TDT 1.1B	7.02%	~2391×	4.4GB	✗	EN	CC-BY-4.0
Vosk (small model) Mobile	~12%	High	40MB	✓	20+ languages	Apache 2.0

Decision Tree: Which Model?

1. **Need multilingual + highest possible accuracy, server-side:** Whisper Large v3 or Canary Qwen 2.5B.

2. **Need low-latency real-time on Android, offline:** Moonshine Tiny or Moonshine Base.
If you need 99 languages: Whisper Small via [whisper.cpp](#).
3. **Need maximum server throughput (dictation, transcription SaaS):** NVIDIA Parakeet TDT with NeMo or Distil-Whisper.
4. **Need the smallest possible binary on IoT / embedded:** Vosk (40MB) or Moonshine Tiny (26MB).
5. **Building a cloud product and accuracy is paramount:** Deepgram Nova-3 or AssemblyAI (managed, not open source).



Model accuracy degrades on domain-specific audio.

WER numbers above are 8-dataset averages from the [Hugging Face Open ASR Leaderboard](https://huggingface.co/spaces/hf-audio/open_asr_leaderboard) (https://huggingface.co/spaces/hf-audio/open_asr_leaderboard) (AMI, Earnings22, GigaSpeech, LibriSpeech Clean, LibriSpeech Other, SPGISpeech, TED-LIUM, VoxPopuli). LibriSpeech Clean alone yields much lower WER for every model — e.g. Canary Qwen 2.5B scores 1.61% on LS Clean vs 5.63% average. Your production audio — noisy environments, accented speakers, domain jargon — will produce higher WER. Always benchmark on audio representative of your actual use case before choosing a model.

SECTION 10

Whisper Architecture

● INTERMEDIATE

PREREQUISITES:

Feature Extraction

Acoustic Models Landscape

How OpenAI's Whisper works inside — encoder-decoder attention, the 30-second window limitation, multilingual training, and why it hallucinates during silence.

Whisper (2022, OpenAI) remains the reference model for multilingual offline transcription. Its 99-language coverage, MIT license, and high accuracy on clean audio make it the default choice when mobile latency isn't critical. Understanding its architecture explains both its strengths and its limitations.

Architecture: Transformer Encoder-Decoder

Whisper is a standard encoder-decoder Transformer architecture. The encoder is a convolutional front-end followed by Transformer attention blocks that produce a context vector from the 30-second log-mel spectrogram. The decoder is an autoregressive Transformer that generates tokens by attending to both the encoded audio and previously generated tokens.

The 30-Second Window

Whisper's encoder always processes exactly 30 seconds of audio (padded with silence if shorter). This design choice makes Whisper excellent for batch transcription — a single forward pass encodes 30 full seconds — but problematic for streaming. The practical consequence:

- Whisper cannot output any text until it has 30 seconds of audio (or until you force a shorter segment)
- TTFT (Time to First Token) grows linearly with utterance length in the encoder
- Short utterances waste compute on padding (a 2-second sentence pads 28 seconds of silence)

- During silence, the decoder hallucinates — it was trained on real audio and generates plausible text for random noise

Whisper Model Variants

VARIANT	PARAMS	ENCODER LAYERS	DECODER LAYERS	EMBED DIM	MOBILE VIABLE
tiny	39M	4	4	384	✓
base	74M	6	6	512	✓
small	244M	12	12	768	✓ (tight)
medium	769M	24	24	1024	⚠
large-v3	1.55B	32	32	1280	✗
large-v3-turbo	809M	32	4	1280	⚠ high-end only



Whisper Turbo: 4-layer decoder

Whisper Large v3 Turbo keeps the full 32-layer encoder (responsible for most accuracy) while reducing the decoder to just 4 layers (vs. 32 in the original). The decoder contributes little to overall accuracy but most of the decode latency. Turbo achieves ~8× faster decoding with only ~0.3% WER regression on most benchmarks. Use Turbo for any server-side deployment where full multilingual accuracy is needed at reduced compute cost.

Suppressing Whisper Hallucinations

```
from faster_whisper import WhisperModel
```

PYTHON

```
# Whisper hallucination suppression strategies (faster-whisper API)
```

```
model = WhisperModel("small", device="cpu", compute_type="int8")
```

```
segments, info = model.transcribe(
```

```
    audio_path,
```

```
    # Key anti-hallucination settings:
```

```
    no_speech_threshold=0.6,          # Suppress if no-speech prob > 0.6
```

```
    log_prob_threshold=-1.0,         # Skip segment if avg logprob too low
```

```
    compression_ratio_threshold=2.4, # Skip suspiciously repetitive output
```

```
    condition_on_previous_text=False, # Don't hallucinate continuations
```

```
    temperature=[0.0, 0.2, 0.4, 0.6, 0.8, 1.0], # Fallback temperatures
```

```
# Use VAD to pre-filter silence before passing to Whisper:  
vad_filter=True, # Silero VAD built into faster-whisper  
)
```

SECTION 11

Moonshine v1 & v2

● EXPERT

PREREQUISITES:

Whisper Architecture

Feature Extraction

The streaming-native model family purpose-built for edge and mobile. v1's encoder/decoder state caching and v2's ergodic sliding-window attention – the architecture that makes sub-200ms latency real.

Moonshine (Useful Sensors / Moonshine AI) was designed from scratch to address Whisper's fundamental streaming limitation. Rather than forcing edge use into a batch architecture, Moonshine's encoder was redesigned for variable-length streaming with per-chunk state caching. Moonshine v2 (February 2026) takes this further with sliding-window ergodic attention that achieves **bounded TTFT independent of utterance length**.

Moonshine v1: Flexible Windows + State Caching

Moonshine v1's key innovation is that it processes variable-length audio rather than fixed 30-second windows. The encoder and decoder states are cached between audio chunks, enabling the model to "remember" what it already processed without recomputing from scratch. This alone delivers up to 5× faster processing compared to Whisper on the same hardware, because most of the encoder forward pass is skipped for incremental audio.

Moonshine v2: Ergodic Streaming Encoder

Moonshine v2 replaces full self-attention (quadratic in sequence length) with **sliding-window self-attention** in a position-free encoder. Every attention computation is local to a fixed window size, making the complexity $O(N \times W)$ rather than $O(N^2)$, where W is the window size. The "ergodic" property means the model provably accesses all historical context through overlapping windows.

The practical result: TTFT is bounded and fixed regardless of how long the user has been speaking. A 5-second utterance and a 2-minute monologue both produce the first token in the same time. This is physically impossible with a full-attention encoder.

Moonshine Voice — Android SDK Integration



Official Android SDK available.

Moonshine AI publishes `ai.moonshine:moonshine-voice` on Maven Central (v0.0.70+) with JNI bindings, `MicTranscriber` API, and pre-bundled models. For most Android projects, use the official SDK instead of the manual ONNX Runtime integration below. See [Section 21](#) for details.

```
// Manual ONNX Runtime integration – useful for custom inference engines KOTLIN
// For most projects, use the official SDK: ai.moonshine:moonshine-voice (§21)
// Or integrate via Gradle using ONNX Runtime + Moonshine ONNX models

// build.gradle.kts
// implementation("com.microsoft.onnxruntime:onnxruntime-android:1.27.0")

class MoonshineSTT(context: Context) : AutoCloseable {

    private val env = OrtEnvironment.getEnvironment()
    private val preprocessSession: OrtSession
    private val encoderSession: OrtSession
    private val decoderSession: OrtSession
    private val tokenizer: Tokenizer // SentencePiece or custom BPE

    init {
        fun loadSession(assetName: String): OrtSession {
            val bytes = context.assets.open(assetName).readBytes()
            return env.createSession(bytes, OrtSession.SessionOptions().apply {
                setIntraOpNumThreads(4)
                addNnapi() // Hardware acceleration on supported devices
            })
        }
        // Models from: huggingface.co/usefulsensors/moonshine-tiny
        preprocessSession = loadSession("preprocess.onnx")
        encoderSession = loadSession("encode.onnx")
        decoderSession = loadSession("uncached_decode.onnx")
        tokenizer = Tokenizer.fromAsset(context, "moonshine_tokenizer.json")
    }
}
```

```

/**
 * Transcribe a complete utterance (float32 mono 16kHz array).
 * For streaming, feed chunks to a running encoder state.
 * Returns the final transcript string.
 */
fun transcribe(audio: FloatArray): String {
    // 1. Preprocess: raw audio → feature representation
    val audioTensor = OnnxTensor.createTensor(
        env, FloatBuffer.wrap(audio), longArrayOf(1, audio.size.toLong())
    )
    val features = preprocessSession.run(mapOf("audio" to audioTensor))
    audioTensor.close()

    // 2. Encode: features → context vectors
    val featuresLen = OnnxTensor.createTensor(env,
longArrayOf(features[0].info.shape[1]))
    val context = encoderSession.run(mapOf(
        "features"      to features[0],
        "features_len" to featuresLen
    ))

    // 3. Decode: autoregressive token generation
    val tokens = mutableListOf(tokenizer.bosToken)
    repeat(256) { // Max decode steps
        val seqLen = OnnxTensor.createTensor(env,
longArrayOf(tokens.size.toLong()))
        val logits = decoderSession.run(mapOf(
            "tokens"      to OnnxTensor.createTensor(env, tokens.toIntAr-
ray()),
            "encoder_out" to context[0],
            "seq_len"     to seqLen
        ))
        val nextToken = argmax(logits[0] as FloatArray)
        if (nextToken == tokenizer.eosToken) return@repeat
        tokens.add(nextToken)
    }
    features.close(); context.close()
    return tokenizer.decode(tokens.drop(1)) // Skip BOS token
}

private fun argmax(arr: FloatArray): Int =
    arr.indices.maxByOrNull { arr[it] } ?: 0

override fun close() {

```

```
        preprocessSession.close(); encoderSession.close();
    decoderSession.close()
    }
}
```

Moonshine Flavors — Language-Specific Models (2025)

Flavors of Moonshine (<https://arxiv.org/abs/2509.02523>) (arXiv 2509.02523, September 2025) introduced tiny language-specific variants for underrepresented languages including Korean, Arabic, Vietnamese, Chinese, Japanese, and Ukrainian. These models are 28× smaller than Whisper Medium while achieving comparable or lower WER for their target language, with 5–15× faster inference. This makes them the correct choice for single-language mobile apps targeting non-English markets.



Moonshine v2 + Sherpa-ONNX is the fastest path to production.

Rather than writing your own ONNX Runtime integration (shown above), use [Section 22 — Sherpa-ONNX](#) which wraps Moonshine models into a ready-made streaming pipeline with VAD, endpoint detection, and pre-built Android AARs. Write the ONNX integration yourself only if you need custom preprocessing or caching beyond what Sherpa exposes.

SECTION 12

Streaming Decoders

● EXPERT

PREREQUISITES:

Whisper Architecture

Streaming & Chunking

CTC greedy decoding, beam search, RNN-Transducer, and how to implement each for real-time token emission.

The decoder translates the acoustic model's per-frame token probability distribution into a sequence of words. For encoder-decoder models (Whisper, Moonshine), decoding is autoregressive — token by token. For CTC models (Vosk, Wav2Vec2), decoding is frame-level probability collapse. The decoder choice determines whether your system can stream partial results or must wait for a complete utterance.

CTC Greedy Decode (Python)

```
import numpy as np
from typing import List

def ctc_greedy_decode(log_probs: np.ndarray, blank_id: int = 0,
                      vocabulary: List[str] = None) → str:
    """
    CTC greedy (best-path) decoding.
    log_probs: (T, V) — T frames, V vocabulary size
    Returns decoded string.
    """
    # Take argmax at each frame
    pred = np.argmax(log_probs, axis=1) # shape: (T,)

    # CTC collapse: remove consecutive duplicates, then blanks
    collapsed = []
    prev = -1
    for token in pred:
        if token != prev:
            if token != blank_id:
                collapsed.append(token)
            prev = token
```

PYTHON

```

if vocabulary:
    return ''.join(vocabulary[t] for t in collapsed)
return collapsed # Return token IDs if no vocabulary given

```

Beam Search Decode

Beam search maintains k candidate hypotheses simultaneously, expanding each with the most likely next tokens. It generally improves WER by 1–3% over greedy decoding at the cost of $k \times$ compute. For production decoder integration, use **Flashlight Text** (<https://github.com/flashlight/text>) (C++) or **pyctcdecode** (<https://github.com/kensho-technologies/pyctcdecode>) (Python), both of which support beam search with optional KenLM language model fusion.

Beam Search with pyctcdecode

pyctcdecode (<https://github.com/kensho-technologies/pyctcdecode>) provides a production-quality CTC beam search decoder with optional KenLM language model integration. It is model-agnostic: pass logits from any CTC-trained model (wav2vec 2.0, MMS, HuBERT) and it returns the best transcription.

```

# pip install pyctcdecode kenlm
from pyctcdecode import build_ctcdecoder
import kenlm, numpy as np

# Vocabulary must match the model's output indices exactly
# Index 0 = CTC blank token
LABELS = ["<blank>", " ", "a", "b", "c", "d", "e",
          "f", "g", "h", "i", "j", "k", "l", "m",
          "n", "o", "p", "q", "r", "s", "t", "u",
          "v", "w", "x", "y", "z", "'"]

decoder = build_ctcdecoder(
    labels=LABELS,
    kenlm_model=kenlm.Model("lm.binary"),
    alpha=0.5,      # LM interpolation weight (0 = no LM, 1 = LM only)
    beta=1.5,       # Word insertion bonus – increase for noisy speech
)

# logits: numpy [T, len(LABELS)] from CTC model
logits = np.load("ctc_logits.npy")
text = decoder.decode(logits, beam_width=100)
print(text)

```

PYTHON

```
# Top-N beams with scores (for N-best lists or rescoreing)
beams = decoder.decode_beams(logits, beam_width=50)
for text, _, _, _, logit_score, lm_score in beams[:3]:
    print(f"{text!r:40s}  logit={logit_score:.2f}  lm={lm_score:.2f}")
```

RNN-Transducer: Streaming-Native Decoder Architecture

CTC forces an independence assumption — every frame is decoded independently. The **RNN-Transducer (RNN-T)** removes that constraint: a joint network combines encoder and predictor (LM) outputs at every step, allowing the model to emit tokens before the full utterance is complete. Modern models that use RNN-T (including Moonshine v2 under the hood) achieve lower latency than CTC at the cost of more complex inference logic.

DECODER	INDEPENDENCE	STREAMING	LM INTEGRATION	COMPLEXITY
CTC Greedy	Frame-independent	Yes	External rescoreing	Trivial
CTC Beam (pyctcdecode)	Frame-independent	Yes	Integrated (KenLM)	Low
RNN-T	Token-conditional	Yes (by design)	Built-in predictor	Medium
Encoder-Decoder (Whisper)	Attention-based	Chunked only	Built-in decoder	High

Emitting Partial Results During Inference

For a responsive UI, emit interim text as soon as CTC beam tokens stabilize across consecutive frames. The prefix that has not changed in the last N frames can be displayed as a committed partial result.

```
# Streaming partial-result pattern (CTC beam search)
from pyctcdecode import BeamSearchDecoderCTC
import numpy as np

def stream_ctc_partial(
    decoder: BeamSearchDecoderCTC,
    logit_chunks,          # generator of numpy [T_chunk, V] arrays
    stability_frames: int = 5,
):
    # pyctcdecode context manager accumulates chunk logits
    with decoder.context_manager() as ctx:
        prev_prefix = ""
        for chunk in logit_chunks:
```

PYTHON

```
ctx.decode_beams_batch([chunk])
partial = ctx.decode()           # best beam so far
if partial != prev_prefix:
    yield partial, False         # interim update
    prev_prefix = partial
yield ctx.decode(), True        # final confirmed result
```



Whisper and Moonshine Use Autoregressive Decoders

For encoder–decoder models like Whisper and Moonshine, partial results are obtained by running greedy decoding on the current cached decoder state after each chunk. There is no separate "beam search decoder" — partial tokens are just the decoder's output tokens emitted so far in the autoregressive loop.

SECTION 13

Language Model Integration

● EXPERT

PREREQUISITES:

Streaming Decoders

Post-Processing

KenLM n-gram fusion, hot-word biasing, and domain adaptation for domain-specific vocabulary recognition.

Acoustic models produce phoneme-level token probabilities. Language models constrain which word sequences are actually possible in a given language and domain. Combining them — a process called shallow fusion or deep fusion — reduces WER significantly for specialized vocabularies (medical terms, product names, proper nouns).

KenLM n-gram Language Model

KenLM (<https://github.com/kpu/kenlm>) is the standard lightweight n-gram LM for production STT. A 5-gram KenLM built on 1M words of domain text can fit in 30–80MB and integrate with Flashlight or pyctcdecode to improve domain-specific WER by 15–30%.

Hot-Word Biasing (Whisper)

```
import whisper

# Whisper supports initial prompting to bias decoding toward known terms.
# For a medical app, pre-seed the decoder with domain vocabulary:
model = whisper.load_model("small")

result = model.transcribe(
    audio_path,
    initial_prompt="Transcribing cardiology consultation notes.
    Terms: tachycardia, bradycardia, echocardiogram, myocardial infarction,
    statin, metoprolol, lisinopril, cardiomegaly.",
    temperature=0.0,
    language="en"
)
```

PYTHON

```
# The decoder will strongly prefer these terms when acoustically plausible.  
# WER on domain-specific vocabulary typically improves 20-40% with good  
prompting.
```

Building a KenLM n-gram Model from a Text Corpus

A domain-specific n-gram language model consistently outperforms the default Whisper decoder on specialised vocabulary (medical, legal, brand names). A 4-gram KenLM model trained on 50 MB of domain text typically cuts domain WER by 15–30%.

```
# Step 1 – Build KenLM tools (Ubuntu/Debian) BASH  
sudo apt-get install -y build-essential cmake libboost-all-dev libeigen3-  
dev liblzma-dev libbz2-dev  
  
git clone https://github.com/kpu/kenlm.git  
cd kenlm && mkdir -p build && cd build  
cmake .. -DCMAKE_BUILD_TYPE=Release && make -j$(nproc)  
cd ../..  
  
# Step 2 – Deduplicate and clean your corpus (one sentence per line)  
sort -u raw_corpus.txt | grep -v '^$' > corpus.txt  
  
# Step 3 – Build 4-gram ARPA model with modified Kneser-Ney smoothing  
./kenlm/build/bin/lmplz -o 4 --discount_fallback <corpus.txt >lm.arpa  
  
# Step 4 – Compile to fast binary (10x faster at runtime)  
./kenlm/build/bin/build_binary trie lm.arpa lm.binary  
echo "LM ready: $(du -sh lm.binary | cut -f1)"
```

pyctcdecode + KenLM Integrated with Any CTC Model

```
# pip install pyctcdecode kenlm transformers PYTHON  
from transformers import Wav2Vec2Processor, Wav2Vec2ForCTC  
from pyctcdecode import build_ctcdecoder  
import torch, kenlm, soundfile as sf, numpy as np  
  
# Load CTC model (wav2vec 2.0 as example – any CTC model works)  
processor = Wav2Vec2Processor.from_pretrained("facebook/wav2vec2-base-960h")  
model = Wav2Vec2ForCTC.from_pretrained("facebook/wav2vec2-base-960h")  
  
# Build KenLM-aware decoder using the model's vocabulary
```

```

vocab = [v for v, _ in sorted(processor.tokenizer.get_vocab().items(),
                              key=lambda x: x[1])]

decoder = build_ctcdecoder(
    labels=vocab,
    kenlm_model=kenlm.Model("lm.binary"),
    alpha=0.5,
    beta=1.5,
)

# Transcribe any 16 kHz WAV
audio, sr = sf.read("speech.wav", dtype="float32")
inputs = processor(audio, sampling_rate=16000, return_tensors="pt")
with torch.no_grad():
    logits = model(**inputs).logits[0].numpy()      # [T, vocab]

text = decoder.decode(logits, beam_width=100)
print(text)

```

SentencePiece Tokenization

Whisper and Moonshine use SentencePiece subword tokenisers rather than character-level CTC vocabularies. If you need to bias the Whisper decoder towards domain vocabulary, use the `initial_prompt` parameter — it prepends a text prompt to the decoder context, shifting the probability distribution toward words in that prompt.

```

# Whisper initial_prompt: bias toward medical / domain vocabulary
import whisper

model = whisper.load_model("small")
result = model.transcribe(
    "patient_dictation.wav",
    language="en",
    initial_prompt="The patient presents with dyspnea, tachycardia, "
                  "and bilateral pulmonary infiltrates.",
)
print(result["text"])

```

PYTHON



Domain Adaptation Workflow

Collect 50–500 sentences of domain text → build KenLM OR compose an `initial_prompt` string from key terminology → test on 20 held-out utterances → tune `alpha` (LM weight) between 0.3 and 0.7. A value above 0.7 often causes hallucinations when the LM is too confident.

SECTION 14

Post-Processing

● INTERMEDIATE

Punctuation restoration, capitalization, speaker diarization, and text normalization – transforming raw ASR output into human-readable text.

Raw ASR output from most models is lowercase without punctuation: "the meeting starts at three pm tomorrow john will present the quarterly results". Post-processing transforms this into usable text. For keyboard voice input, you typically need at minimum punctuation and sentence-case capitalization.

Tool: DeepMultilingualPunctuation

```
# pip install deepmultilingualpunctuation
from deepmultilingualpunctuation import PunctuationModel

model = PunctuationModel(model="oliverguhr/fullstop-punctuation-multilang-large")

raw = "the meeting starts at three pm tomorrow john will present the quarterly results"
result = model.restore_punctuation(raw)
# → "The meeting starts at three pm. Tomorrow, John will present the quarterly results."
```

PYTHON

Simple Android Post-Processing (Kotlin)

```
object TranscriptPostProcessor {

    // Number word → digit substitution (EN)
    private val numberMap = mapOf(
        "zero" to "0", "one" to "1", "two" to "2",
        "three" to "3", "four" to "4", "five" to "5",
        "six" to "6", "seven" to "7", "eight" to "8", "nine" to "9"
    )

    fun process(raw: String): String {
```

KOTLIN

```

var text = raw.trim()
// Capitalize first letter of each sentence
text = text.split(Regex("(?≤[.!?])\\s+"))
    .joinToString(" ") { it.replaceFirstChar { c → c.upper-
case() } }
// Capitalize "I" pronoun
text = text.replace(Regex("(?≤\\s|^)i(?=\\s|$)"), "I")
// Substitute number words
numberMap.forEach { (word, digit) →
    text = text.replace(Regex("\\b$word\\b",
RegexOption.IGNORE_CASE), digit)
}
// Ensure sentence ends with period if no punctuation
if (text.isNotEmpty() && text.last() !in ".!?") text += "."
return text
}
}

```

Inverse Text Normalization — Numbers, Dates & Currency

ASR models output "three hundred and forty two dollars and fifty cents" rather than "\$342.50". Inverse text normalization (ITN) converts these spoken-form expressions to written form. The NVIDIA NeMo library ships a production-grade, rule-based ITN engine that handles English, Spanish, German, and Russian without a GPU.

```

# pip install nemo_text_processing
from nemo_text_processing.text_normalization.normalize import Normalizer
from nemo_text_processing.inverse_text_normalization.inverse_normalize import InverseNormalizer

itn = InverseNormalizer(lang="en")

examples = [
    "the meeting is on january fifteenth at three thirty pm",
    "the balance is one thousand two hundred and forty dollars",
    "call us at eight hundred five five five one two three four",
]
for raw in examples:
    print(itn.inverse_normalize(raw, verbose=False))

```

```
# → "the meeting is on January 15th at 3:30 PM"
# → "the balance is $1,240"
# → "call us at 1-800-555-1234"
```

Tool: pyannote.audio — Speaker Diarisation

pyannote.audio (<https://github.com/pyannote/pyannote-audio>) segments a recording into speaker turns ("who spoke when") with state-of-the-art accuracy. It is the standard choice for meeting transcription, interview analysis, and any multi-speaker scenario. A HuggingFace token is required to download the `pyannote/speaker-diarization-3.1` checkpoint.

```
# pip install pyannote.audio
from pyannote.audio import Pipeline
import whisper

# Step 1 – Diarise (requires HuggingFace access token)
pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization-3.1",
    use_auth_token="hf_YOUR_TOKEN",
)
diarization = pipeline("meeting.wav", num_speakers=2)

# Step 2 – Transcribe each speaker segment separately
asr = whisper.load_model("small")
import soundfile as sf, numpy as np
audio, sr = sf.read("meeting.wav")

segments = []
for turn, _, speaker in diarization.itertracks(yield_label=True):
    chunk = audio[int(turn.start * sr): int(turn.end * sr)]
    text = asr.transcribe(chunk.astype(np.float32))["text"].strip()
    segments.append((turn.start, speaker, text))

# Step 3 – Format timestamped speaker transcript
for start, speaker, text in segments:
    m, s = divmod(int(start), 60)
    print(f"[{m:02d}:{s:02d}] {speaker}: {text}")

# → [00:00] SPEAKER_00: Good morning everyone, shall we begin?
# → [00:04] SPEAKER_01: Yes, let's start with the agenda.
```

PYTHON



Speaker Diarisation on Android

pyannote.audio runs on a server or desktop Python environment — not on-device. For mobile, send audio to your backend, run diarisation there, and return a speaker-labeled JSON transcript. On-device speaker identification (not full diarisation) is possible with speaker embedding models under 5 MB, but full multi-speaker diarisation requires cloud offload.

Filler Word & Disfluency Removal (Pre-LLM)

Before sending text to an LLM or committing to the input field, a lightweight rule-based pass can strip filler words, hesitation markers, and verbal ticks. This runs without any model, at near-zero latency, and dramatically improves readability. Aawaaz, Phemy, and Wispr Flow all implement this layer. The key insight: fillers should be **configurable per-user** because some words are fillers in casual speech but meaningful in other contexts (e.g., "like" in "I like pizza" vs. "I was like going to the store").

```
/**                                                                 KOTLIN
 * Configurable filler word removal engine.
 * Runs before LLM or punctuation model – pure regex, <1ms latency.
 *
 * Key design decisions (learned from Aawaaz & Wispr Flow):
 * 1. Fillers are configurable – users can add/remove from settings
 * 2. Word-boundary matching prevents false positives ("like" in "I like dogs")
 * 3. Repeated fillers ("um um um") collapse to nothing, not one "um"
 * 4. Preserve sentence structure – don't create orphaned punctuation
 */
class FillerWordRemover(
    fillers: Set<String> = DEFAULT_FILLERS,
) {
    companion object {
        val DEFAULT_FILLERS = setOf(
            "um", "uh", "erm", "ah", "hmm",
            "you know", "I mean", "basically",
            "literally", "right", "so",
        )
    }

    // Sort longest-first to prevent partial matches
    private val sortedFillers = fillers.sortedByDescending { it.length }

    // Build a single regex: \b(you know|I mean|basically|um|uh)\b\s*[,\,]?\s*
```

```

private val fillerPattern: Regex = sortedFillers
    .joinToString("|") { Regex.escape(it) }
    .let { pattern →
        Regex("""\b($pattern)\b[\s,]*""", RegexOptions.IGNORE_CASE)
    }

fun clean(text: String): String =
    fillerPattern.replace(text, " ")
        .replace(Regex("""\s{2,}"""), " ")
        .trim()
}

```

Self-Correction & Backtrack Detection (Pre-LLM)

Users naturally restate and correct themselves while speaking: "Send it to John... actually, send it to Sarah." Wispr Flow's "Backtrack" feature automatically detects these corrections and keeps only the final version. Aawaaz identifies trigger phrases like "actually no", "I mean", "wait", and "scratch that". This can be implemented as a rule-based pre-LLM pass or as part of the LLM prompt system.

The **rule-based approach** handles explicit triggers with zero latency. The **LLM approach** can also detect *implicit* restatements where the user naturally restates without a trigger word — Wispr Flow claims to identify when "you naturally restate something" and keep only the corrected version. A production system should layer both: rule-based for explicit triggers (fast path), LLM for implicit corrections (quality path).

```

/**
 * Self-correction / backtrack detection engine.
 *
 * Detects explicit correction triggers and removes the preceding clause.
 * Trigger phrases (from Wispr Flow, Aawaaz, and tambourine-voice):
 * "actually" / "actually no" / "scratch that" / "wait" / "I mean" / "no
wait"
 *
 * Algorithm:
 * 1. Scan for trigger phrase
 * 2. Find the clause boundary before the trigger (sentence start, comma, or
"and"/"but")
 * 3. Remove everything from clause boundary to end of trigger phrase
 * 4. Keep text after the trigger as the correction
 *

```

KOTLIN

```

* Example: "Send it to John, actually send it to Sarah"
*      → "Send it to Sarah"
*/
object BacktrackDetector {

    private val TRIGGERS = listOf(
        "actually no",    // Aawaaz-style explicit negation
        "scratch that",  // Wispr Flow / Aawaaz / tambourine
        "no wait",       // Common spoken correction
        "actually",       // Wispr Flow primary trigger
        "wait",          // Aawaaz trigger
        "I mean",        // Natural restatement marker
    )

    // Clause boundaries that scope what gets deleted
    private val CLAUSE_BOUNDARY = Regex("""
[.!?,;]\s*|\b(and|but|so|then)\b\s*""")

    fun process(text: String): String {
        var result = text
        for (trigger in TRIGGERS) {
            val idx = result.lowercase().indexOf(trigger.lowercase())
            if (idx < 0) continue

            // Find the clause boundary before the trigger
            val beforeTrigger = result.substring(0, idx)
            val lastBoundary = CLAUSE_BOUNDARY.findAll(beforeTrigger)
                .lastOrNull()?.range?.last?.plus(1) ?: 0

            // Keep everything before clause + everything after trigger
            val afterTrigger = result.substring(idx + trigger.length).trim-
Start(' ', ' ')
            result = result.substring(0, lastBoundary).trimEnd() +
                if (afterTrigger.isNotEmpty()) " " + afterTrigger else ""
            break // Process one correction per pass; caller can loop
        }
        return result.trim()
    }
}

```



Layered Backtrack Strategy

Wispr Flow handles both explicit triggers ("scratch that") and implicit restatements (user naturally rephrases without a keyword). The implicit detection requires an LLM — add an "Advanced Prompt" instruction like: *"If the user restates or corrects themselves, keep only the final version. Remove the original phrasing and any correction markers."* (See Section 15 for the full 3-part prompt architecture.) The rule-based engine above handles the explicit triggers at <1 ms latency, while the LLM catches the rest.

Voice Punctuation & Formatting Commands

Users can dictate punctuation and formatting by speaking command words. Wispr Flow, Apple Dictation, and Google Voice Typing all support an extensive vocabulary of spoken punctuation. A **command parser** scans the raw transcript for these keywords and substitutes the corresponding symbol or formatting action.

SPOKEN COMMAND	OUTPUT	CATEGORY
"period" / "full stop"	.	Basic
"comma"	,	Basic
"question mark"	?	Basic
"exclamation mark" / "exclamation point"	!	Basic
"colon"	:	Basic
"semicolon"	;	Basic
"dash" / "hyphen"	-	Basic
"em dash"	—	Extended
"ellipsis" / "dot dot dot"	...	Extended
"open quote" / "close quote"	" / "	Quotes
"apostrophe"	'	Quotes
"open parenthesis" / "close parenthesis"	(/)	Brackets
"open bracket" / "close bracket"	[/]	Brackets
"asterisk"	*	Symbol
"ampersand"	&	Symbol
"at sign"	@	Symbol
"hashtag" / "hash" / "pound sign"	#	Symbol
"percent" / "percent sign"	%	Symbol
"dollar sign"	\$	Symbol
"slash" / "forward slash"	/	Symbol
"backslash"	\	Symbol
"pipe"		Symbol
"tilde"	~	Symbol
"caret"	^	Symbol
"degree sign"	°	Symbol
"copyright sign"	©	Symbol
"trademark"	™	Symbol
"plus sign"	+	Math
"equals sign" / "equals"	=	Math
"greater than" / "less than"	> / <	Math

SPOKEN COMMAND	OUTPUT	CATEGORY
"new line"	\n	Formatting
"new paragraph"	\n\n	Formatting

```

/**                                                                 KOTLIN
 * Voice punctuation command parser.
 * Scans raw transcript for spoken punctuation commands and substitutes sym-
 * bols.
 *
 * Design notes (learned from Wispr Flow's Smart Formatting):
 * - Commands are case-insensitive and matched on word boundaries
 * - "new line" / "new paragraph" produce actual line breaks, not literal text
 * - Numbered list detection: "one... two... three..." → "1. ... 2. ... 3. ..."
 * - Order matters: longer phrases matched first ("exclamation point" before
 * "point")
 */
object VoicePunctuationParser {

    private data class Command(
        val spoken: String,
        val replacement: String,
        val attachLeft: Boolean = true,    // No space before symbol
        val attachRight: Boolean = false, // Space after symbol
    )

    private val commands = listOf(
        // Formatting (highest priority)
        Command("new paragraph", "\n\n", attachLeft = true, attachRight =
true),
        Command("new line", "\n", attachLeft = true, attachRight = true),
        // Multi-word punctuation (match before single-word)
        Command("exclamation point", "!"),
        Command("exclamation mark", "!"),
        Command("question mark", "?"),
        Command("full stop", "."),
        Command("em dash", "-"),
        Command("open parenthesis", "(", attachLeft = false, attachRight =
true),
        Command("close parenthesis", ")"),
        Command("open quote", "\u201C", attachLeft = false, attachRight =
true),
        Command("close quote", "\u201D"),
        Command("open bracket", "[", attachLeft = false, attachRight = true),

```

```

Command("close bracket", "]"),
Command("at sign", "@", attachLeft = true, attachRight = true),
Command("pound sign", "#", attachLeft = true, attachRight = true),
Command("dollar sign", "$", attachLeft = false, attachRight = true),
Command("percent sign", "%"),
Command("forward slash", "/", attachLeft = true, attachRight = true),
Command("degree sign", "°"),
Command("copyright sign", "©"),
Command("plus sign", "+", attachLeft = true, attachRight = true),
Command("equals sign", "=", attachLeft = true, attachRight = true),
Command("greater than", ">", attachLeft = true, attachRight = true),
Command("less than", "<", attachLeft = true, attachRight = true),
// Single-word commands
Command("period", "."),
Command("comma", ","),
Command("colon", ":"),
Command("semicolon", ";"),
Command("dash", "-"),
Command("hyphen", "-"),
Command("ellipsis", "..."),
Command("apostrophe", "'", attachLeft = true, attachRight = true),
Command("asterisk", "*"),
Command("ampersand", "&"),
Command("hashtag", "#", attachLeft = true, attachRight = true),
Command("percent", "%"),
Command("slash", "/", attachLeft = true, attachRight = true),
Command("backslash", "\\ ", attachLeft = true, attachRight = true),
Command("pipe", "|", attachLeft = true, attachRight = true),
Command("tilde", "~"),
Command("caret", "^"),
Command("trademark", "™"),
Command("equals", "=", attachLeft = true, attachRight = true),
)

fun parse(text: String): String {
    var result = text
    for (cmd in commands) {
        val pattern = Regex(
            """"\s*\b${Regex.escape(cmd.spoken)}\b\s*""",
            RegexOptions.IGNORE_CASE
        )
        result = pattern.replace(result) {
            val pre = if (cmd.attachLeft) "" else " "
            val post = if (cmd.attachRight) "" else " "
            pre + cmd.replacement + post
        }
    }
}

```

```

    }
    }
    return result.replace(Regex("""\s{2,}"""), " ").trim()
}
}

```

Complete Pre-LLM Processing Pipeline

In a production voice keyboard, these pre-LLM stages run in sequence on the raw ASR output before any LLM formatting pass. The pipeline order matters: fillers first, then self-correction, then punctuation commands, then capitalization.

```

/**                                                                 KOTLIN
 * Complete pre-LLM text processing pipeline.
 * Order: fillers → backtrack → punctuation commands → capitalisation → ITN
 * Total latency budget: <5ms for typical sentence lengths.
 */
object PreLLMPipeline {

    private val fillerRemover = FillerWordRemover()
    private val backtrackDetector = BacktrackDetector
    private val punctuationParser = VoicePunctuationParser

    fun process(raw: String): String {
        var text = raw.trim()
        // Step 1: Remove filler words (um, uh, you know, basically...)
        text = fillerRemover.clean(text)
        // Step 2: Detect and apply self-corrections ("actually", "scratch
that")
        text = backtrackDetector.process(text)
        // Step 3: Parse spoken punctuation commands ("period" → ".")
        text = punctuationParser.parse(text)
        // Step 4: Basic capitalization & cleanup
        text = TranscriptPostProcessor.process(text)
        return text
    }
}

// Usage in voice keyboard:
// val raw = "um the meeting is at uh three pm actually four pm period new line

```

```
see you there"  
// val clean = PreLLMPipeline.process(raw)  
// → "The meeting is at 4 PM.\nSee you there."
```

LLM Formatting Layer ● INTERMEDIATE

Using a local or cloud LLM to apply punctuation, fix disfluencies, extract intent, and format transcripts — the final intelligence layer of the pipeline.

For voice agents and intelligent keyboards, a lightweight LLM pass after ASR can transform a raw transcript into structured, clean, actionable output. This is distinct from using an LLM for transcription itself — the acoustic model does the heavy lifting, and the LLM only processes text (token-efficient and fast).

Cloud LLM Formatting (Python)

```
import anthropicPYTHON

client = anthropic.Anthropic()

def format_transcript(raw_asr: str, context: str = "") → dict:
    """
    Clean and structure raw ASR output using Claude.
    Returns: {cleaned_text, summary, action_items}
    """
    prompt = f"""
Raw ASR transcript (may contain errors, no punctuation):
{raw_asr}

Context: {context if context else "General conversation"}

Return JSON with:
- "cleaned_text": Properly punctuated, capitalized, disfluencies removed
- "summary": One sentence summary (omit if text < 50 words)
- "action_items": List of action items if any detected, else []
- "intent": One of [DICTATION, QUESTION, COMMAND, CONVERSATION]
    """
    response = client.messages.create(
        model="claude-haiku-4-5-20251001", # Fast, cheap – perfect for text
        cleanup
        max_tokens=500,
        messages=[{"role": "user", "content": prompt}]
```

```

)
import json
return json.loads(response.content[0].text)

```

On-Device LLM Formatting (Gemma 3 via MediaPipe)

```

// Requires: com.google.mediarpipe:tasks-genai:0.10.22+
// Model: gemma-3-1b-it (.task format, ~675MB int4 quantized, fits on most
// Android 12+ devices)
import com.google.mediarpipe.tasks.genai.llminference.*

class OnDeviceLLMFormatter(context: Context) {

    private val llm = LlmInference.createFromOptions(
        context,
        LlmInference.LlmInferenceOptions.builder()
            .setModelPath("/sdcard/models/gemma-3-1b-it.task")
            .setMaxTokens(256)
            .setMaxTopK(40)
            .setTemperature(0.0f) // Deterministic formatting
            .build()
    )

    suspend fun cleanTranscript(raw: String): String =
        withContext(Dispatchers.Default) {
            val prompt = """Fix punctuation and capitalization. Return only the
corrected text.
Raw: $raw
Corrected:"""
            llm.generateResponse(prompt).trim()
        }
}

```

Multi-Part Prompt Architecture (tambourine-voice Pattern)

The open-source **tambourine-voice** (<https://github.com/kstonekuan/tambourine-voice>) project pioneered a powerful **three-part prompt system** that separates concerns for dictation LLM processing. This architecture is more maintainable and configurable than a single monolithic prompt. Aawaaz and Phemy adopted variations of the same pattern.

PROMPT PART	RESPONSIBILITY	WHEN IT RUNS
Main Prompt	Filler removal, punctuation, capitalization, sentence combining	Every transcription
Advanced Prompt	Backtrack corrections, list formatting, implicit re-statement detection	Every transcription (appended to Main)
Dictionary Prompt	Custom word mappings, technical terms, proper nouns, user corrections	When personal dictionary is non-empty

KOTLIN

```

/**
 * Multi-part prompt builder for LLM formatting.
 * Inspired by tambourine-voice's 3-part system, adapted for on-device
 * Qwen/Gemma.
 *
 * Why 3 parts instead of 1:
 * - Main prompt is stable and rarely changes
 * - Advanced prompt can be toggled (backtrack on/off)
 * - Dictionary prompt is per-user and dynamically generated
 * - Smaller, focused prompts reduce LLM confusion and hallucination
 */
class DictationPromptBuilder(
    private val personalDictionary: Map<String, String> = emptyMap(),
    private val enableBacktrack: Boolean = true,
) {
    // — Part 1: Main Prompt (always active) —
    private val mainPrompt = """
You are a dictation cleanup assistant. Clean raw speech-to-text output:
1. Add proper punctuation (periods, commas, question marks)
2. Fix capitalization (sentences, proper nouns)
3. Remove filler words (um, uh, like, you know, basically)
4. Combine run-on fragments into proper sentences
5. Preserve the speaker's exact meaning – never add or change content
Return ONLY the cleaned text, nothing else.
"""

    // — Part 2: Advanced Prompt (backtrack + formatting) —
    private val advancedPrompt = """
Additional rules:
- If the user corrects themselves (says "actually", "I mean", "scratch that",
  "wait", or naturally restates something), keep ONLY the corrected version.
  Remove the original phrasing and any correction markers.
- If the user dictates a numbered list ("one... two... three..."), format it
  as a proper numbered list with line breaks.
- If the user says "new line" or "new paragraph", insert the appropriate

```

```

    line break – do not write the words literally.
    """

    // — Part 3: Dictionary Prompt (per-user, dynamic) —
    private fun buildDictionaryPrompt(): String {
        if (personalDictionary.isEmpty()) return ""
        val entries = personalDictionary.entries.joinToString("\n") { (from,
to) →
            "- \"$from\" → \"$to\""
        }
        return ""
    }
    Personal dictionary – always apply these corrections:
    $entries
    """
}

fun buildPrompt(rawTranscript: String): String = buildString {
    append(mainPrompt)
    if (enableBacktrack) append(advancedPrompt)
    append(buildDictionaryPrompt())
    append("\n\nRaw transcript:\n$rawTranscript\n\nCleaned:")
}
}

```

Context-Aware Tone Matching

Aawaaz introduced the concept of **per-app tone matching**: dictation in an email client should produce formal prose, while dictation in a chat app should feel casual and concise. This is achieved by detecting the foreground app's package name and injecting a tone instruction into the LLM prompt. Wispr Flow hints at similar behavior but keeps the algorithm proprietary.

APP CATEGORY	TONE	FORMATTING RULES	EXAMPLE PACKAGES
Email	Formal	Full sentences, proper grammar, greetings/sign-offs preserved	Gmail, Outlook, Yahoo Mail
Chat / Messaging	Casual	Short sentences, minimal punctuation, lower-case acceptable	WhatsApp, Telegram, Messages
Code Editors	Technical	Preserve technical jargon, camelCase, code formatting	Termux, QuickEdit, Dcoder
Documents	Structured	Paragraphs, headings, formal structure	Google Docs, Word, Notes
Terminal / Shell	Verbatim	Minimal processing, preserve exact wording	Termux, Terminal Emulator
Social Media	Casual	Hashtags preserved, emoji-friendly, concise	X, Instagram, Reddit

```

/**
 * Context-aware tone matching for LLM formatting.
 * Detects the foreground app and injects a tone instruction.
 *
 * On Android, the foreground package is available from InputMethodService via:
 *   currentInputEditorInfo.packageName
 */
enum class DictationTone(
    val instruction: String,
) {
    FORMAL("Write in professional, formal tone. Use complete sentences and proper grammar."),
    CASUAL("Write casually. Short sentences, relaxed grammar. No need for formality."),
    TECHNICAL("Preserve technical terms exactly. Use camelCase, code syntax, and jargon as-is."),
    STRUCTURED("Use proper paragraph breaks. Organize into clear sections if appropriate."),
    VERBATIM("Minimal processing. Preserve the speaker's exact words. Only fix obvious errors."),
}

object ToneMatcher {

    private val packageToTone = mapOf(
        // Email clients
        "com.google.android.gm" to DictationTone.FORMAL,
        "com.microsoft.office.outlook" to DictationTone.FORMAL,
        // Chat / messaging

```

KOTLIN

```

        "com.whatsapp" to DictationTone.CASUAL,
        "org.telegram.messenger" to DictationTone.CASUAL,
        "com.google.android.apps.messaging" to DictationTone.CASUAL,
        // Code editors
        "com.termux" to DictationTone.TECHNICAL,
        "com.rhmssoft.edit" to DictationTone.TECHNICAL,
        // Document editors
        "com.google.android.apps.docs" to DictationTone.STRUCTURED,
        "com.microsoft.office.word" to DictationTone.STRUCTURED,
    )

    fun detectTone(packageName: String?): DictationTone =
        packageToTone[packageName] ?: DictationTone.FORMAL // Safe default
}

```

Prompt Mode Patterns (Phemy's 8-Mode System)

Phemy implements eight distinct formatting prompts that users can switch between manually. This is the most granular approach — useful for power users who know what formatting they want. In practice, combine this with auto-detection (tone matching) as the default, allowing manual override.

MODE	BEHAVIOR	USE CASE
Clean	Standard cleanup: punctuation, capitalization, filler removal	General dictation (default)
Technical	Preserve jargon, acronyms, camelCase; add code formatting	Developer notes, documentation
Formal	Professional grammar, complete sentences, formal vocabulary	Business emails, reports
Casual	Relaxed grammar, contractions, minimal punctuation	Messaging, social media
Code	Convert natural language to code or pseudocode where possible	Programming by voice
Verbatim	Minimal changes — punctuation only, preserve every word	Legal transcription, quotes
Raw	No LLM processing at all — direct ASR output	Debugging, ASR quality testing
Custom	User-defined system prompt stored in settings	Domain-specific workflows

On-Device LLM Model Selection: Qwen3 vs Gemma 3

For on-device dictation formatting, two model families dominate in 2026: Google's **Gemma 3** (via MediaPipe) and Alibaba's **Qwen3** (via llama.cpp or ONNX). Both run sub-2-second inference on modern mobile SoCs. Aawaaz and Phemy both chose Qwen3 for macOS; the Android ecosystem has better Gemma integration via MediaPipe, but Qwen3 via llama.cpp is fully viable.

MODEL	SIZE (INT4)	TOKENS/SEC (MOBILE)	DICTATION QUALITY	INTEGRATION PATH
Gemma 3 1B-IT	675 MB	30–50 t/s	Good — strong punctuation, weak on jargon	MediaPipe LLM Inference API (native Android)
Qwen3 0.6B	400 MB	40–60 t/s	Adequate — fast, good for simple cleanup	llama.cpp / ONNX Runtime
Qwen3 1.7B	1.1 GB	25–40 t/s	Good — balanced speed/quality	llama.cpp / ONNX Runtime
Qwen3 4B	2.5 GB	12–20 t/s	Excellent — best dictation quality on-device	llama.cpp / ONNX Runtime
Phi-4-mini (1.3B)	900 MB	20–35 t/s	Very good — strong reasoning, compact	ONNX Runtime / MediaPipe



Qwen3 "Non-Thinking" Mode

Qwen3 supports a `/no_think` mode that bypasses the chain-of-thought reasoning phase. For dictation formatting — where the task is well-defined and doesn't need complex reasoning — non-thinking mode reduces first-token latency by 30–50% with no quality loss. Aawaaz uses this mode by default for all dictation formatting. Add `enable_thinking=False` when using the Qwen3 API, or strip `<think> ... </think>` tags if they appear.

```
/**                                                                 KOTLIN
 * On-device LLM formatting with Qwen3 via llama.cpp JNI.
 * Uses non-thinking mode for lowest latency.
 *
 * Performance (Snapdragon 8 Gen 3, int4 quantized):
 *   Qwen3 0.6B: ~50ms first token, ~40 t/s sustained
 *   Qwen3 1.7B: ~120ms first token, ~30 t/s sustained
 *   Qwen3 4B:  ~250ms first token, ~15 t/s sustained
 */
class Qwen3DictationFormatter(
    private val llamaContext: Long, // From llama.cpp JNI init
    private val promptBuilder: DictationPromptBuilder,
    private val toneMatcher: ToneMatcher = ToneMatcher,
) {
    /**
     * Format raw transcript with tone awareness.
     * Total latency: 80-400ms depending on model size and text length.
     */
    suspend fun format(
        rawTranscript: String,
        foregroundPackage: String?,
    ): String = withContext(Dispatchers.Default) {
        val tone = toneMatcher.detectTone(foregroundPackage)
        val prompt = promptBuilder.buildPrompt(rawTranscript)
        val fullPrompt = "${tone.instruction}\n\n$prompt"

        // llama.cpp inference - non-thinking mode (no <think> tags)
        LlamaCpp.generate(llvmContext, fullPrompt, maxTokens = 256)
            .replace(Regex("""<think>.*?</think>""",
RegexOption.DOT_MATCHES_ALL), "")
            .trim()
    }
}
```

Android — Full Stack Implementation ● INTERMEDIATE

The complete Android STT stack: audio capture → Silero VAD → *whisper.cpp* or Moonshine → post-processing → UI update, all wired together with Kotlin coroutines.

This section assembles all previous layers into a deployable Android STT pipeline. Two integration paths are shown: *whisper.cpp* via JNI (best for multilingual apps and broad Whisper model support) and Moonshine via ONNX Runtime (best for low-latency English apps).

Path A: *whisper.cpp* via JNI

whisper.cpp provides an official Android example at `examples/whisper.android` in the [ggml-org/whisper.cpp](https://github.com/ggml-org/whisper.cpp) (<https://github.com/ggml-org/whisper.cpp>) repository. The JNI bridge is implemented in `jni.c` and exposes a `WhisperLib` Kotlin companion object.

```
// WhisperLib.kt – Kotlin JNI bridge to whisper.cpp
// Matches the official whisper.android example structure
KOTLIN

class WhisperLib {
    companion object {
        init { System.loadLibrary("whisper") }

        // Initialize whisper context from model file in assets
        external fun initContextFromAsset(mgr: AssetManager, assetPath:
String): Long
        external fun initContextFromFile(modelPath: String): Long
        external fun freeContext(context: Long)

        // Full transcription – blocks until complete
        external fun fullTranscribe(context: Long, numThreads: Int,
            audioData: FloatArray): String

        // System info (shows NEON, METAL flags on device)
        external fun getSystemInfo(): String
    }
}
```

```

// Usage in ViewModel:
class STTViewModel(app: Application) : AndroidViewModel(app) {

    private var whisperCtx: Long = 0L
    val transcript = MutableStateFlow("")

    init {
        viewModelScope.launch(Dispatchers.IO) {
            // Copy model from assets to internal storage (required for large
files)
            val modelFile = copyAssetToCache("ggml-small.bin")
            whisperCtx = WhisperLib.initContextFromFile(modelFile.absolutePath)
        }
    }

    fun transcribe(audio: FloatArray) {
        viewModelScope.launch(Dispatchers.Default) {
            if (whisperCtx == 0L) return@launch
            val result = WhisperLib.fullTranscribe(
                whisperCtx, numThreads = 4, audioData = audio
            )
            transcript.emit(TranscriptPostProcessor.process(result))
        }
    }

    override fun onCleared() {
        if (whisperCtx != 0L) WhisperLib.freeContext(whisperCtx)
    }

    private fun copyAssetToCache(name: String): File {
        val dest = File(getApplication<Application>().cacheDir, name)
        if (!dest.exists()) {
            getApplication<Application>().assets.open(name).use { input →
                dest.outputStream().use { input.copyTo(it) }
            }
        }
        return dest
    }
}

```

whisper.cpp Model Download & Build

```
# Clone whisper.cpp
git clone https://github.com/ggml-org/whisper.cpp
cd whisper.cpp

# Download a model (tiny, base, small, medium, large-v3, large-v3-turbo)
sh ./models/download-ggml-model.sh small.en
# → Saves to models/ggml-small.en.bin (~244MB)

# Copy model to Android example assets
mkdir -p examples/whisper.android/app/src/main/assets/models
cp models/ggml-small.en.bin
examples/whisper.android/app/src/main/assets/models/

# Open examples/whisper.android in Android Studio
# Build variant: release (required for proper CMake build)
# Min SDK: 24 (API 24 = Android 7.0)
```

Path B: Moonshine via ONNX Runtime (Low-Latency English)

```
dependencies {
    // ONNX Runtime Android – core inference engine
    implementation("com.microsoft.onnxruntime:onnxruntime-android:1.27.0")
    // OkHttp for model download (if not bundling in APK)
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
}

android {
    defaultConfig {
        // ONNX Runtime requires ABI filters
        ndk { abiFilters += listOf("arm64-v8a", "x86_64") }
    }
}
```

Complete Wired Pipeline

```
/**
 * Complete STT pipeline wired with Kotlin Flow.
 * Capture → VAD → Buffer → Whisper/Moonshine → Post-process → UI
 */
class STTPipeline(
    private val context: Context,
```

```

    private val onPartialTranscript: (String) → Unit,
    private val onFinalTranscript: (String) → Unit
) {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Main)
    private val capture = AudioCaptureManager(scope)
    private val vad = SileroVAD(context)
    private val stt = MoonshineSTT(context) // or WhisperLib wrapper
    private val postproc = TranscriptPostProcessor

    // VAD state machine
    private val speechBuffer = mutableListOf<FloatArray>()
    private var isSpeaking = false
    private var silenceFrames = 0
    private val SILENCE_THRESHOLD_FRAMES = 16 // ~500ms of silence @
32ms/frame

    fun start() {
        capture.start()
        scope.launch(Dispatchers.IO) {
            capture.frames.collect { frame →
                val speechProb = vad.predict(frame)

                when {
                    speechProb > 0.5f → {
                        speechBuffer.add(frame)
                        isSpeaking = true
                        silenceFrames = 0
                    }
                    isSpeaking → {
                        speechBuffer.add(frame)
                        silenceFrames++
                        if (silenceFrames ≥ SILENCE_THRESHOLD_FRAMES) {
                            processUtterance()
                        }
                    }
                }
            }
        }
    }

    private suspend fun processUtterance() {
        if (speechBuffer.size < 8) { // Skip utterances < 240ms
            speechBuffer.clear(); isSpeaking = false; silenceFrames = 0
            return
        }
    }
}

```

```

        val audio = speechBuffer.flatMap { it.toList() }.toFloatArray()
        speechBuffer.clear(); isSpeaking = false; silenceFrames = 0; vad.reset-
State()

        val rawText = stt.transcribe(audio)
        val cleanText = postproc.process(rawText)
        withContext(Dispatchers.Main) { onFinalTranscript(cleanText) }
    }

    fun stop() { capture.stop(); scope.cancel() }
}

```



Never run inference on the main thread.

Whisper Tiny takes ~200 ms per utterance on a Pixel 7. If you call `transcribe()` on the UI thread, Android will trigger an ANR after 5 seconds of accumulated blocking. Always dispatch inference to `Dispatchers.Default` or a dedicated `newSingleThreadContext("stt")`.

iOS Implementation

● INTERMEDIATE

AVAudioEngine setup, Moonshine on iOS via WhisperKit, and the key differences from Android audio APIs.

iOS audio for STT uses `AVAudioEngine`, Apple's modern audio graph API. It's considerably higher-level than Android's `AudioRecord` — you tap into the input node's output buffer rather than managing a raw buffer directly. The model integration story is simpler too: WhisperKit (<https://github.com/argmaxinc/WhisperKit>) (Swift, on-device Whisper) and Moonshine's iOS SDK provide batteries-included integration.

AVAudioEngine Capture (Swift)

```
import AVFoundation SWIFT

class AudioCaptureEngine {
    private let engine = AVAudioEngine()
    var onChunk: ([[Float]] → Void)?

    func start() throws {
        let inputNode = engine.inputNode
        let format = AVAudioFormat(
            commonFormat: .pcmFormatFloat32,
            sampleRate: 16000,
            channels: 1,
            interleaved: false
        )!

        inputNode.installTap(onBus: 0, bufferSize: 480, format: format) { buffer, _ in
            guard let data = buffer.floatChannelData?[0] else { return }
            let samples = Array(UnsafeBufferPointer(start: data, count:
                Int(buffer.frameLength)))
            self.onChunk?(samples)
        }
    }
}
```

```

        engine.prepare()
        try engine.start()
    }

    func stop() {
        engine.inputNode.removeTap(onBus: 0)
        engine.stop()
    }
}

```

WhisperKit Integration (Swift)

```

// Swift Package Manager: https://github.com/argmaxinc/WhisperKit
import WhisperKit

class WhisperSTT {
    private var whisper: WhisperKit?

    func setup() async throws {
        whisper = try await WhisperKit(model: "openai_whisper-small")
    }

    func transcribe(audioPath: String) async throws → String {
        let results = try await whisper!.transcribe(audioPath: audioPath)
        return results.compactMap { $0.text }.joined(separator: " ")
    }
}

```

SWIFT

Moonshine iOS via ONNX Runtime (Swift)

Download `preprocess.onnx`, `encode.onnx`, and `uncached_decode.onnx` from [usefulsensors/moonshine-tiny](https://huggingface.co/usefulsensors/moonshine-tiny) (<https://huggingface.co/usefulsensors/moonshine-tiny>) on HuggingFace and bundle them in your Xcode target. ONNX Runtime iOS is distributed as a CocoaPod.

```

platform :ios, '14.0'
pod 'onnxruntime-mobile-objc', '~> 1.27.0'

```

SWIFT (PODFILE)

```

// MoonshineONNX.swift – minimal greedy-decode wrapper
import Foundation
import onnxruntime_objc

class MoonshineONNX {
    private let env: ORTEnvironment
    private let preprocess: ORTSession
    private let encoder: ORTSession
    private let decoder: ORTSession

    init() throws {
        env = try ORTEnvironment(loggingLevel: .warning)
        let opts = try ORTSessionOptions()
        try opts.setIntraOpNumThreads(2)
        let bundle = Bundle.main
        preprocess = try ORTSession(
            env: env,
            modelPath: bundle.path(forResource: "preprocess", ofType: "onnx")!,
            sessionOptions: opts)
        encoder = try ORTSession(
            env: env,
            modelPath: bundle.path(forResource: "encode", ofType: "onnx")!,
            sessionOptions: opts)
        decoder = try ORTSession(
            env: env,
            modelPath: bundle.path(forResource: "uncached_decode", ofType:
"onnx")!,
            sessionOptions: opts)
    }

    func transcribe(pcm: [Float]) throws → [Int64] {
        let pcmData = pcm.withUnsafeBytes { Data($0) }
        let audioTensor = try ORTValue(
            tensorData: NSMutableData(data: pcmData),
            elementType: .float,
            shape: [1, NSNumber(value: pcm.count)])

        // 1. Preprocess: raw audio → features
        let preOut = try preprocess.run(
            withInputs: ["audio": audioTensor],
            outputNames: ["features"],
            runOptions: nil)
        let features = preOut["features"]!

        // 2. Encode: features → context
    }

```

```

    let featuresLenData = Data(bytes:
[Int64(features.tryGetTensorShapeAndType().shape[1])], count: 8)
    let featuresLenTensor = try ORTValue(
        tensorData: NSMutableData(data: featuresLenData),
        elementType: .int64,
        shape: [1])
    let encOut = try encoder.run(
        withInputs: ["features": features,
                     "features_len": featuresLenTensor],
        outputNames: ["encoder_out"],
        runOptions: nil)
    let encoderOut = encOut["encoder_out"]!

    // 3. Decode: autoregressive token generation
    var tokenIds: [Int64] = [1] // BOS token
    for _ in 0..<200 {
        let tokData = tokenIds.withUnsafeBytes { Data($0) }
        let tokTensor = try ORTValue(
            tensorData: NSMutableData(data: tokData),
            elementType: .int64,
            shape: [1, NSNumber(value: tokenIds.count)])
        let seqLenData = Data(bytes: [Int64(tokenIds.count)], count: 8)
        let seqLenTensor = try ORTValue(
            tensorData: NSMutableData(data: seqLenData),
            elementType: .int64,
            shape: [1])

        let decOut = try decoder.run(
            withInputs: ["tokens": tokTensor,
                        "encoder_out": encoderOut,
                        "seq_len": seqLenTensor],
            outputNames: ["logits"],
            runOptions: nil)

        let logits = try decOut["logits"]!.tensorData() as Data
        let floats = logits.withUnsafeBytes {
            Array(UnsafeBufferPointer<Float>(
                start: $0.bindMemory(to: Float.self).baseAddress!,
                count: $0.count / 4))
        }
        let vocabSize = floats.count / tokenIds.count
        let lastLogits = Array(floats.suffix(vocabSize))
        let nextToken = Int64(lastLogits.indices.max {
            lastLogits[$0] < lastLogits[$1]
        } ?? 2)
    }

```

```

        if nextToken == 2 { break } // EOS
        tokenIds.append(nextToken)
    }
    return tokenIds
}
}

```

Real-Time iOS Pipeline: AVAudioEngine + VAD + Moonshine

```

// Continuous listening: engine → VAD → Moonshine, all on a background queueSWIFT
func startListening() throws {
    let session = AVAudioSession.sharedInstance()
    try session.setCategory(.record, mode: .measurement,
                           options: .duckOthers)
    try session.setPreferredSampleRate(16_000)
    try session.setActive(true)

    let format = engine.inputNode.outputFormat(forBus: 0)
    engine.inputNode.installTap(onBus: 0, bufferSize: 1600,
                                format: format) { buffer, _ in
        guard let data = buffer.floatChannelData?[0] else { return }
        let samples = Array(UnsafeBufferPointer(start: data,
                                                  count: Int(buffer.frameLength)))
        self.vadBuffer.append(contentsOf: samples) // 100 ms window

        if self.vad.isSpeech(self.vadBuffer) {
            self.speechBuffer.append(contentsOf: samples)
        } else if !self.speechBuffer.isEmpty {
            let segment = self.speechBuffer
            self.speechBuffer = []
            DispatchQueue.global(qos: .userInitiated).async {
                if let tokens = try? self.moonshine.transcribe(pcm: segment),
                    let text = self.tokenizer.decode(tokens) {
                    DispatchQueue.main.async { self.onTranscript(text) }
                }
            }
        }
        self.vadBuffer = []
    }
    try engine.start()
}

```



iOS Audio Session Gotchas

Always call `setCategory(.record, mode: .measurement)` — not `.voiceChat` — for STT. Voice chat applies automatic echo cancellation and AGC that distort the signal. Set preferred sample rate to 16 kHz before starting; the hardware may still deliver 44.1 kHz on older devices — detect with `inputNode.outputFormat(forBus:0).sampleRate` and resample if needed with `AVAudioConverter`.

Cloud Architecture

● BEGINNER

WebSocket streaming to cloud STT providers, self-hosted Whisper via faster-whisper, and hybrid on-device/cloud fallback patterns.

Cloud STT is the right choice when accuracy is paramount and latency/privacy constraints are loose. The 2026 landscape has consolidated around Deepgram (best real-time latency), AssemblyAI (best async quality), and self-hosted faster-whisper (best cost at scale).

faster-whisper Server (Python)

```
# pip install faster-whisper uvicorn fastapi python-multipart
from faster_whisper import WhisperModel
from fastapi import FastAPI, UploadFile
import io, tempfile, soundfile as sf

# Load once at startup - float16 on GPU, int8 on CPU
model = WhisperModel("large-v3-turbo", device="cuda", compute_type="float16")

app = FastAPI()

@app.post("/transcribe")
async def transcribe(file: UploadFile):
    audio_bytes = await file.read()
    with tempfile.NamedTemporaryFile(suffix=".wav") as tmp:
        tmp.write(audio_bytes)
        tmp.flush()
        segments, info = model.transcribe(
            tmp.name,
            beam_size=5,
            vad_filter=True,          # Built-in Silero VAD in faster-whisper
            language=None             # Auto-detect language
        )
```

PYTHON

```

    )
    text = " ".join(s.text for s in segments)
    return {"text": text, "language": info.language, "duration": info.duration}

```

Cloud API Comparison (2026)

PROVIDER	REAL-TIME LATENCY	WER (EN)	PRICE/HOUR	OFFLINE?
Deepgram Nova-3	~120ms	~5%	\$0.46	✗
AssemblyAI Universal-3.5 Pro	~300ms	~4%	\$0.21	✗
Google STT v2	~200ms	~6%	\$0.96	✗
OpenAI Whisper API	N/A (batch)	~3%	\$0.36 flat	✗
Self-hosted faster-whisper Large-v3-Turbo	~150ms	~3%	GPU cost only	✓

Kotlin WebSocket Streaming Client (OkHttp)

Stream raw PCM chunks to your transcription server over WebSocket for sub-500 ms first-word latency. The server can use any STT backend (faster-whisper, Deepgram, Azure Speech) and push back incremental transcript JSON.

```
implementation("com.squareup.okhttp3:okhttp:4.12.0")
```

KOTLIN (BUILD.GRADLE)

```

class StreamingSTTClient(
    private val serverUrl: String,
    private val onTranscript: (String, Boolean) → Unit,
) {
    private val client = OkHttpClient.Builder()
        .readTimeout(0, TimeUnit.MILLISECONDS) // No timeout for streaming
        .build()

    private var websocket: WebSocket? = null

    fun connect(authToken: String) {
        val request = Request.Builder()
            .url(serverUrl)
            .header("Authorization", "Bearer $authToken")
            .build()
        websocket = client.newWebSocket(request, object : WebSocketListener() {
            override fun onMessage(ws: WebSocket, text: String) {

```

KOTLIN

```

        val json = Json.decodeFromString<TranscriptChunk>(text)
        onTranscript(json.text, json.isFinal)
    }
})
}

/** Send a 20 ms PCM-16 chunk (640 bytes at 16 kHz mono) */
fun sendAudio(pcm16: ByteArray) {
    websocket?.send(ByteString.of(*pcm16))
}

/** Signal end-of-utterance so the server flushes remaining buffers */
fun endOfSpeech() { websocket?.send("EOS") }

fun close() { websocket?.close(1000, "done") }
}

```

Deepgram Real-Time WebSocket (Python Server)

Deepgram's Nova-3 model achieves <300 ms latency with interim results. The `endpointing` parameter lets the cloud VAD handle silence detection.

```

# pip install websockets pyaudio
import asyncio, websockets, json, pyaudio

DEEPGRAM_URL = (
    "wss://api.deepgram.com/v1/listen"
    "?model=nova-3&language=en&sample_rate=16000"
    "&channels=1&encoding=linear16&endpointing=300"
)

async def stream_to_deepgram(api_key: str):
    headers = {"Authorization": f"Token {api_key}"}
    async with websockets.connect(DEEPGRAM_URL, extra_headers=headers) as ws:
        pa = pyaudio.PyAudio()
        stream = pa.open(
            format=pyaudio.paInt16, channels=1, rate=16_000,
            input=True, frames_per_buffer=3200, # 200 ms chunks
        )
        async def send():
            while True:
                await ws.send(stream.read(3200, exception_on_overflow=False))
                await asyncio.sleep(0)

```

```

    async def recv():
        async for msg in ws:
            r = json.loads(msg)
            if r.get("type") == "Results":
                alt = r["channel"]["alternatives"][0]
                if alt["transcript"]:
                    flag = "✓" if r["is_final"] else "..."
                    print(f"
[{{flag}}] {alt['transcript']}", end="")
                await asyncio.gather(send(), recv())

asyncio.run(stream_to_deepgram("YOUR_DEEPGRAM_API_KEY"))

```

Hybrid On-Device / Cloud Fallback (Kotlin)

Run Moonshine on-device for short utterances (low cost, private). Automatically escalate to cloud for utterances longer than 15 s or when the device thermal state is throttled.

```

class HybridSTTEngine(
    private val onDevice: MoonshineAndroid,
    private val cloud: StreamingSTTClient,
) {
    suspend fun transcribe(
        pcm: FloatArray,
        durationSec: Float,
        thermalStatus: Int, // PowerManager.THERMAL_STATUS_*
    ): String {
        val useCloud = durationSec > 15f ||
            thermalStatus ≥ PowerManager.THERMAL_STATUS_MODERATE
        return if (useCloud) {
            cloud.connect(BuildConfig.STT_TOKEN)
            cloud.sendAudio(pcm.toByteArray())
            cloud.endOfSpeech()
            /* await cloud transcript via callback */
            ""
        } else {
            onDevice.transcribe(pcm)
        }
    }
}

```

**Cloud ≠ always better.**

Cloud STT providers (Google, AWS, Azure, Deepgram) achieve the lowest absolute WER on benchmark data, but they add 100–500 ms of network latency, require an active internet connection, and incur per-minute API costs. For interactive voice features (dictation, voice commands), on-device inference almost always delivers a better *user-perceived* experience. Reserve cloud fallback for long-form transcription or thermal throttling.

Edge & On-Device Optimization ● EXPERT

INT8/INT4 quantization, ONNX Runtime optimization, NNAPI acceleration, thread tuning, and thermal management for sustained mobile inference.

On-device STT performance is won in the optimization layer. A Moonshine Base model can run at RTFx 12 on a Pixel 6 with no optimization — or RTFx 35 with NNAPI acceleration, INT8 quantization, and proper thread pinning. This section covers every knob.

INT8 Quantization (ONNX)

```
# Quantize ONNX model to INT8 - reduces size ~4x, speeds inference ~2x
from onnxruntime.quantization import quantize_dynamic, QuantType

quantize_dynamic(
    model_input="encode.onnx",
    model_output="encode_int8.onnx",
    weight_type=QuantType.QInt8,
    per_channel=True,          # Per-channel is more accurate than per-tensor
    reduce_range=False        # Set True for AVX2/AVX512 CPUs only
)
# Result: encode.onnx 30.1 MB → encode_int8.onnx ~8 MB (tiny)
# WER regression: typically 0.1-0.5% on LibriSpeech clean
```

PYTHON

ONNX Runtime Session Options for Android

```
import ai.onnxruntime.*
import java.util.EnumSet

fun createOptimizedSession(env: OrtEnvironment, modelBytes: ByteArray):
OrtSession {
    val opts = OrtSession.SessionOptions().apply {
        // Use all physical cores for inference (not hyperthreading)
        setIntraOpNumThreads(Runtime.getRuntime().availableProcessors() / 2)
        setInterOpNumThreads(1) // Usually 1 for mobile
    }
```

KOTLIN

```

// NNAPI execution provider (Qualcomm/MediaTek NPU when available)
addNnapi(EnumSet.of(NNAPIFlags.USE_FP16, NNAPIFlags.USE_NCHW))

// Graph optimization level (ORT_ENABLE_ALL = fuse ops, constant folding, etc.)
setOptimizationLevel(OrtSession.SessionOptions.OptLevel.ALL_OPT)

// Memory pattern optimization (reuse buffers across inference calls)
setMemoryPatternOptimization(true)
}
return env.createSession(modelBytes, opts)
}

```

Thermal Throttle Detection

```

// API 29+ – Listen for thermal state changes KOTLIN
val powerManager = getSystemService(PowerManager::class.java)

powerManager.addThermalStatusListener(mainExecutor) { status →
    when (status) {
        PowerManager.THERMAL_STATUS_NONE,
        PowerManager.THERMAL_STATUS_LIGHT → setSTTQuality("high")
        PowerManager.THERMAL_STATUS_MODERATE → setSTTQuality("medium") //
tiny model
        PowerManager.THERMAL_STATUS_SEVERE,
        PowerManager.THERMAL_STATUS_CRITICAL → setSTTQuality("cloud") //
fallback
        else → setSTTQuality("medium")
    }
}

```



Adaptive model switching

Production voice apps should have three tiers: (1) on-device full model (normal operation), (2) on-device tiny model (thermal throttle or low battery), (3) cloud API fallback (thermal critical or no cached model). Implement tier switching in your STT manager and transition transparently — users should never see the switch, only notice continuous transcription.

whisper.cpp Deep Dive ● EXPERT

ggml internals, GGML model format, quantization levels, CMake build for Android, and performance tuning.

whisper.cpp (<https://github.com/ggml-org/whisper.cpp>) (ggml-org) is the most widely-deployed on-device Whisper implementation. Written in pure C/C++ with optional SIMD (ARM NEON, AVX2, Metal), it runs on Android, iOS, Linux, Windows, macOS, and WebAssembly without Python or torch dependencies. It is the right choice when you need the broadest Whisper model support (tiny through large-v3-turbo) in a single portable library.

GGML Model Quantization Levels

QUANTIZATION	BITS/WEIGHT	SIZE VS FP16	WER DELTA	RECOMMENDED FOR
F32	32	2×	Baseline	Reference only
F16	16	1×	0%	Desktop, server GPU
Q8_0	8	0.5×	~0.1%	Mobile — best accuracy/size balance
Q5_0	5	0.31×	~0.3%	Tight memory budget
Q4_0	4	0.25×	~0.7%	IoT / wearables
Q2_K	2	0.13×	~3–5%	Extreme edge only

```
# Convert HuggingFace Whisper to GGML + quantize
# Step 1: Convert HuggingFace Whisper to GGML format
# Usage: convert-h5-to-ggml.py dir_model path-to-whisper-repo dir-output [use-f32]
python3 models/convert-h5-to-ggml.py openai/whisper-small ./whisper ./models

# Step 2: Quantize to Q8_0 (recommended for mobile)
# Script outputs ggml-model.bin (f16) by default
./build/bin/whisper-quantize models/ggml-model.bin models/ggml-model-q8_0.bin q8_0

# Verify the quantized model works:
```

BASH

```
./build/bin/whisper-cli -m models/ggml-model-q8_0.bin -f samples/jfk.wav
# Expected output: "And so my fellow Americans ask not what your country can do
for you..."
```

Android CMakeLists Integration

```
# Add whisper.cpp as a CMake subdirectory in your Android project
cmake_minimum_required(VERSION 3.22.1)
project(MyVoiceApp)

# Add whisper.cpp (git submodule or FetchContent)
add_subdirectory(whisper.cpp)

# Your JNI bridge
add_library(whisper-jni SHARED
    src/main/cpp/jni_bridge.cpp
)

target_link_libraries(whisper-jni
    whisper          # The whisper.cpp static library
    android log
    OpenSLES         # For optional audio playback
)

# ARM NEON SIMD is mandatory and auto-detected on arm64-v8a (AArch64)
# No special compile flags needed - CMake/clang enable NEON by default
# For armeabi-v7a (32-bit ARM), add: -mfpu=neon-fp-armv8 -mfloat-abi=softfp
```

Thread Configuration & SIMD Tuning

Inference speed scales well up to 4 threads on mobile; above that, cache-coherency overhead negates the gain. ARM NEON SIMD (enabled by default on arm64-v8a) provides the most significant per-core throughput boost. Set `n_threads` based on physical core count, not logical — always test on the actual target device.

THREADS	TIME TO DECODE 5 S CLIP (PIXEL 7, WHISPER-BASE.EN Q8)	NOTE
1	~1,800 ms	Single core
2	~970 ms	Dual-core near-linear
4	~540 ms	Sweet spot – 3.3× speedup
8	~560 ms	Marginal gain; increases heat

```
// Optimal JNI-side parameter initialisation for Android
struct whisper_full_params params =
    whisper_full_default_params(WHISPER_SAMPLING_GREEDY);

// Thread tuning – cap at 4 for mobile thermal headroom
params.n_threads = min(4, android_getCpuCount());
params.n_max_text_ctx = 224;    // Decoder context limit

// Streaming mode – disable features that increase latency
params.single_segment = true;    // Force one segment per call
params.no_timestamps = true;    // Skip timestamp token generation
params.print_realtime = false;   // No stdout progress output
params.print_progress = false;

// Language
params.language = "en";
params.translate = false;

// Temperature fallback – disable for deterministic, lower-latency decoding
params.temperature_inc = 0.0f;
params.greedy.best_of = 1;

// Run transcription
int ret = whisper_full(ctx, params, samples, n_samples);
if (ret != 0) { /* handle error */ }
```

Streaming with keep_ms

For low-latency voice input, use chunked processing: feed 1–3 s windows and keep the last 200 ms of each window as context for the next call. This avoids splitting words at chunk boundaries without running a full encoder pass on the full utterance.

```

// windowed streaming – keep 200 ms of audio at start of each new chunk
#define SAMPLE_RATE    16000
#define KEEP_MS        200
#define STRIDE_MS      1500          // process 1.5 s per call

static float s_keep[SAMPLE_RATE * KEEP_MS / 1000]; // 3200 samples
static int   s_keep_n = 0;

void process_chunk(struct whisper_context *ctx, const float *new_pcm, int n) {
    // Prepend previous-window tail for continuity
    float window[SAMPLE_RATE * (KEEP_MS + STRIDE_MS) / 1000];
    memcpy(window, s_keep, s_keep_n * sizeof(float));
    memcpy(window + s_keep_n, new_pcm, n * sizeof(float));
    int total = s_keep_n + n;

    // Save last KEEP_MS samples as context for next chunk
    int keep = min(SAMPLE_RATE * KEEP_MS / 1000, total);
    memmove(s_keep, window + total - keep, keep * sizeof(float));
    s_keep_n = keep;

    struct whisper_full_params p =
whisper_full_default_params(WHISPER_SAMPLING_GREEDY);
    p.n_threads = 4; p.single_segment = true; p.no_timestamps = true;
    whisper_full(ctx, p, window, total);

    const char *text = whisper_full_get_segment_text(ctx, 0);
    if (text) on_partial_result(text);
}

```



Measured Throughput

On a Pixel 8 (Tensor G3) with whisper-base.en Q8_0, 4 threads, NEON enabled: 1 s of audio transcribes in ~290 ms (3.4× real-time). On a mid-range Snapdragon 778G device, expect ~480 ms (2.1× real-time). Always benchmark on your slowest supported device before committing to a model size.

Moonshine Deep Dive

● EXPERT

Architecture internals, Moonshine v2 sliding-window attention math, ONNX export, and the complete SDK integration flow.

Moonshine's architecture is best understood as the answer to a single question: how do you build an encoder-decoder ASR model that is simultaneously accurate, streaming-native, and small enough for a Raspberry Pi? Every architectural choice flows from that constraint.

Moonshine Architecture Comparison: v1 vs v2

PROPERTY	MOONSHINE V1	MOONSHINE V2
Encoder attention	Full self-attention (global)	Sliding-window (ergodic)
TTFT scaling	$O(N)$ — grows with utterance	$O(1)$ — bounded, independent of length
Complexity	$O(N^2)$ per layer	$O(N \times W)$ per layer
State caching	Encoder + decoder states	Window-local states only
Streaming native	Yes (via state caching)	Yes (architectural guarantee)
Accuracy vs Whisper Large v3	Comparable at 6× smaller	Matches at 6× smaller, faster

Moonshine Python SDK — Streaming Transcription

PYTHON

```
# pip install moonshine-voice
# Download models: python -m moonshine_voice.download --language en
# Quick start:     python -m moonshine_voice.mic_transcriber --language en

from moonshine_voice import (
    MicTranscriber,
    TranscriptEventListener,
    get_model_for_language,
)
import time

# Load model for language (downloads automatically on first run)
```

```

model_path, model_arch = get_model_for_language("en")

# Create microphone transcriber – handles VAD, segmentation, and transcription
mic_transcriber = MicTranscriber(
    model_path=model_path,
    model_arch=model_arch,          # or ModelArch.TINY_STREAMING for lower la-
    tency
    update_interval=0.5,            # How often to emit transcript updates (sec-
    onds)
    samplerate=16000,               # Optimal for Moonshine models
    blocksize=1024                  # Audio buffer size
)

# Event listener receives incremental and final transcripts
class MyListener(TranscriptEventListener):
    def on_line_started(self, event):
        print(f"\r ⏳ {event.line.text}", end="", flush=True)

    def on_line_completed(self, event):
        print(f"\n✓ {event.line.text}")

mic_transcriber.add_listener(MyListener())
mic_transcriber.start()

try:
    while True:
        time.sleep(0.1)
except KeyboardInterrupt:
    mic_transcriber.stop()
    mic_transcriber.close() # Release microphone and model resources

```

Downloading Moonshine ONNX Models for Android Deployment



Official Android SDK available.

Moonshine AI publishes `ai.moonshine:moonshine-voice` on Maven Central (v0.0.70+) with JNI bindings, `MicTranscriber` API, and pre-bundled models. For most Android projects, use the official SDK instead of the manual ONNX export below. The manual path remains useful for custom inference engines (e.g., sherpa-onnx) or when you need fine-grained control over session options.

The `useful-moonshine-onnx` Python package provides ONNX models pre-exported and downloadable via HuggingFace. Download once on a development machine; ship the `.onnx` files in your Android `assets/` folder. Total size (8-bit quantized): ~26 MB for `tiny`, ~59 MB for `base`. Unquantized float32 models are ~133 MB (tiny) and ~235 MB (base).

```
# pip install useful-moonshine-onnx huggingface-hub
import moonshine_onnx as moonshine

# Models are pre-exported to ONNX – downloaded automatically from HuggingFace
# For Android deployment, copy these .onnx files to app/src/main/assets/
from moonshine_onnx import MoonshineOnnxModel

model = MoonshineOnnxModel(model_name="moonshine/tiny")
# Model files (v1 separate, float32 – downloaded to ~/.cache/huggingface/):
#   encode.onnx           [30.1 MB]
#   uncached_decode.onnx [128 MB]
#   cached_decode.onnx   [120 MB – for KV-cached streaming]
#   preprocess.onnx       [6.8 MB]
# Total float32: ~285 MB (tiny) – use 8-bit quantized merged models (~26 MB)
# for mobile

# Transcribe audio (16kHz mono float32 numpy array)
# Returns a list of transcript strings
text = moonshine.transcribe("audio.wav", model)[0]
```

PYTHON

Android ONNX Runtime Integration (Kotlin)

```
implementation("com.microsoft.onnxruntime:onnxruntime-android:1.27.0")
```

KOTLIN (BUILD.GRADLE)

```
class MoonshineAndroid(private val assets: AssetManager) {

    private val env = OrtEnvironment.getEnvironment()
    private val opts = OrtSession.SessionOptions().apply {
        setIntraOpNumThreads(2)
        setInterOpNumThreads(1)
        addNnapi() // Delegate to Android Neural Networks API
    }

    private val preprocessor = env.createSession(
        assets.open("moonshine/preprocess.onnx").readBytes(), opts)
    private val encoder = env.createSession(
```

KOTLIN

```

        assets.open("moonshine/encode.onnx").readBytes(), opts)
private val decoder = env.createSession(
    assets.open("moonshine/uncached_decode.onnx").readBytes(), opts)

fun transcribe(pcm16k: FloatArray): String {
    // --- Preprocess: PCM → log-mel features ---
    val pcmTensor = OnnxTensor.createTensor(
        env, FloatBuffer.wrap(pcm16k),
        longArrayOf(1, pcm16k.size.toLong()))
    val ppResult = preprocessor.run(mapOf("audio" to pcmTensor))
    val features = ppResult[0] as OnnxTensor

    // --- Encode: mel features → hidden states ---
    val featLen = OnnxTensor.createTensor(
        env, LongBuffer.wrap(longArrayOf(features.info.getShape()[1])),
        longArrayOf(1))
    val encResult = encoder.run(
        mapOf("features" to features, "features_len" to featLen))
    val hidden = encResult[0] as OnnxTensor

    // --- Greedy decode ---
    val tokenIds = mutableListOf(1L) // BOS
    repeat(200) {
        val tokArr = tokenIds.toLongArray()
        val tokTensor = OnnxTensor.createTensor(
            env, LongBuffer.wrap(tokArr),
            longArrayOf(1, tokArr.size.toLong()))
        val seqLen = OnnxTensor.createTensor(
            env, LongBuffer.wrap(longArrayOf(tokArr.size.toLong())),
            longArrayOf(1))
        val decOut = decoder.run(
            mapOf("tokens" to tokTensor,
                "encoder_out" to hidden,
                "seq_len" to seqLen))
        val logits = (decOut[0] as OnnxTensor).floatBuffer
        val vocabSize = logits.capacity() / tokArr.size
        val offset = (tokArr.size - 1) * vocabSize
        var best = 0L; var bestVal = Float.NEGATIVE_INFINITY
        for (i in 0 until vocabSize) {
            val v = logits.get(offset + i)
            if (v > bestVal) { bestVal = v; best = i.toLong() }
        }
        if (best == 2L) return@repeat // EOS
        tokenIds.add(best)
    }
}

```

```

        return tokenizer.decode(tokenIds)
    }
}

```

Choosing Moonshine tiny vs base

MODEL	SIZE (ONNX)	WER (LIBRISPEECH CLEAN)	TTFT @ 3 S CLIP	USE CASE
moonshine/ tiny	~26 MB (int8)	~4.5%	~80 ms (Pixel 8)	Keyboard dictation, wearables
moonshine/ base	~59 MB (int8)	~3.4%	~145 ms (Pixel 8)	Voice assistant, meeting notes



NNAPI Acceleration

Calling `addNnapi()` in `SessionOptions` delegates supported ONNX operators to Android's Neural Networks API, which maps to GPU or NPU on devices that support it (most phones since Android 10). Fallback is automatic — if NNAPI rejects an op, ORT falls back to CPU. Always test NNAPI on your minimum-SDK device: old NNAPI drivers have known bugs on pre-Android-14 firmware.

Sherpa-ONNX Pipeline

● EXPERT

k2-fsa/sherpa-onnx: the production-grade cross-platform ASR framework supporting streaming CTC, Transducer, Moonshine, Whisper, and Silero VAD in a unified Android/iOS/Linux API.

Sherpa-ONNX (<https://github.com/k2-fsa/sherpa-onnx>) (k2-fsa) is the most complete open-source on-device ASR framework available in 2026. It wraps ONNX Runtime with purpose-built streaming pipelines for every major ASR architecture, provides pre-built Android/iOS AAR/XCFramework packages, and ships dozens of pre-exported models. For production deployments that need to support multiple model backends without custom JNI code, Sherpa-ONNX is the best option.

Sherpa-ONNX Android Setup

```
repositories {  
    maven { url = uri("https://jitpack.io") }  
    mavenCentral()  
}  
  
dependencies {  
    // Sherpa-ONNX AAR – bundles ONNX Runtime + all ASR pipelines  
    implementation("com.github.k2-fsa:sherpa-onnx:v1.13.4")  
}
```

GRADLE (BUILD.GRADLE.KTS)

```
import com.k2fsa.sherpa.onnx.*  
  
// Moonshine via Sherpa-ONNX (OfflineRecognizer – batch mode with v2 streaming  
models)  
class SherpaSTT(context: Context) {  
  
    private val recognizer: OfflineRecognizer  
  
    init {  
        val config = OfflineRecognizerConfig(  
            featConfig = FeatureConfig(sampleRate = 16000, featureDim = 80),  

```

KOTLIN

```

        modelConfig = OfflineModelConfig(
            // Moonshine v2: only 2 models needed (encoder + mergedDecoder)
            // v1 requires 4: preprocessor, encoder, uncachedDecoder,
cachedDecoder
            // Download from huggingface.co/csukuangfj/sherpa-onnx-moon-
shine-tiny-en-quantized-2026-02-27
            moonshine = OfflineMoonshineModelConfig(
                encoder          = copyAsset(context, "moonshine-
tiny/encoder_model.onnx"),
                mergedDecoder    = copyAsset(context, "moonshine-
tiny/decoder_model_merged.onnx")
            ),
            tokens = copyAsset(context, "moonshine-tiny/tokens.txt"),
            numThreads = 4,
            debug = false
        )
    )
    recognizer = OfflineRecognizer(config)
}

/**
 * Transcribe a complete utterance (batch mode).
 * For streaming, pair with Silero VAD to segment audio, then
 * call transcribe() on each detected speech segment.
 */
fun transcribe(samples: FloatArray): String {
    val stream = recognizer.createStream()
    stream.acceptWaveform(samples, sampleRate = 16000)
    recognizer.decode(stream)
    val result = recognizer.getResult(stream).text
    stream.release()
    return result
}
}

```

Supported Model Architectures

Sherpa-ONNX wraps ONNX Runtime to support every major ASR architecture via a unified pipeline API. The table below lists what ships out-of-the-box, with model sizes and Android-tested latency measured on a Pixel 7 (Tensor G2, 8 GB RAM).

ARCHITECTURE	PIPELINE	STREAMING?	PRE-EXPORTED MODELS	SMALLEST SIZE	ENGLISH WER (L
Moonshine	OfflineRecognizer	⚠ Batch (VAD-segmented)	tiny, small, medium (int8 & fp32)	~26 MB (tiny-int8)	~4.8 %
Zipformer (transducer)	OnlineRecognizer	✅ Yes	en, zh, multilingual	~12 MB (small-en)	~4.9 %
Paraformer	OfflineRecognizer	❌ Batch	zh, en, multilingual	~70 MB	~5.2 %
Whisper (ONNX export)	OfflineRecognizer	❌ Batch	tiny → large-v3-turbo	~39 MB (tiny)	~4.2 % (large-v3-
SenseVoice	OfflineRecognizer	❌ Batch	small	~230 MB	~4.8 %
NeMo CTC	OfflineRecognizer	❌ Batch	en, multilingual	~90 MB	~5.5 %



Model selection rule of thumb:

For real-time dictation on mobile, use

Moonshine tiny-int8

(lowest latency) or

Zipformer small-en

(smallest footprint). For maximum accuracy on short audio clips, use

Whisper large-v3-turbo

via OfflineRecognizer. Paraformer excels for Mandarin Chinese workloads.

Built-In VAD Integration

Sherpa-ONNX ships with **Silero VAD v5/v6** bundled as a first-class pipeline component (v5 and v6 share the same ONNX interface — 3 inputs, 2 outputs, single LSTM state tensor [2,1,128]). You can combine it with any offline model for automatic segmentation without external VAD libraries:

```
val vadConfig = VadModelConfig(
    sileroVad = SileroVadModelConfig(
        model = copyAsset(context, "silero_vad.onnx"),
        threshold = 0.5f,
        minSilenceDuration = 0.25f,
        minSpeechDuration = 0.25f,
```

KOTLIN

```

        windowSize = 512
    ),
    sampleRate = 16000,
    numThreads = 1
)

// Create VAD-aware offline recognizer
val config = OfflineRecognizerConfig(
    modelConfig = OfflineModelConfig(
        whisper = OfflineWhisperModelConfig(
            encoder = copyAsset(context, "whisper-tiny/encoder.int8.onnx"),
            decoder = copyAsset(context, "whisper-tiny/decoder.int8.onnx"),
            tokens = copyAsset(context, "whisper-tiny/tokens.txt")
        ),
        numThreads = 4
    )
)

val recognizer = OfflineRecognizer(config)

// VAD segments → offline decode for each segment
val vad = Vad(vadConfig)
vad.acceptWaveform(audioSamples)
while (!vad.isEmpty()) {
    val segment = vad.front()
    val stream = recognizer.createStream()
    stream.acceptWaveform(segment.samples, sampleRate = 16000)
    recognizer.decode(stream)
    val text = recognizer.getResult(stream).text
    stream.release()
    // Process text for each speech segment
    vad.pop()
}

```

Offline Model Management

Production apps need a strategy for distributing and updating models without inflating APK size. Sherpa-ONNX models are plain ONNX files, so standard asset delivery approaches work:

- **Assets at build time** — Ship models in `src/main/assets/`. Simplest for models < 50 MB. Increases APK/AAB size directly.

- **Play Asset Delivery (on-demand)** — Use an on-demand asset pack for models > 50 MB. Downloaded the first time the user activates STT, with a loading indicator. Keeps initial install fast.
- **App-managed download** — Host models on your CDN and download into `context.filesDir`. Enables A/B testing different model versions. Use HTTP `ETag` headers for cache invalidation.

Whichever strategy you pick, always verify the model checksum after download and before loading to prevent corrupted files from crashing ONNX Runtime.

Platform Support Matrix

PLATFORM	PACKAGE	MIN VERSION	GPU ACCELERATION
Android	AAR via JitPack / Maven Central	API 21 (Lollipop)	NNAPI (API 27+)
iOS	XCFramework / Swift Package	iOS 13	CoreML delegate
Linux / macOS	C++ shared library / Python wheel	glibc 2.17+ / macOS 11+	CUDA, CoreML
Windows	C++ DLL / Python wheel	Windows 10	DirectML, CUDA
WebAssembly	npm package	WASM SIMD support	—

Model Benchmarks & Comparison

● BEGINNER

Verified 2026 WER, RTFx, and on-device latency benchmarks across all major STT models — what actually matters for your deployment.

Choosing the right STT model requires comparing verified numbers, not marketing claims. This section presents 2026 benchmark data from the Hugging Face Open ASR Leaderboard and on-device measurements, covering WER accuracy, real-time factor, model size, and mobile latency across every major model family.

2026 Open ASR Leaderboard Snapshot

All WER values below are from the [Hugging Face Open ASR Leaderboard](https://huggingface.co/spaces/hf-audio/open_asr_leaderboard) (https://huggingface.co/spaces/hf-audio/open_asr_leaderboard) as of early 2026. RTF values for mobile are measured on a Pixel 7 (Tensor G2 SoC, 8GB RAM) running Android 14. Size values are float32 (unquantized); INT8 quantized models are typically 50–65% smaller (e.g., Moonshine Base: ~235 MB float32 → ~59 MB INT8, Moonshine Tiny: ~133 MB float32 → ~26 MB INT8). **Note:** WER Clean values here are LibriSpeech test-clean scores specifically. The 8-dataset Open ASR Leaderboard averages (used in [Section 09](#)) are higher — e.g., Whisper Large v3: 7.44%, Moonshine Base: 9.99%, Moonshine Tiny: 12.65%.

MODEL	WER CLEAN	WER NOISY	RTFX GPU	RTF MOBILE	SIZE (FLOAT32)
Canary Qwen 2.5B	5.63%	~8%	418×	✗ Too slow	~5GB
Whisper Large v3	2.01%	~3.9%	~145×	~RTF 8.0	~6.2GB
Whisper Large v3 Turbo	2.10%	~4.2%	~200×	~RTF 3.0	~3.2GB
Whisper Small	~3.0%	~7.3%	~269×	RTF ~0.6	~976MB
Whisper Tiny	~5.7%	~15.5%	~348×	RTF ~0.2	~156MB
Moonshine Base	~3.4%	~8.2%	~566×	RTF ~0.05	~235MB
Moonshine Tiny	~4.5%	~11.6%	~753×	RTF ~0.03	~133MB
Parakeet TDT 1.1B	~4.5%	~7%	~2391×	✗ Too slow	4.4GB
Vosk (small)	~12%	~18%	N/A	RTF ~0.1	40MB

Mobile Benchmark: Android On-Device (Pixel 7)

26MB Moonshine Tiny *Smallest viable model*

<50ms First Token *Moonshine v2 TTFT*

RTF 0.03 Moonshine Tiny *33× faster than real-time*

RTF 0.6 Whisper Small *Practical mobile limit*

4.5% Moonshine Tiny WER *LibriSpeech clean*

<1ms Silero VAD v6 *Per 32ms chunk*

Quantisation WER Impact — whisper-base.en

Every bit cut from model weights reduces accuracy slightly. The Q8_0 point is the optimal trade-off: half the size of FP16 with less than 0.2% absolute WER increase on LibriSpeech clean. Q4_0 is acceptable for low-memory devices; avoid Q2_K for dictation.

QUANTISATION	MODEL SIZE (BASE.EN)	WER (LIBRISPEECH CLEAN)	WER Δ VS FP16	DECODE TIM
FP32	290 MB	4.01%	—	Not recomm
FP16	145 MB	4.03%	+0.02%	~820 ms
Q8_0	75 MB	4.15%	+0.12%	~540 ms ✓ r
Q5_0	47 MB	4.37%	+0.34%	~460 ms
Q4_0	38 MB	4.72%	+0.69%	~390 ms
Q2_K	20 MB	7.10%	+3.07%	~310 ms — a

Battery & Thermal Budget Per Transcription

Measure with `adb shell dumpsys batterystats --reset` before your test and `dumpsys batterystats` after. Values below are approximate for a 10 s utterance on a Pixel 8 at 20°C ambient.

ENGINE	MAH / TRANSCRIPTION	PEAK CPU TEMP Δ	WALL TIME
Moonshine tiny (ONNX + NNAPI)	~0.8 mAh	+4 °C	~290 ms
Moonshine base (ONNX)	~1.4 mAh	+6 °C	~520 ms
whisper-base Q8 (whisper.cpp)	~2.1 mAh	+9 °C	~540 ms
whisper-small Q8 (whisper.cpp)	~5.8 mAh	+15 °C	~1,400 ms
Cloud (Deepgram round-trip)	~0.3 mAh (radio)	+1 °C	200–600 ms (LTE)



Benchmarking Methodology

Always benchmark on real hardware with the screen off and Wi-Fi disconnected to minimize background activity. Run at least 20 passes and discard the first 3 (JIT warm-up). Report p50, p95, and p99 latency — p99 reflects the user experience on slow or hot devices more accurately than mean.

VoiceKeyboard IME Integration ● INTERMEDIATE

Wiring a full STT pipeline inside an Android InputMethodService — the complete voice keyboard architecture.

A voice keyboard is an IME that supplements or replaces touch input with on-device speech recognition. It must manage the STT pipeline lifecycle within the constraints of `InputMethodService` — no Activity, no fragment, no standard lifecycle callbacks. This section covers the complete wiring pattern.

IME Voice Button Architecture

```
class VoiceKeyboardIME : InputMethodService() {
    private val imeScope = CoroutineScope(SupervisorJob() + Dispatchers.Main)
    private lateinit var sttPipeline: STTPipeline
    private var voiceActive = false

    override fun onCreate() {
        super.onCreate()
        sttPipeline = STTPipeline(
            context = this,
            onPartialTranscript = { partial →
                // Show partial in composing region
                currentInputConnection?.setComposingText(partial, 1)
            },
            onFinalTranscript = { final →
                // Commit finalized text and clear composing
                currentInputConnection?.commitText(final + " ", 1)
                currentInputConnection?.finishComposingText()
            }
        )
    }

    override fun onCreateInputView(): View {
        return ComposeView(this).apply {
            setContent {
                KeyboardLayout(

```

```

        onVoiceButtonPress = ::startVoice,
        onVoiceButtonRelease = ::stopVoice
    )
}
}

private fun startVoice() {
    if (!hasMicPermission(this)) { requestMicFromIME(); return }
    voiceActive = true
    sttPipeline.start()
}

private fun stopVoice() {
    voiceActive = false
    sttPipeline.stop()
}

override fun onFinishInput() {
    // Always stop recording when input field loses focus
    if (voiceActive) stopVoice()
    super.onFinishInput()
}

override fun onDestroy() {
    imeScope.cancel()
    sttPipeline.stop()
    super.onDestroy()
}

private fun requestMicFromIME() {
    // IMEs cannot request permissions directly – open settings
    val intent = Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS)
        .apply { data = Uri.fromParts("package", packageName, null) }
        .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
    startActivity(intent)
}
}

```



IMEs cannot request permissions — plan for this in advance

`InputMethodService` does not extend `Activity` and has no `requestPermissions()` method. If `RECORD_AUDIO` is not granted when the user presses the voice button, you must either launch an Activity (opens outside the IME) or show an in-keyboard message directing the user to Settings → App Permissions. Test this flow before shipping — it is always surprising to users.

Performance & Optimization

● EXPERT

Thread strategies, memory budget, battery profiling, end-to-end latency measurement, and the key optimizations ranked by impact.

Raw model accuracy means nothing if your app drains the battery in twenty minutes or triggers thermal throttling after two voice messages. This section quantifies every optimization that matters — thread strategy, memory budget, quantization gains, and battery profiling — ranked by real-world impact on mobile devices.

End-to-End Latency Budget

For a voice agent targeting sub-500ms response time, here is a typical breakdown on a Pixel 7 with Moonshine Tiny:

~20ms Audio Capture *AudioRecord frame latency*

~1ms Silero VAD *Per 32ms chunk*

~50ms Moonshine TTFT *First token latency*

~30ms Full decode *5-word utterance*

~5ms Post-process *Punct, capitalize*

~106ms Total P50 *End-to-end*

Optimization Ranking by Impact

1. **Use Silero VAD** — prevents the model from processing silence. Reduces total compute by 50–85% for typical voice interactions. Impact: massive.

2. **NNAPI/NPU acceleration** — on Snapdragon 888+ and Tensor G3+, ONNX Runtime NNAPI can 2–4× the inference speed with no model changes.
3. **INT8 quantization** — ~2× speed, ~4× size reduction, ~0.3% WER regression. Always worth it for mobile.
4. **Model selection** — Moonshine Tiny is 10× faster than Whisper Small with competitive English WER. Model choice is the single biggest lever.
5. **Thread pinning** — Use `Process.setThreadPriority(Process.THREAD_PRIORITY_URGENT_AUDIO)` on the inference thread.
6. **Pre-warming** — Run a dummy inference on app start to JIT-compile the model. Eliminates first-call latency spike.

Memory Budget

COMPONENT	MEMORY (FP16)	MEMORY (INT8)	NOTES
Moonshine Tiny model	~67MB	~26MB	Fits in any device
Moonshine Base model	~118MB	~59MB	Safe for 2GB+ RAM
Silero VAD	2MB	2MB	Always use same size
Audio ring buffer (5s)	~1.6MB	—	Float32 at 16kHz
ONNX Runtime runtime	~30MB	~30MB	Fixed overhead

Profiling with Android System Trace (Perfetto)

Use Perfetto to trace CPU scheduling, thread wakeups, and model inference timing in a single unified timeline. The trace reveals whether latency is in audio capture, feature extraction, encoder, or decoder.

```
# 1 – Download Perfetto recording script
curl -O
https://raw.githubusercontent.com/google/perfetto/master/tools/record_android_trace
chmod +x record_android_trace

# 2 – Record a 15-second trace (connect device via USB, USB debugging on)
python3 record_android_trace -o trace.perfetto-trace -t 15s -b 64mb
sched freq idle am wm binder_driver gfx view dalvik

# 3 – Open at https://ui.perfetto.dev/ (no upload – processed locally in
```

BASH

```
browser)
# Look for: thread "AudioRecord" → capture latency
#           thread "inference"   → encoder + decoder timing
#           "Choreographer#doFrame" → UI jank from heavy inference
```

```
// Add Perfetto atrace slices to your own code for custom timing
import android.os.Trace

fun runInference(pcm: FloatArray): String {
    Trace.beginSection("STT.encode")
    val hidden = encoder.run(pcm)
    Trace.endSection()

    Trace.beginSection("STT.decode")
    val tokens = decoder.run(hidden)
    Trace.endSection()

    return tokenizer.decode(tokens)
}
```

KOTLIN

Battery Optimization: Wake Locks & Doze Mode

Transcription wakes the CPU from low-power idle. Acquire a *partial wake lock* only for the duration of inference — never hold it indefinitely. On Android 12+, register for thermal status callbacks and downgrade model quality when the device is throttled.

```
class TranscriptionWakeLockGuard(private val context: Context) : AutoCloseable {
    {
        private val pm = context.getSystemService(Context.POWER_SERVICE) as
        PowerManager
        private val lock = pm.newWakeLock(
            PowerManager.PARTIAL_WAKE_LOCK,
            "MyApp::STT")

        init { lock.acquire(30_000L) } // 30 s max - safety timeout

        override fun close() { if (lock.isHeld) lock.release() }
    }

    // Usage - wake lock released automatically even on exception:
    TranscriptionWakeLockGuard(context).use {
        result = engine.transcribe(pcm)
    }
}
```

KOTLIN

```

}

// Thermal throttling – reduce model quality when device is hot
if (Build.VERSION.SDK_INT ≥ 31) {
    pm.addThermalStatusListener(mainExecutor) { status →
        when (status) {
            PowerManager.THERMAL_STATUS_MODERATE →
engine.setModel("moonshine/tiny")
            PowerManager.THERMAL_STATUS_SEVERE → engine.pauseSTT()
            PowerManager.THERMAL_STATUS_NONE →
engine.setModel("moonshine/base")
        }
    }
}
}

```

Audio Thread Priority Pinning

Set your audio capture thread to `THREAD_PRIORITY_AUDIO` and inference to `THREAD_PRIORITY_URGENT_AUDIO` to prevent the Linux scheduler from preempting them for background work. This eliminates the sporadic 50–200 ms stalls visible in Perfetto traces.

```

// In your audio capture coroutine dispatcher or HandlerThread:
val audioThread = HandlerThread("AudioCapture").apply {
    priority = Process.THREAD_PRIORITY_AUDIO    // -16 (highest for audio)
    start()
}

// Inference thread – slightly below audio capture
val inferThread = HandlerThread("STTInfer").apply {
    priority = Process.THREAD_PRIORITY_URGENT_AUDIO    // -19
    start()
}

// Run inside the thread once started:
fun pinCurrentThread() {
    Process.setThreadPriority(Process.THREAD_PRIORITY_AUDIO)
}

```



AudioSource.VOICE_RECOGNITION

Always use `MediaRecorder.AudioSource.VOICE_RECOGNITION` rather than `VOICE_COMMUNICATION` for STT recording. `VOICE_COMMUNICATION` applies AGC, noise suppression, and echo cancellation that corrupt the mel spectrogram and increase WER by 5–15%. `VOICE_RECOGNITION` delivers a clean signal.

Privacy & Security ● BEGINNER

On-device processing guarantees, audio data handling, permission transparency, and regulatory considerations for voice applications.

Voice applications carry significant privacy responsibility. Audio data is among the most sensitive personal data — it can reveal health conditions, relationships, financial status, and location. On-device processing is the only architecture that provides a meaningful privacy guarantee: audio never leaves the device.

Privacy Architecture Principles

- **Never persist raw audio** — Transcribe in memory, discard the float array immediately after inference. Raw audio stored to disk is a GDPR liability.
- **Show a clear visual indicator** — Display a prominent mic-active indicator whenever `AudioRecord.startRecording()` is called. Android 12+ overlays a system mic indicator; don't suppress it.
- **Stop recording immediately on focus loss** — In an IME, call `captureManager.stop()` in `onFinishInput()`. In a standard app, stop in `onPause()`.
- **Implement push-to-talk rather than always-on** — Require an explicit gesture to activate the microphone for any voice feature that isn't a dedicated voice assistant.

Android Privacy Indicators (API 31+)

```
// API 31+: Check if the microphone indicator is being shown by the system KOTLIN
val privacyManager = getSystemService(SensorPrivacyManager::class.java)

if (privacyManager != null &&
    privacyManager.supportsSensorToggle(SensorPrivacyManager.Sensors.MICROPHONE)) {
    val isMicBlocked = privacyManager.isSensorPrivacyEnabled(
        SensorPrivacyManager.Sensors.MICROPHONE
    )
}
```

```
if (isMicBlocked) {  
    // Mic is toggled off at system level – show explanation to user  
    showMicBlockedDialog()  
    return  
}  
}
```

GDPR / CCPA Considerations

- If you use cloud STT for any users in the EU/EEA, you must have a Data Processing Agreement with the provider and declare audio processing in your privacy policy.
- On-device processing with no transmission is categorically exempt from many cloud data processing obligations, but you must still disclose microphone access in your privacy policy.
- Transcripts are personal data if they can be attributed to an individual. Treat them as such.



Audio data breaches are catastrophic.

Unlike leaked passwords, leaked voice recordings cannot be changed. A single exposure can compromise biometric voiceprints, reveal medical conversations, and create legal liability under GDPR Article 9 (special category data). Design your architecture to ensure raw audio *never* touches persistent storage or network I/O unless explicitly required by the product.

On-Device vs Cloud vs Hybrid: Privacy Trade-Offs

PROPERTY	ON-DEVICE ONLY	CLOUD STT	HYBRID (ON-DEVICE + CLOUD FALLBACK)
Audio leaves device?	✗ Never	✓ Always	⚠ Only on fallback
GDPR processing basis needed?	No (no data transfer)	Yes (consent or legitimate interest)	Yes (for cloud path)
Data Processing Agreement?	Not required	Required with provider	Required with provider
Works offline?	✓ Full capability	✗ Requires connectivity	✓ Degraded accuracy
Server-side logging risk?	None	High (provider TOS may allow retention)	Medium
User perception	Highest trust	Lowest trust	Trust depends on transparency

Encryption & Data-at-Rest

Even on-device architectures must protect intermediate artifacts. Follow these guardrails:

- **Models** — Store ONNX/GGML model files in the app's internal storage (`context.filesDir`), which is encrypted at rest via Android file-based encryption (FBE) on all devices running Android 10+.
- **Transcripts** — If you persist transcripts locally (e.g., for user history), use `EncryptedSharedPreferences` or `EncryptedFile` from the Jetpack Security library.
- **Language model priors** — Personalisation data (n-gram frequencies, custom vocabulary) should be stored in the app sandbox and excluded from cloud backup via `android:allowBackup="false"` or backup rules.

- **Debug logs** — Never log audio samples, transcript fragments, or model outputs at any log level in release builds. Use a `BuildConfig.DEBUG` guard on all verbose logging.

Audit Logging & Transparency

Build trust by making microphone usage verifiable:

```
// Record mic access events for user transparency / privacy dashboard
data class MicAccessEvent(
    val timestamp: Long = System.currentTimeMillis(),
    val durationMs: Long,
    val source: String,          // "dictation", "voice_command", "voice_search"
    val audioSentToCloud: Boolean = false
)

// Expose a privacy dashboard where users can see every mic access
fun getRecentMicAccess(since: Long): List<MicAccessEvent> =
    dao.getAccessEventsSince(since)
```

On Android 12+ the system already surfaces mic-access notifications, but a dedicated privacy dashboard in your app settings demonstrates proactive compliance and builds user confidence.

Troubleshooting

● BEGINNER

The definitive diagnostic guide for every common STT failure mode – with root cause analysis and fix for each.

When your STT pipeline produces silence, gibberish, or repeated hallucinations, the root cause is almost always one of a handful of known failure modes. This diagnostic matrix maps every common symptom to its most likely cause and

confirmed fix — use it to resolve issues in minutes rather than hours.

Diagnostic Matrix

SYMPTOM	MOST LIKELY CAUSE	DIAGNOSTIC STEP	FIX
Empty transcript for all input	Wrong sample rate	Print <code>audioRecord.sampleRate</code>	Set 16000 explicitly in AudioRecord constructor
Transcript is garbled / nonsensical	Stereo audio fed to mono model	Check <code>CHANNEL_IN_MONO</code> vs <code>CHANNEL_IN_STEREO</code>	Always use <code>CHANNEL_IN_MONO</code>
Model hallucinations during silence	No VAD — model sees silent audio	Check if VAD is gating the pipeline	Add Silero VAD, set <code>no_speech_threshold=0.6</code> for Whisper
High latency (>2s for short utterance)	Whisper's 30s window or no quantization	Profile inference time separately from capture	Switch to Moonshine, add INT8 quantization, or shorten input segments
AudioRecord init fails	Buffer too small or missing permission	Check <code>AudioRecord.STATE_INITIALIZED</code>	Use <code>getMinBufferSize()</code> , verify <code>RECORD_AUDIO</code> permission
OOM crash on model load	Model too large for device RAM	Check logcat for <code>OutOfMemoryError</code>	Use INT8 quantized model, switch to Moonshine Tiny
ONNX session creation fails	Model file corrupt / wrong ONNX version	Check <code>OrtException</code> message in logcat	Verify ONNX opset compatibility (ORT 1.27 supports opset 7–26; ONNX 1.22 / opset 27 requires ORT 1.28+)
Silero VAD never triggers	State not reset between sessions	Check if <code>resetState()</code> is called	Call <code>vad.resetState()</code> before each new recording session
Words cut off at start of utterance	VAD onset too late / no look-behind	Compare VAD trigger time to actual speech onset	Prepend 300ms audio before VAD trigger to the utterance
Words cut off at end of utterance	Silence threshold too short	Log when pipeline cuts audio	Increase <code>SILENCE_THRESHOLD_FRAMES</code> to 17+ (500ms)

Whisper-Specific: Hallucination Patterns

```
# Detect and filter Whisper hallucinations programmatically
```

PYTHON

```

HALLUCINATION_PATTERNS = [
    "Thank you for watching",
    "Subscribe to",
    "Thanks for watching",
    "Subtitles by",
    "Translation by",
    "www.",
]

def is_hallucination(text: str, no_speech_prob: float,
                    avg_logprob: float) → bool:
    if no_speech_prob > 0.6: return True          # High confidence of silence
    if avg_logprob < -1.0: return True           # Low confidence output
    if len(set(text.split())) / (len(text.split()) + 1) < 0.5: return True #
    Repetitive
    for pattern in HALLUCINATION_PATTERNS:
        if pattern.lower() in text.lower(): return True
    return False

```



Most "STT bugs" are audio bugs.

Before investigating model issues, verify your audio pipeline: check the sample rate matches the model's expectation (16 kHz), confirm samples are Float32 in $[-1, 1]$, and ensure no silence padding is being misinterpreted. See [Section 5 — Audio Fundamentals](#) for the correct capture setup.

SECTION 28

QA Checklist

• BEGINNER

The complete pre-ship testing checklist for production STT systems – signal integrity, model quality, edge cases, and platform compliance.

Ship with confidence. This checklist distills the critical verification points from every section of this guide into a single pre-release testing protocol. Work through each category before your first beta and again before every production release.

Audio Capture

- ✓ AudioRecord initializes correctly on all target device API levels (24–36)
- ✓ Sample rate confirmed as 16 000 Hz via `audioRecord.sampleRate`
- ✓ Channel count confirmed as mono via `CHANNEL_IN_MONO`
- ✓ Buffer size is at or above `AudioRecord.getMinBufferSize()` result
- ✓ Float32 normalization: max amplitude of spoken test phrase is in range [0.05, 0.95]
- ✓ Recording stops correctly in `onFinishInput()` / `onPause()`
- ✓ AudioRecord resources are released in all exit paths (back button, app kill, rotation)

Voice Activity Detection

- ✓ Silero VAD triggers within 100 ms of speech onset in quiet environment
- ✓ Silero VAD does not trigger on HVAC / keyboard sounds at typical office levels
- ✓ VAD state is reset between recording sessions
- ✓ Look-behind buffer (≥ 300 ms) is prepended to captured utterances
- ✓ Silence threshold (≥ 400 ms) correctly segments consecutive utterances

- ✓ Short non-speech sounds (< 200 ms) are correctly ignored

STT Model Quality

- ✓ WER < 10 % on clean microphone input with model's target language
- ✓ WER < 20 % in 60 dB background-noise environment
- ✓ No hallucinations on 10 seconds of silence
- ✓ Proper names in `initial_prompt` are correctly spelled in output
- ✓ Numbers spoken as words are correctly normalized (e.g., "three" → "3")
- ✓ Post-processing preserves proper-nouns capitalization

Android Platform

- ✓ RECORD_AUDIO permission is requested before first use
- ✓ Denial is handled gracefully — UI shows explanation, no crash
- ✓ Android 12+ mic indicator is visible during active recording
- ✓ Microphone toggle (privacy sensor) is checked on API 31+
- ✓ No audio continues recording after app is backgrounded (if not a Service)

IME & Voice Keyboard

- ✓ Voice input stops when text field loses focus
- ✓ Partial transcript is shown in composing region during inference
- ✓ Final transcript is committed with a trailing space
- ✓ Switching from voice to touch input does not corrupt composing text

- ✓ IME handles `InputConnection` becoming null during transcript delivery

Performance

- ✓ End-to-end latency P50 < 300 ms on target device
- ✓ End-to-end latency P99 < 800 ms on target device
- ✓ No ANR — all inference on a background thread
- ✓ Memory stable after 100 consecutive utterances (no leak)
- ✓ CPU usage < 15 % average during active voice input
- ✓ Battery drain acceptable: < 3 % per 10 minutes of continuous voice input

Edge Cases

- ✓ Handle Bluetooth headset microphone source correctly
- ✓ Handle headphone-jack microphone (different audio routing)
- ✓ Correct behavior during an incoming phone call (audio session interruption)
- ✓ Correct behavior if user switches IME while voice is active
- ✓ Very long utterances (> 30 s) are handled or gracefully segmented
- ✓ Non-speech sounds (cough, laugh, ambient music) do not produce wrong text



Automate this checklist.

Convert each item into an instrumentation test or a manual QA script that can be re-run for every release. Track pass/fail rates per device and OS version in a spreadsheet or CI dashboard. See [Section 3 — Evaluation Metrics](#) for accuracy measurement methodology.

Open Source Index & References ● BEGINNER

Complete annotated index of all referenced libraries, models, frameworks, and documentation resources.

Every library, model, and tool referenced in this guide is listed here with its license, primary purpose, and the sections where it appears. Use this as your starting point when evaluating dependencies or checking license compatibility for your project.

Models & Inference Engines

PROJECT	TYPE	LICENSE	PRIMARY USE IN THIS GUIDE
whisper.cpp (https://github.com/ggml-org/whisper.cpp)	Inference Engine	MIT	Section 16 , Section 20 — Android on-device Whisper via JNI
moonshine (https://github.com/moonshine-ai/moonshine)	Model + SDK	MIT	Section 11 , Section 16 , Section 21 — Streaming on-device ASR
openai/whisper (https://github.com/openai/whisper)	Model	MIT	Section 10 — Reference architecture and Python API
faster-whisper (https://github.com/SYSTRAN/faster-whisper)	Inference Engine	MIT	Section 18 — Server-side high-throughput Whisper
sherpa-onnx (https://github.com/k2-fsa/sherpa-onnx)	Framework	Apache 2.0	Section 22 — Cross-platform ASR framework for Android
vosk-api (https://github.com/alphacep/vosk-api)	Framework	Apache 2.0	Section 09 — Offline ASR, small models
WhisperKit (https://github.com/argmaxinc/WhisperKit)	iOS SDK	MIT	Section 17 — On-device Whisper for iOS/Swift
onnxruntime (https://github.com/microsoft/onnxruntime)	Inference Engine	MIT	Section 06 , Section 11 , Section 16 — ONNX model execution on Android

VAD & Audio Processing

PROJECT	LICENSE	USE IN GUIDE
silero-vad (https://github.com/snakers4/silero-vad)	MIT	Section 06 — Primary VAD for all deployment

PROJECT	LICENSE	USE IN GUIDE
ad)		targets
RNNoise (https://jmvalin.ca/demo/rnnoise/)	BSD	Section 05 — Neural noise suppression
WebRTC APM (https://webrtc.googlesource.com/src/)	BSD	Section 05 — AEC, AGC, NS in one C++ library

Decoders & Language Models

Flashlight Text — Beam Search Decoder (<https://github.com/flashlight/text>)

github.com/flashlight/text

KenLM — Fast N-gram Language Model (<https://github.com/kpu/kenlm>)

github.com/kpu/kenlm

pyctcdecode — CTC Beam Search with KenLM (<https://github.com/kensho-technologies/pyctcdecode>)

github.com/kensho-technologies/pyctcdecode

Android Documentation

AudioRecord API Reference (<https://developer.android.com/reference/android/media/AudioRecord>)

developer.android.com/reference/android/media/AudioRecord

NoiseSuppressor — Android AudioEffect (<https://developer.android.com/reference/android/media/audiofx/NoiseSuppressor>)

developer.android.com/reference/android/media/audiofx/NoiseSuppressor

PowerManager — Thermal Status API (<https://developer.android.com/reference/android/os/PowerManager>)

developer.android.com/reference/android/os/PowerManager

Creating an Input Method — IME Development (<https://developer.android.com/develop/ui/views/touch-and-input/creating-input-method>)

developer.android.com/develop/ui/views/touch-and-input/creating-input-method

MediaPipe LLM Inference API — On-Device LLM (https://ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/android)

ai.google.dev/edge/mediapipe/solutions/genai/llm_inference/android

Papers

Radford et al. — Robust Speech Recognition via Large-Scale Weak Supervision (Whisper) (<https://arxiv.org/abs/2212.04356>)

arxiv.org/abs/2212.04356 — OpenAI, 2022

Jeffries et al. — Moonshine: Speech Recognition for Live Transcription and Voice Commands (<https://arxiv.org/abs/2410.15608>)

arxiv.org/abs/2410.15608 — Useful Sensors, 2024

Moonshine v2: Ergodic Streaming Encoder ASR for Latency-Critical Applications (<https://arxiv.org/abs/2602.12241>)

arxiv.org/abs/2602.12241 — Moonshine AI, February 2026

King et al. — Flavors of Moonshine: Tiny Specialized ASR Models for Edge Devices (<https://arxiv.org/abs/2509.02523>)

arxiv.org/abs/2509.02523 — September 2025

Leaderboards & Benchmarks

HuggingFace Open ASR Leaderboard (https://huggingface.co/spaces/hf-audio/open_asr_leaderboard)

huggingface.co/spaces/hf-audio/open_asr_leaderboard

Best Open-Source STT Models 2026 — Northflank (<https://northflank.com/blog/best-open-source-speech-to-text-stt-model-in-2026-benchmarks>)

northflank.com/blog/best-open-source-speech-to-text-stt-model-in-2026-benchmarks

Advanced Debugging Field Manual ● EXPERT

Comprehensive taxonomy of real-world STT bugs, GGML/ONNX runtime crashes, AudioRecord failures, cloud API disconnects, VAD edge cases, and speaker diarization anomalies — each with root cause, diagnostic command, and confirmed fix. Researched from upstream issue trackers, production incident reports, and field experience across Android, iOS, Python, and C++ stacks.

When standard troubleshooting fails, you need a field manual. This section catalogs the hard bugs — GGML runtime crashes, ONNX shape mismatches, AudioRecord edge cases, cloud API timeouts, and VAD false triggers — with the exact diagnostic command and verified fix for each.

How to Use this Manual



Triage order:

Start with the *symptom* column. Match the error message or observable failure. Read *Root Cause* first — fixing symptoms without understanding roots wastes hours. Apply the *Fix*, run the *Diagnostic*, verify with the provided assertion. Every fix below has been reproduced and confirmed; notes link upstream issue numbers where available.

whisper.cpp: GGML Runtime Errors

ERROR / SYMPTOM	ROOT CAUSE	DIAGNOSTIC	FIX
GGML_ASSERT(false && "MUL MAT-MAT not implemented")	Quantized model attempting GPU matrix multiply on unsupported GGML backend op	Check <code>model.type</code> ; reproduce with <code>-m model-q4_0.bin -l en</code>	Downgrade to Q5_K_M or use FP16; or add <code>-ng</code> flag to disable GPU
ggml_new_tensor_impl: not enough space in scratch memory	Static scratch buffer allocated at model-load time is too small for run-time tensors	Increase log verbosity: <code>whisper_ctx_params.use_gpu = false</code> and retry	Rebuild with larger <code>GGML_MAX_SCRATCH_BUFFERS</code> ; use a smaller Whisper variant (base vs large)

ERROR / SYMPTOM	ROOT CAUSE	DIAGNOSTIC	FIX
Phantom text / hallucination on silence (issue #1724)	Encoder always runs on 30s window; silent audio produces low-confidence output that decoder fills with training hallucinations	Feed 5s of pure silence; Whisper outputs "Thank you for watching", "Subtitles by..." etc.	Gate pipeline with Silero VAD; set <code>params.no_speech_threshold = 0.6f</code> and <code>params.logprob_threshold = -1.0f</code>
Memory leak — context grows unbounded after repeated <code>whisper_free()</code> / <code>whisper_init()</code> (issue #2467)	GGML backend buffer allocator holds OS memory pages after <code>free()</code> ; re-init does not reclaim them	Monitor RSS with <code>adb shell cat /proc/PID/status grep VmRSS</code> ; watch growth per reinit cycle	Keep single <code>whisper_context</code> * per process lifetime; call <code>whisper_reset_timings()</code> between sessions instead of <code>free+init</code>
GPU init failure: CUDA illegal memory access	Model tensor not aligned to GPU page boundary; common with custom quantized models	Run with <code>CUDA_LAUNCH_BLOCKING=1</code> to get synchronous stack trace	Use official quantized models from HuggingFace; or set <code>use_gpu = false</code> and file upstream issue with model provenance
Metal init failure on iPadOS 15/16 (see issue #1387)	<code>GGML_USE_METAL=1</code> requires Metal 3 features not available before A15 chip; iOS 16 with A14 crashes at shader compile	Check device chip: A14 = iPad Air 4th gen = affected; A15+ = safe	Detect chip at run-time via <code>ProcessInfo.processInfo.processorCount</code> proxy; fall back to CPU path with <code>cparams.use_gpu = false</code> for A14 and older
First-run latency spike: 10–40s before model starts (issue #1278)	Metal/Vulkan shader pipeline compilation occurs on first inference; shaders are then cached in device cache	Time second run — it should be 3–8× faster; check for <code>.metallib</code> cache at <code>~/Library/Caches</code>	Warm the shader cache at app install time by running one silent inference pass in the background
Token decoding produces garbled UTF-8 or missing characters	Transcription result joined incorrectly; whisper tokens include leading space bytes that encode word boundaries	Print raw token ids; compare against <code>whisper_token_to_str()</code> for each id	Use only <code>whisper_full_get_segment_text(ctx, i)</code> — never join token strings directly

ERROR / SYMPTOM	ROOT CAUSE	DIAGNOSTIC	FIX
Quantized model wrong re-sults with CUBLAS (issue #1661)	Q4_O GEMM kernels have rounding differences on CUBLAS compared to CPU path; affects models with Q4_O + GPU	Run same audio on CPU vs GPU and com-pare WER on known reference	Switch to Q5_K_M or Q8_O for GPU in-ference; or disable GPU for quantized models
Streaming example not pro-duc-ing output (#416)	VAD chunker logic in streaming demo uses frame_ms=30 but model expects 10ms frames; mismatch silently dis-cards chunks	Add <code>printf</code> around <code>vad_accept_wavefo</code> <code>rm</code> ; verify frame count	Ensure <code>frame_ms</code> matches your ring-buffer push interval; use official streaming example from main branch (not examples/stream from pre-v1.5)

```

/* Defensive whisper.cpp initialization with GPU fallback */
whisper_context_params cparams = whisper_context_default_params();
cparams.use_gpu      = true;
cparams.flash_attn = true;

whisper_context* ctx = whisper_init_from_file_with_params(model_path, cparams);

if (!ctx) {
    /* GPU init failed - common on CUDA 11 + Android Vulkan older drivers */
    WHISPER_LOG_WARN("GPU init failed, falling back to CPU-only");
    cparams.use_gpu      = false;
    cparams.flash_attn = false;
    ctx = whisper_init_from_file_with_params(model_path, cparams);
}
assert(ctx != nullptr && "Model load failed on both GPU and CPU paths");

/* Keep this context alive for the app lifetime - avoid free+reinit pattern */

```



Never call `whisper_free()` + `whisper_init()` in a loop.

The GGML allocator does not return heap to the OS between cycles. A single app session with 100 reinit cycles can exhaust 2–4 GB of virtual address space. Reuse one context; call `whisper_reset_timings(ctx)` and change parameters via `whisper_full_params` on each call instead.

Android AudioRecord: Initialization & Runtime Failures

ERROR / SYMPTOM	ROOT CAUSE	FIX
<code>AudioRecord.STATE_UNINITIALIZED</code> after constructor	Buffer size below hardware minimum, or sample rate unsupported on this SoC	Always call <code>AudioRecord.getMinBufferSize(16000, CHANNEL_IN_MONO, ENCODING_PCM_16BIT)</code> ; multiply by 2 for safety; use 16000 Hz (universally supported)
<code>RecordingException: start() returned false</code>	Another app holds the audio focus / microphone exclusive lock	Request <code>AudioFocus</code> before starting; listen to <code>OnAudioFocusChangeListener</code> for preemption events
Intermittent silence in captured audio	Buffer underrun: read loop slower than hardware fill rate	Increase buffer multiplier to 4×; move <code>audioRecord.read()</code> to a dedicated high-priority thread with <code>Process.setThreadPriority(THREAD_PRIORITY_URGENT_AUDIO)</code>
Empty transcript despite audio visible in waveform	PCM read returns <code>AudioRecord.ERROR_INVALID_OPERATION</code> silently; data array is zeros	Assert return value of <code>audioRecord.read() > 0</code> on every call; log <code>ERROR_DEAD_OBJECT</code> and recreate
Microphone returns stereo data even with <code>CHANNEL_IN_MONO</code>	Some Qualcomm audio HALs force stereo capture; interleaved samples cause 2× pitch shift in mel features	Always specify channel count in <code>AudioRecord</code> constructor; verify with <code>audioRecord.channelCount == 1</code>
<code>RECORD_AUDIO</code> permission granted but <code>AudioRecord</code> fails on Android 14	Android 14 introduced <code>READ_MEDIA_AUDIO</code> splitting; some builds also require <code>MODIFY_AUDIO_SETTINGS</code> for certain capture presets	Check <code>PermissionChecker.checkSelfPermission()</code> for all three; request <code>RECORD_AUDIO</code> at runtime with rationale dialog

ERROR / SYMPTOM	ROOT CAUSE	FIX
AudioRecord works in foreground, fails when app moves to background	Background microphone access requires a foreground service with <code>microphone</code> type (Android 10+)	Declare <code>foregroundServiceType="microphone"</code> in <code>AndroidManifest.xml</code> and call <code>startForeground()</code> with matching type

```

// Safe AudioRecord factory with full error checking
fun createAudioRecord(): AudioRecord {
    val minBuf = AudioRecord.getMinBufferSize(
        16_000, AudioFormat.CHANNEL_IN_MONO, AudioFormat.ENCODING_PCM_16BIT
    )
    require(minBuf != AudioRecord.ERROR && minBuf !=
AudioRecord.ERROR_BAD_VALUE) {
        "getMinBufferSize failed – device does not support 16 kHz mono PCM"
    }
    val bufSize = minBuf * 4 // 4× safety margin against underrun
    val recorder = AudioRecord(
        MediaRecorder.AudioSource.VOICE_RECOGNITION, // Best for STT – applies
beamforming
        16_000,
        AudioFormat.CHANNEL_IN_MONO,
        AudioFormat.ENCODING_PCM_16BIT,
        bufSize
    )
    check(recorder.state == AudioRecord.STATE_INITIALIZED) {
        "AudioRecord failed to initialize – check RECORD_AUDIO permission"
    }
    return recorder
}

// Read loop – always check return value
val buf = ShortArray(1600) // 100ms at 16 kHz
val read = recorder.read(buf, 0, buf.size)
if (read ≤ 0) {

```

KOTLIN

```

    Timber.e("ERROR AudioRecord.read returned - recreating recorder")
    recreateAudioRecord()
}

```

ONNX Runtime on Android: NNAPI & ABI Issues

ERROR	CAUSE	FIX
ONNX format model is not supported in this build	Using ORT mobile package (<code>onnxruntime-android</code>) with a full ONNX model; mobile builds require pre-converted <code>.ort</code> format	Convert model offline: <code>python -m onnxruntime.tools.convert_onnx_models_to_ort model.onnx</code> ; use resulting <code>.ort</code> file with ORT Mobile
ANEURALNETWORKS_BAID_DATA from NNAPI execution	Model contains ops not supported by target device's NNAPI version; Pixel 7a known to fail on certain FP16 graphs	Use NNAPI Model Usability Checker: <code>adb shell cmd media_transcoding run --checker</code> ; fall back to CPU EP for unsupported layers
unordered_map::at: key not found when USE_FP16 is set on Pixel 7a	Edge TPU path activated by NNAPI flags conflicts with ONNX node remapping on specific Tensor SoC firmware	Disable <code>NNAPI_FLAG_USE_FP16</code> ; add node-level FP16 cast ops manually before NNAPI subgraph boundary
App crashes with signal 11 on Android 15 devices	Prebuilt <code>libonnxruntime.so</code> compiled without 16 KB page-size alignment required by Android 15 (issue #21837)	Use ORT 1.20+ which adds <code>-Wl, -z,max-page-size=16384</code> ; check with <code>readelf -d libonnxruntime.so grep ALIGN</code>
Slow inference despite NNAPI enabled	NNAPI compilation cache miss on first N runs; subsequent runs use compiled binary cache	Pre-warm by running one dummy inference at startup; use <code>SessionOptions.setExecutionMode(SEQUENTIAL)</code> to avoid parallel graph fragmentation
No content provider: assets in React Native	ORT model loaded from <code>assets/</code> via incorrect URI scheme; asset path scheme not registered	Copy model to <code>filesDir</code> at app launch and load via absolute file path, not <code>android_asset://</code> URI



NNAPI debugging workflow:

Enable ORT verbose logging with `SessionOptions.setLogSeverityLevel(0)` and filter log-cat for `nnapi`. The output shows which ops were delegated to NNAPI and which fell back to CPU. Cross-reference with the NNAPI op support table at

developer.android.com/ndk/guides/neuralnetworks (<https://developer.android.com/ndk/guides/neuralnetworks>).

iOS: AVAudioEngine & WhisperKit Issues

ERROR / SYMPTOM	ROOT CAUSE	FIX
<code>AVAudioSession.setCategory</code> failed: Error Domain=NSOSStatusErrorDomain Code=-50	Audio session category changed while another app holds the session (phone call, Siri active)	Wrap <code>setCategory()</code> in <code>NotificationCenter</code> observer for <code>AVAudioSessionInterruptionNotification</code> ; retry after interruption ends
WhisperKit transcription returns empty on A14 devices (iPhone 12)	ANE (Apple Neural Engine) dispatch fails for models >500 MB on A14; falls back to CPU but with wrong buffer sizes	Use <code>WhisperKit.ModelVariant.bas</code> or <code>.small</code> on A14; reserve <code>.large-v3</code> for A17+
Audio capture stops after ~3 minutes in background	iOS 15+ enforces strict background audio limits; capture halts without active voip or audio background mode	Add <code>audio</code> to <code>UIBackgroundModes</code> in <code>Info.plist</code> ; keep <code>AVAudioSession.sharedInstance().isOtherAudioPlaying</code> active with a silent loopback
Microphone permission denied silently (no system dialog shown)	<code>NSMicrophoneUsageDescription</code> key missing from <code>Info.plist</code> ; iOS suppresses dialog and returns denied without alerting user	Add <code>NSMicrophoneUsageDescription</code> with descriptive string; check <code>AVCaptureDevice.authorizationStatus(for: .audio)</code> before starting engine
<code>AVAudioEngine: required condition is false: nullptr != mAVAudioNode</code>	Trying to attach/detach nodes while engine is running	Call <code>engine.stop()</code> , then modify node graph, then <code>engine.prepare()</code> and <code>try engine.start()</code>

```
// Handle AVAudioSession interruptions correctly
func setupInterruptionHandling() {
    NotificationCenter.default.addObserver(
        self,
        selector: #selector(handleAudioInterruption),
        name: AVAudioSession.interruptionNotification,
        object: nil
    )
}

@objc func handleAudioInterruption(_ notification: Notification) {
    guard let typeValue = notification.userInfo?
[AVAudioSessionInterruptionTypeKey] as? UInt,
        let type = AVAudioSession.InterruptionType(rawValue: typeValue) else
    { return }
    switch type {
    case .began:
        stopTranscription()
    case .ended:
        if let optValue = notification.userInfo?
[AVAudioSessionInterruptionOptionKey] as? UInt,
            AVAudioSession.InterruptionOptions(rawValue:
optValue).contains(.shouldResume) {
            restartTranscription() // Only resume if system says safe
        }
    }
}
```

```
@unknown default: break
}
}
```

Silero VAD: Edge Cases & False Triggers

SYMPTOM	CAUSE	FIX
VAD never detects speech despite normal microphone input	State tensors not reset between calls; LSTM hidden state accumulates to saturation	Call <code>model.reset_states()</code> (PyTorch) or <code>vad.resetState()</code> (ORT wrapper) before each recording session
VAD triggers on keyboard clicks, door slams (false positives)	VAD threshold too low; impulse noises have broadband energy that triggers onset detector	Increase <code>threshold</code> from 0.5 to 0.6–0.7; add 80ms minimum speech duration guard; skip segments shorter than 4 consecutive positive frames
Speech cut off at word boundaries (clipping)	VAD end offset too tight; silence detection ends segment 1 frame after speech probability drops	Pad 300ms before onset and 500ms after offset; if using <code>VadIterator</code> : set <code>min_silence_duration_ms=500</code>
ONNX export fails with "branches produce different tensor ranks" (#728)	Conditional branches in Silero v5 graph have incompatible output ranks; affects static ONNX type inference	Use pre-exported ONNX from HuggingFace (<code>snakers4/silero-vad</code> release assets); avoid exporting from source unless on torch≥2.2 with <code>dynamo=True</code>
Silero v6 fails on Raspberry Pi with torch-codec error (#735)	Silero v6 depends on <code>torch-codec</code> which has no AArch64 wheel	Use Silero v5 ONNX (still maintained); or install from source: <code>pip install torch-codec --no-binary torchcodec</code> after installing libav

Moonshine: Audio & Runtime Issues

SYMPTOM	CAUSE	FIX
Empty decoded text for <code>io.BytesIO</code> input (issue #52)	BytesIO pointer position at end after write; model receives zero-length audio array	Call <code>audio_buffer.seek(0)</code> before passing to <code>moonshine.transcribe(audio_buffer)</code>
Transcription produces repeated sequences: "hello hello hello..." (issue #83)	Greedy beam search collapses when audio confidence is low; repetition penalty not applied in base Moonshine config	Use streaming Moonshine with CTC forced alignment; or add post-processing: detect N-gram repetition with sliding window dedup

SYMPTOM	CAUSE	FIX
Unable to reproduce published WER re-sults with streaming model (#148)	Streaming models evaluated with 8ms frame shifts; re-implementing with different shift pro-duces stale state accumulation	Match exactly: 16 kHz input, 8ms frame shift, reset encoder state every 30s even in stream-ing mode
Android integration complexity for Moonshine	Moonshine now ships an official Android SDK (<code>ai.moonshine:moonshine-voice</code>) via Maven Central (v0.0.70+), with JNI bindings and <code>MicTranscriber</code> API. However, ONNX model export still requires manual conversion via the <code>useful-moonshine-onnx</code> package	Use the official <code>moonshine-voice</code> Maven package for Android (Section 21); or run Moonshine via sherpa-onnx Android bindings (Section 22) for unified pipeline management
<code>nn.py:545: UserWarning: softmax over axis 3 of shape [1,8,1,1]</code> (#56)	Attention mask with sequence length 1; harmless numerics warning, not an error	Filter with <code>warnings.filterwarnings("ignore", category=UserWarning, module="moonshine.nn")</code>

pyannote.audio: Speaker Diarization Failures

ERROR / SYMPTOM	CAUSE	FIX
<code>AttributeError: module 'torchaudio' has no attri-bute 'AudioMetaData'</code> (#1952)	<code>torchaudio</code> ≥ 2.1 removed <code>AudioMetaData</code> class; <code>pyannote.audio</code> 3.x depends on it	Pin <code>torchaudio==2.0.2</code> ; or up-grade to <code>pyannote.audio</code> 4.x which removed the dependency
<code>Could not load libtorch-codec</code> (#1940)	<code>pyannote</code> 4.x uses <code>torchcodec</code> for audio I/O; missing shared li-brary on some Linux distros	Install: <code>pip install torch-codec</code> ; on Alpine/musl: use <code>soundfile</code> backend instead via <code>PYANNOTE_AUDIO_BACKEND=soundfile</code>
<code>std(): degrees of freedom is ≤ 0</code> UserWarning (#1861)	Single-speaker segment pro-duces degenerate covariance matrix in clustering step; harm-less but noisy	Filter: <code>warnings.filterwarnings("ignore", "std")</code> ; or set <code>min_cluster_size=2</code> in the di-arization pipeline config
"Found speech but no valid (non-NaN) speaker embeddings" (#1961)	Audio segment shorter than speaker embedding model's minimum context (usually 0.5s)	Ensure all segments passed to embedding extraction are ≥ 0.5 s; pad with silence: <code>np.pad(segment, (0, max(0, 8000-len(segment))))</code>

ERROR / SYMPTOM	CAUSE	FIX
Pipeline unstable on audio >1 min (#1966)	LSTM state in segmentation model accumulates drift over long files; memory grows quadratically with segment count in clustering	Process in 60s chunks with 5s overlap; merge diarization annotations afterward with <code>pyannote.core.Annotation.support(collar=0.1)</code>
pyannote 4.0.3 uses 6× more VRAM than 3.3.2 (#1963)	New segmentation model (pyannote/segmentation-3.0) is significantly larger; VRAM now peaks at 9.5 GB vs 2.6 GB	Use <code>pyannote/speaker-diarization-3.1</code> (v3 pipeline) explicitly; or use <code>device="cpu"</code> for batch-offline processing where latency is acceptable



pyannote.audio requires a HuggingFace token

accepted through the model card. The pipeline will silently return no results if authentication fails — it does not raise an exception. Always verify: `from huggingface_hub import`

`whoami; whoami()` before loading the pipeline.

Cloud API: WebSocket & Streaming Issues

```
# Robust Deepgram/AssemblyAI WebSocket client with reconnect
import asyncio, websockets, json

MAX_RETRIES = 5
BACKOFF_BASE = 1.5

async def transcribe_with_retry(audio_stream):
    retries = 0
    while retries < MAX_RETRIES:
        try:
            async with websockets.connect(
                "wss://api.deepgram.com/v1/listen?
encoding=linear16&sample_rate=16000",
                extra_headers={"Authorization": f"Token {API_KEY}"},
                ping_interval=20, # Keep-alive: prevents 1011 idle disconnect
                ping_timeout=30,
                close_timeout=10,
            ) as ws:
                async for chunk in audio_stream:
                    await ws.send(chunk)
                await ws.send(json.dumps({"type": "CloseStream"}))
                return await collect_results(ws)
        except:
            retries += 1
            await asyncio.sleep(BACKOFF_BASE ** retries)
```

```

except websockets.exceptions.ConnectionClosedError as e:
    retries += 1
    if e.code == 1008:
        raise RuntimeError("Auth rejected – check API key") from e
    if e.code == 1011:
        # Server error / idle timeout – retry with backoff
        await asyncio.sleep(BACKOFF_BASE ** retries)
except websockets.exceptions.InvalidStatusCode as e:
    if e.status_code == 429:
        await asyncio.sleep(60) # Rate limit – wait 1 min
    else:
        raise
raise RuntimeError(f"WebSocket failed after {MAX_RETRIES} retries")

```

WEBSOCKET CODE	MEANING	ACTION
1008	Policy violation — bad auth token or malformed request	Check API key; verify request headers; do not retry automatically
1011	Server error or idle timeout (no audio for >10s)	Retry with exponential backoff; send audio more frequently or add keep-alive audio
1006	Abnormal close (network drop, DNS failure)	Retry with backoff; increase <code>ping_interval</code>
429 (HTTP)	Rate limit exceeded	Implement token bucket; back off 60s; check concurrency limits in API dashboard

Debugging Toolkit: Commands, Tools & Log Patterns

```

# — Android logcat filters for STT debugging — BASH

# Whisper.cpp GGML errors
adb logcat -s whisper.cpp:* GGML:E *:F

# NNAPI op failures
adb logcat -s NeuralNetworks:E NNAPI:E OrtInference:E

# AudioRecord issues
adb logcat -s AudioRecord:E AudioFlinger:W

# Memory pressure
adb shell dumsys meminfo com.yourapp.package | grep -E "TOTAL|Heap|Pss"

# GPU timeline (Vulkan)
adb shell atrace --async_start -a com.yourapp.package gfx vulkan

```

```
adb shell atrace --async_stop -o /tmp/trace.html -a com.yourapp.package gfx
vulkan
# Open trace in chrome://tracing

# — Python: profile whisper inference —————
import cProfile, pstats, io

pr = cProfile.Profile()
pr.enable()
result = whisper.transcribe(audio)
pr.disable()

s = io.StringIO()
pstats.Stats(pr, stream=s).sort_stats("cumulative").print_stats(20)
print(s.getvalue())

# — iOS Instruments CLI —————
xcrun xctrace record --template "Allocations" \
  --device "My iPhone" --launch -- ./YourApp.app/YourApp
```



Systematic regression isolation:

When a bug appears after updating a library, use `git bisect` on the dependency lock-file (Gradle `dependency:constraints`, Python `pip-compile`, Swift `Package.resolved`). Binary search the version range in 3–5 steps rather than testing every intermediate version. Always reproduce on a clean emulator image with no ADB port forwards to rule out state pollution.

Production Error Handling Patterns

```
// Sealed result type for STT pipeline errors – no silent failures KOTLIN
sealed interface TranscriptionResult {
    data class Success(val text: String, val confidence: Float) :
TranscriptionResult
    data class Error(val code: ErrorCode, val message: String) :
TranscriptionResult
    data object NoSpeech : TranscriptionResult // VAD gated – not an error
    data object Timeout : TranscriptionResult
}

enum class ErrorCode {
    AUDIO_INIT_FAILED, // AudioRecord STATE_UNINITIALIZED
```

```

    PERMISSION_DENIED,      // RECORD_AUDIO not granted
    MODEL_LOAD_FAILED,      // OOM or corrupt model file
    INFERENCE_CRASHED,      // GGML_ASSERT, NNAPI fatal, native exception
    AUDIO_SESSION_LOST,     // iOS interruption / Android
    AudioRecord.ERROR_DEAD_OBJECT
    NETWORK_UNAVAILABLE,    // For cloud API mode
    RATE_LIMITED,          // HTTP 429
}

fun handleTranscription(result: TranscriptionResult) {
    when (result) {
        is TranscriptionResult.Success → commitText(result.text)
        is TranscriptionResult.NoSpeech → { /* idle – do nothing */ }
        is TranscriptionResult.Timeout → showTimeoutHint()
        is TranscriptionResult.Error → when (result.code) {
            ErrorCode.MODEL_LOAD_FAILED → promptModelDownload()
            ErrorCode.PERMISSION_DENIED → openPermissionSettings()
            ErrorCode.RATE_LIMITED → scheduleRetryAfter(60_000)
            else → logAndShowGenericError(result)
        }
    }
}

```



Golden rule:

Never return a bare empty string from a transcription function. An empty string from a failure looks identical to an empty string from actual silence, making bugs invisible. Use typed result unions (sealed interfaces in Kotlin, Result in Swift, dataclasses in Python) that distinguish *no speech detected*, *error*, and *empty successful transcript* as three distinct states.

Backtrack & Self-Correction Engine

● EXPERT

Deep dive into detecting and resolving user self-corrections during dictation. Covers trigger-word detection, clause-scoped deletion, implicit restatement via LLM, and the layered architecture used by Wispr Flow, aawaaz, and tambourine-voice.

Humans don't dictate perfectly. They say "the meeting is at three pm actually four pm" and expect the system to output only "the meeting is at 4 PM." Backtrack detection is what separates a transcription tool from a dictation tool.

The Backtrack Problem

Self-correction in speech comes in two forms — **explicit** (triggered by marker phrases) and **implicit** (natural restatement without any trigger). A production-grade dictation system must handle both.

TYPE	EXAMPLE	EXPECTED OUTPUT	DETECTION METHOD
Explicit — single trigger	"send it on Monday actually Tuesday"	"send it on Tuesday"	Keyword matching
Explicit — phrase trigger	"I need the report scratch that I need the presentation"	"I need the presentation"	Keyword matching
Explicit — multiple corrections	"set alarm for 6 no wait 7 actually 7:30"	"set alarm for 7:30"	Iterative keyword scan
Implicit — semantic restatement	"the project costs about fifty thousand, it's around fifty K"	"the project costs around \$50K"	LLM semantic analysis
Implicit — mid-sentence restart	"we should probably, let's go ahead and approve it"	"let's go ahead and approve it"	LLM disfluency detection

Trigger Word Vocabulary

Across Wispr Flow, aawaaz, and tambourine-voice, the following trigger words are consistently recognized. Sorted by frequency-of-use from real dictation data:

TRIGGER	SCOPE	BEHAVIOR
actually	Previous clause or last noun/value	Replace preceding value with following correction
I mean	Previous clause	Replace preceding clause
scratch that	Previous sentence / previous dictation segment	Delete everything since last sentence boundary
no wait / wait	Previous clause	Replace preceding clause
actually no	Previous sentence	Delete entire previous sentence
never mind	Entire pending buffer	Discard all uncommitted text
or rather	Previous phrase	Replace preceding phrase
correction	Previous word or value	Replace last word with following word

Layered Backtrack Architecture

The most effective approach — used by Wispr Flow — is a **two-layer system** where a fast rule-based engine handles explicit triggers (sub-millisecond) and an LLM handles implicit restatement (50–400ms). This avoids LLM latency for the common case while still catching the hard cases.

```

/**
 * Two-layer backtrack engine.
 *
 * Layer 1: Rule-based trigger detection (fast, deterministic)
 *   - Scans for known trigger words
 *   - Deletes preceding clause/sentence/value
 *   - Runs in <1ms — no LLM needed
 *
 * Layer 2: LLM implicit restatement (slow, semantic)
 *   - Asks the LLM to detect and resolve implicit self-corrections
 *   - Only runs when Layer 1 finds no triggers AND confidence is low
 *   - 50-400ms latency depending on model size
 */
class LayeredBacktrackEngine(
    private val ruleBasedDetector: BacktrackDetector,
    private val llmFormatter: Qwen3DictationFormatter?,
) {
    data class BacktrackResult(
        val text: String,
        val backtrackDetected: Boolean,

```

KOTLIN

```

        val layer: Int, // 1 = rule-based, 2 = LLM
    )

    suspend fun process(
        rawTranscript: String,
        foregroundPackage: String? = null,
    ): BacktrackResult {
        // Layer 1: fast rule-based scan
        val layer1Result = ruleBasedDetector.resolveBacktracks(rawTranscript)
        if (layer1Result != rawTranscript) {
            return BacktrackResult(layer1Result, backtrackDetected = true,
layer = 1)
        }

        // Layer 2: LLM implicit detection (only if LLM is available)
        if (llmFormatter != null) {
            val llmResult = llmFormatter.format(rawTranscript, foregroundPack-
age)

            val changed = llmResult.length < rawTranscript.length * 0.85
            return BacktrackResult(llmResult, backtrackDetected = changed,
layer = 2)
        }

        return BacktrackResult(rawTranscript, backtrackDetected = false, layer
= 0)
    }
}

```

Clause Boundary Detection

When a trigger like "actually" is detected, the engine must determine *how much* text to delete. The answer depends on clause boundaries — punctuation, conjunctions, and natural pause points. The regex approach used by aawaaz and adapted in Section 14's

`BacktrackDetector` :

```

// Clause boundary pattern – splits text at natural correction points
(?:^|[\.\!\?]\s+|,\s+|\s+(?:and|but|or|so|then|because)\s+)

```

REGEX

```

// Example: "send the email to John, actually to Sarah"
//   Clauses: ["send the email to John", "actually to Sarah"]

```

```
// Trigger "actually" found in clause[1]
// Delete clause[0]'s last value → replace with clause[1]'s value
// Result: "send the email to Sarah"
```



Deletion scope matters.

"Scratch that" should delete the previous sentence, not the previous word. "Actually" should replace the previous value, not the previous sentence. Getting scope wrong is worse than not detecting the backtrack at all — users lose content they intended to keep.

Wispr Flow's Approach

Wispr Flow's proprietary backtrack engine is their most distinctive feature. Based on public documentation and user reports:

- **Explicit triggers** — "actually" and "scratch that" are confirmed
- **Implicit detection** — Wispr detects natural restatement *without* any trigger word, reportedly using a fine-tuned LLM
- **Scope intelligence** — "actually" replaces only the preceding value; "scratch that" deletes the entire previous dictation segment
- **No false positives** — "I actually think this is good" does NOT trigger deletion because "actually" is used as an adverb, not a correction marker



The false-positive problem.

Naive keyword matching on "actually" fires on sentences like "I actually agree with you." Wispr Flow reportedly avoids this by checking whether the word following "actually" is semantically a replacement for what precedes it. This requires either (a) a POS-tagger to detect adverb vs conjunction usage, or (b) an LLM prompt that explicitly handles both cases.

Personal Dictionary & Auto-Learning

● INTERMEDIATE

Architecture for user-managed and auto-learned personal dictionaries that improve recognition of names, jargon, and domain-specific vocabulary. Covers manual entries, contacts import, CSV bulk import, auto-learning from corrections, and fuzzy matching for voice shortcuts.

Every user has words that no general ASR model knows — their colleagues' names, product names, technical acronyms, and slang. A personal dictionary bridges this gap by post-processing ASR output to replace misrecognised words with the user's preferred spellings.

Dictionary Entry Types

TYPE	SOURCE	EXAMPLE	PRIORITY
Manual entry	User types in settings	"kubernetes" → "Kubernetes"	Highest — user explicitly defined
Contacts import	Device contacts provider	"john smith" → "John Smith"	High — real names with known spelling
CSV bulk import	User-uploaded file	Medical/legal terminology lists	Medium — batch import, less trusted
Auto-learned	User corrections over time	"kubernetes" → "Kubernetes" (after 3 corrections)	Low — requires confirmation threshold

Dictionary Architecture

```
/**                                                                 KOTLIN
 * Personal dictionary with priority-based replacement.
 * Entries are sorted by priority (manual > contacts > CSV > auto-learned)
 * and by length (longer patterns match first to prevent partial matches).
 *
 * Storage: Room database with indexed spoken_form column for fast lookup.
 */
data class DictionaryEntry(
    val spokenForm: String,          // What the user says (lowercase)
    val writtenForm: String,        // What gets inserted
```

```

    val source: EntrySource,           // MANUAL, CONTACTS, CSV, AUTO_LEARNED
    val confirmations: Int = 0,        // For auto-learned: times user confirmed
    this correction
    val lastUsed: Long = 0L,           // Epoch millis – for LRU eviction
}

enum class EntrySource(
    val priority: Int,
) {
    MANUAL(100),
    CONTACTS(80),
    CSV(60),
    AUTO_LEARNED(40),
}

class PersonalDictionary(
    private val entries: List<DictionaryEntry>,
) {
    // Pre-sorted: highest priority first, then longest spoken form first
    private val sortedEntries = entries
        .filter { it.source != EntrySource.AUTO_LEARNED || it.confirmations >=
3 }
        .sortedWith(
            compareByDescending<DictionaryEntry> { it.source.priority }
                .thenByDescending { it.spokenForm.length }
        )

    private val patterns: List<Pair<Regex, String>> = sortedEntries.map { entry
→
        Regex("\\b${Regex.escape(entry.spokenForm)}\\b",
RegexOption.IGNORE_CASE) to entry.writtenForm
    }

    fun apply(text: String): String {
        var result = text
        for ((pattern, replacement) in patterns) {
            result = pattern.replace(result, replacement)
        }
        return result
    }
}

```

Auto-Learning from User Corrections

Aawaaz pioneered **auto-learning**: when a user manually corrects a transcription (e.g., replaces "kubernetes" with "Kubernetes"), the system records the correction. After the same correction is observed N times (default: 3), it becomes a dictionary entry. This creates a feedback loop where the system improves with use.

```
/**                                                                 KOTLIN
 * Tracks user corrections to auto-learn dictionary entries.
 * Called by the IME when the user edits committed text.
 *
 * Algorithm:
 * 1. Detect text replacement (composing region changed)
 * 2. If old text was from ASR and new text is user-typed:
 *    record (old → new) as a correction candidate
 * 3. When same (old → new) seen  $N$  times → promote to dictionary
 */
class CorrectionTracker(
    private val dao: DictionaryDao,
    private val threshold: Int = 3,
) {
    suspend fun recordCorrection(original: String, corrected: String) {
        if (original.equals(corrected, ignoreCase = true)) return
        if (original.isBlank() || corrected.isBlank()) return

        val existing = dao.findBySpokenForm(original.lowercase())
        if (existing != null) {
            dao.incrementConfirmations(existing.id)
        } else {
            dao.insert(
                DictionaryEntry(
                    spokenForm = original.lowercase(),
                    writtenForm = corrected,
                    source = EntrySource.AUTO_LEARNED,
                    confirmations = 1,
                    lastUsed = System.currentTimeMillis(),
                )
            )
        }
    }
}
```

```

    }
}
}

```

Contacts Import (Android)

Aawaaz imports device contacts as dictionary entries, solving the #1 pain point in dictation: people's names. On Android, this requires `READ_CONTACTS` permission and a one-time sync.

```

/**                                                                 KOTLIN
 * Import device contacts as personal dictionary entries.
 * Each contact's display name becomes a (spoken → written) mapping.
 *
 * Security: Only reads display names – no phone numbers or email addresses.
 */
suspend fun importContacts(
    contentResolver: ContentResolver,
    dao: DictionaryDao,
) = withContext(Dispatchers.IO) {
    val cursor = contentResolver.query(
        ContactsContract.Contacts.CONTENT_URI,
        arrayOf(ContactsContract.Contacts.DISPLAY_NAME_PRIMARY),
        null, null, null,
    ) ?: return@withContext

    cursor.use {
        val nameIdx = it.getColumnIndexOrThrow(
            ContactsContract.Contacts.DISPLAY_NAME_PRIMARY
        )
        while (it.moveToNext()) {
            val name = it.getString(nameIdx) ?: continue
            if (name.length < 2) continue
            dao.insertIfAbsent(
                DictionaryEntry(
                    spokenForm = name.lowercase(),
                    writtenForm = name,
                    source = EntrySource.CONTACTS,
                )
            )
        }
    }
}

```

```
}  
  }  
}
```

Voice Shortcuts (Fuzzy Matching)

Aawaaz extends the personal dictionary concept into **voice shortcuts** — short spoken phrases that expand into longer text. Unlike dictionary entries (which correct single words), shortcuts can map a phrase to a paragraph or template. Fuzzy matching with Levenshtein distance handles ASR variations.

SPOKEN SHORTCUT	EXPANSION	USE CASE
"my address"	"123 Main Street, Springfield, IL 62701"	Auto-fill address
"sign off"	"Best regards,\nJohn Smith\nSenior Engineer"	Email signature
"standup"	"Yesterday I worked on...\nToday I'm working on...\nBlockers:"	Meeting template



LLM Dictionary Prompt.

The personal dictionary serves double duty: it feeds the pre-LLM replacement engine (fast, regex-based) AND the LLM prompt's dictionary section (Section 15's `DictationPromptBuilder`). The pre-LLM pass handles exact matches; the LLM catches context-dependent variations. Use both.

Voice Command Mode Architecture

● EXPERT

Design and implementation of a voice-activated command system for text manipulation, web search, and app control. Covers Wispr Flow's "Command Mode", aawaaz's command vocabulary, activation triggers, highlighted-text transformation, and integration with the IME lifecycle.

Beyond dictation, voice can *control* the editing experience. Command Mode lets users say "rewrite this more formally" or "translate to Spanish" and have the selected text transformed in-place. Wispr Flow made this their flagship feature.

Command Mode vs Dictation Mode

ASPECT	DICTATION MODE	COMMAND MODE
Activation	Press mic button	Long-press mic / speak "command" / hardware shortcut
Input	Voice → text insertion at cursor	Voice → instruction to transform selected text
Context	Cursor position + surrounding text	Selected / highlighted text (or none for web search)
LLM Usage	Optional (formatting cleanup)	Required (transformation engine)
Latency Budget	50–400ms (real-time feel)	1–5s acceptable (user expects processing)

Wispr Flow Command Vocabulary

Wispr Flow supports free-form commands (any instruction the user speaks) plus recognized shortcuts for common operations:

COMMAND	WITH SELECTED TEXT	WITHOUT SELECTED TEXT
"rewrite this more formally"	Rewrites selection in formal tone	N/A
"translate to Spanish"	Translates selection	N/A
"summarize"	Condenses selection	N/A
"expand on this"	Elaborates on selection	N/A
"make it shorter"	Condenses selection	N/A
"fix grammar"	Grammar-corrects selection	N/A
"search for X"	N/A	Opens Perplexity web search
"press enter"	Inserts Enter keypress	Inserts Enter keypress
Any free-form instruction	Transforms using LLM	Composes new text per instruction

Architecture

```

/**                                                                 KOTLIN
 * Voice Command Mode engine for the IME.
 *
 * Flow:
 * 1. User activates command mode (long-press or trigger word)
 * 2. ASR captures the spoken instruction
 * 3. Engine determines context: is text selected?
 * 4. If selected: send (selectedText + instruction) to LLM for transforma-
tion
 * 5. If not selected + search intent: open browser search
 * 6. If not selected + generation intent: generate text at cursor
 * 7. Replace selected text with LLM output OR insert at cursor
 *
 * Wispr Flow limits: 1-1000 words for selected text transformation.
 */
sealed interface CommandResult {
    data class TextTransformation(
        val original: String,
        val transformed: String,
    ) : CommandResult

    data class WebSearch(
        val query: String,
    ) : CommandResult

    data class TextGeneration(
        val generated: String,

```

```

    ) : CommandResult

    data class KeyAction(
        val keyCode: Int,
    ) : CommandResult
}

class VoiceCommandEngine(
    private val llm: Qwen3DictationFormatter,
) {
    private val searchPattern = Regex(
        """"^(?:search\s+(?:for\s+)?|look\s+up\s+|google\s+)(.+)$""",
        RegexOption.IGNORE_CASE,
    )
    private val enterPattern = Regex(
        """"^press\s+enter$""",
        RegexOption.IGNORE_CASE,
    )

    suspend fun execute(
        spokenCommand: String,
        selectedText: String?,
    ): CommandResult {
        // Check for key actions first
        if (enterPattern.matches(spokenCommand)) {
            return CommandResult.KeyAction(android.view.KeyEvent.KEYCODE_ENTER)
        }

        // Check for web search intent
        val searchMatch = searchPattern.find(spokenCommand)
        if (searchMatch != null && selectedText.isNullOrEmpty()) {
            return CommandResult.WebSearch(searchMatch.groupValues[1])
        }

        // Text transformation or generation
        return if (!selectedText.isNullOrEmpty()) {
            val prompt = """"Transform the following text according to the in-
instruction.
Instruction: $spokenCommand
Text: $selectedText
Return ONLY the transformed text.""
            val result = llm.format(prompt, foregroundPackage = null)
            CommandResult.TextTransformation(selectedText, result)
        } else {
            val prompt = """"Generate text based on this instruction:

```

```

$spokenCommand
Return ONLY the generated text."""
    val result = llm.format(prompt, foregroundPackage = null)
    CommandResult.TextGeneration(result)
}
}
}

```

IME Integration

On Android, the IME accesses selected text via `InputConnection`:

```

// Inside InputMethodService KOTLIN
fun handleCommandResult(result: CommandResult) {
    val ic = currentInputConnection ?: return
    when (result) {
        is CommandResult.TextTransformation → {
            // Replace selected text with transformed text
            ic.commitText(result.transformed, 1)
        }
        is CommandResult.TextGeneration → {
            ic.commitText(result.generated, 1)
        }
        is CommandResult.WebSearch → {
            val intent = Intent(Intent.ACTION_WEB_SEARCH).apply {
                putExtra(SearchManager.QUERY, result.query)
                addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            }
            startActivity(intent)
        }
        is CommandResult.KeyAction → {
            ic.sendKeyEvent(KeyEvent(KeyEvent.ACTION_DOWN, result.keyCode))
            ic.sendKeyEvent(KeyEvent(KeyEvent.ACTION_UP, result.keyCode))
        }
    }
}

// Getting selected text for command context
fun getSelectedText(): String? {
    val ic = currentInputConnection ?: return null
    return ic.getSelectedText(0)?.toString()?.takeIf { it.isNotBlank() }
}

```



Security: Validate LLM output before committing.

Never commit raw LLM output that could contain prompt injection artifacts, HTML/JS, or control characters. Sanitize the output string and limit to the expected character set before calling `commitText()`.

CITATION

How to Cite This Guide

If you reference this guide in academic work, documentation, or project READMEs, please use one of the formats below. Both are consistent with the output generated by GitHub's "Cite this repository" button and the `cffconvert` (<https://github.com/citation-file-format/cffconvert>) tool from the repository's `CITATION.cff` file.

APA (7th Edition)

```
Made in Jurgistan. (2026). Mobile Speech-to-Text Engineering Guide (Version 2026.2.0) [Computer software]. https://Made-in-Jurgistan.github.io/mobile-stt-engineering-guide/
```

BibTeX

```
@software{Made_in_Jurgistan_Mobile_Speech_2026,  
  author = {Made in Jurgistan},  
  month = {7},  
  title = {{Mobile Speech-to-Text Engineering Guide}},  
  url = {https://Made-in-Jurgistan.github.io/mobile-stt-engineering-guide/},  
  version = {2026.2.0},  
  year = {2026}  
}
```



GitHub "Cite this repository" button

A `CITATION.cff` file at the repository root also enables GitHub's built-in "Cite this repository" button (right sidebar on the repo page), which auto-generates APA and BibTeX citations from the same metadata. This button appears on `github.com` only — it does not render on the published GitHub Pages site, which is why the citation blocks above are included here.

Mobile Speech-to-Text Engineering Guide

2026 Complete Edition — 33 Sections · Kotlin · Python · C++ · ONNX · Moonshine v2 · whisper.cpp · Silero VAD v6

Version 2026.2.0 · Published 2026-07-23 · Updated 2026-07-23 (ISO 8601)

Made in Jurgistan — Complete Edition

Research: github.com/moonshine-ai/moonshine · github.com/ggml-org/whisper.cpp

github.com/snakers4/silero-vad · github.com/k2-fsa/sherpa-onnx

arxiv.org/abs/2602.12241 (Moonshine v2, Feb 2026)

Licensed under CC BY-SA 4.0 — Free to share and adapt with attribution

Accessibility: Conforms to WCAG 2.2 AA. Includes keyboard navigation, skip links, focus-visible outlines, reduced-motion support, forced-colors support, and semantic HTML.

Report issues to: github.com/Made-in-Jurgistan/mobile-stt-engineering-guide/issues

↑ Return to top